QUANTUMBRIDGE

Official Research Log

Quantum Chip Emulator Platform — Phase 1: Foundations

Author: Darknight (Mirr) | BTech AI/ML
Affiliation: Lachoo Memorial College, Jodhpur ()

Research Start Date: July 2026

About this document

This is the official QuantumBridge research log. Every experiment, setup step, observation, and decision made during this project is recorded here with full context — what was done, why it was done, how it works, and what was observed. This document will grow entry by entry throughout the project and will form the basis of any future research publication or report.

It is written so that any researcher reading it cold can understand every decision made.

Project summary

QuantumBridge is a machine learning-based quantum chip emulator platform. It collects experimental data from real quantum hardware (IBM, Google), trains ML models to learn each chip's unique noise patterns and behavior, and packages those models as downloadable software — enabling faithful offline quantum simulation on consumer laptops.

Think: Ollama, but for quantum computers instead of LLMs.

Target market: Indian engineering colleges, small research teams, individual developers.

How to read this log

Each entry covers one experiment or setup milestone. Every entry contains:

1. What was done | 2. Why it was necessary | 3. How it works (theory) | 4. Results observed

5. What it means for the project | 6. Next step triggered by this entry

Research Log — Table of Contents

Entry	Title	Date	Status
Entry 001	Environment Setup — Python 3.11 + Qiskit Installation	July 11, 2026	Complete
Entry 002	Experiment 1 — Bell State Circuit on Aer Simulator	July 11, 2026	Complete
Entry 003	Experiment 2 — Bell State on Real IBM Hardware (ibm_fez)	July 13, 2026	Complete
Entry 004	Noise Dataset Collection — 20-Run Study (ibm_fez)	July 14, 2026	Complete
Entry 005	Multi-Circuit Dataset & Noise Scaling (80 runs)	July 14, 2026	Complete
Entry 006	CNOT-Scaling Test — 4-5 Qubits, Cross-Chip Finding	July 15, 2026	Complete
Entry 007	First Predictive Model — R2=0.747 (60 samples)	July 15, 2026	Complete
Entry 008	Single-Qubit Gate Cost & Cross-Session Drift	July 16, 2026	Complete
Entry 009	3-Feature Model — Confound Drops R2 (105 samples)	July 16, 2026	Complete
Entry 010	Deconfounded Gate-Cost Measurement (165+ samples)	July 17, 2026	Complete
Entry 011	3-Feature Retrain Reveals CNOT-Backend Collinearity	July 17, 2026	Complete
Entry 012	Partial Factorial Fix, Interrupted by Quota Limit	July 18, 2026	Complete
Entry 013	Complete Factorial Design (post quota reset)	Next session	Up next

This table will be updated after every new entry. Entries marked 'Upcoming' will be filled in as the research progresses. Each entry corresponds to a real working session on the project.

Research Log — Updated Index

Addendum covering Entries 013–016. The original Table of Contents (page 2) lists Entries 001–012; this page extends it rather than replacing it, to avoid disturbing the original document.

Entry	Title	Date	Status
Entry 013	Offline Topology Discovery & SWAP-Routing Fix	Jul 29–30, 2026	Complete
Entry 014	Circuit-Level Validation of Emulator v3	Jul 30, 2026	Complete
Entry 015	Why Per-Edge SWAP Cost Made Things Worse	Aug 1, 2026	Complete
Entry 016	Optimization Level & Real Gate Count — Isolating the Compounding Gap	Aug 1, 2026	Complete

Summary of this arc: Entry 013 fixed a three-layer connectivity bug in Emulator v3's SWAP router (wrong topology source, a genuine Qiskit-internal API disagreement, and a calibration-completeness gap). Entry 014 validated the fixed router against a realistic offline noise simulation, finding strong accuracy on direct-connection circuits but a meaningful gap on multi-hop routed circuits. Entry 015 attempted a per-edge SWAP-cost fix, which unexpectedly widened that gap, and traced the real cause to gate-count agreement without cost-compounding agreement. Entry 016 isolates the remaining gap to v3's multiplicative error-compounding formula itself, independent of both routing path length and real transpiled gate count.

ENTRY 001 Environment Setup — July 11, 2026 — Status: COMPLETE

Entry 001 — Environment Setup
Python 3.11 + Qiskit Installation on MacBook (Apple Silicon)

1. What Was Done
The complete development environment for QuantumBridge was set up on a MacBook (Apple Silicon, ARM64 architecture) running macOS. This included installing a modern Python runtime, the Homebrew package manager, the Qiskit quantum computing framework, and the Qiskit Aer local simulator.

Component	Version Installed	Purpose
Python	3.9.6 (pre-existing)	Base runtime — present on system
Python	3.11.15 (via Homebrew)	Primary runtime for project — replaces 3.9
Homebrew	Latest (July 2026)	macOS package manager for installing Python 3.11
Qiskit	2.5.0	Core quantum computing framework
Qiskit Aer	0.17.2	Local quantum circuit simulator
Qiskit IBM Runtime	0.43.1	Interface to real IBM quantum hardware
Matplotlib	3.11.0	Plotting and visualizing quantum results

2. Why This Step Was Necessary
Before any quantum experiment can be run, a working software environment must exist. This is the foundation on which the entire QuantumBridge project is built. Without Qiskit, there is no way to programmatically build quantum circuits, simulate them locally, or submit them to real IBM quantum hardware.

Python 3.9.6 was present on the system but was flagged as deprecated by Qiskit 2.1.0 — support for Python 3.9 is scheduled to end in Qiskit 2.3.0. Upgrading to Python 3.11 ensures the project runs on a supported, stable foundation for the full duration of Month 1 through Month 12 of the roadmap.

Why Python 3.11 specifically?

Python 3.11 offers 10-60% performance improvements over 3.9 for compute-heavy tasks.

Qiskit 2.x officially supports Python 3.9, 3.10, 3.11, and 3.12 — 3.11 is the stable middle choice.

All major ML libraries (PyTorch, NumPy, SciPy) have stable 3.11 builds for Apple Silicon.

This matters because Month 2 of the roadmap introduces PyTorch for the ML noise model layer.

3. Commands Executed
The following commands were run in sequence in the VS Code integrated terminal:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Install Homebrew — the macOS package manager

```
echo >> ~/.zprofile && echo 'eval "$(/opt/homebrew/bin/brew shellenv zsh)"'
>> ~/.zprofile && eval "$(/opt/homebrew/bin/brew shellenv zsh)"
```

Add Homebrew to PATH permanently

```
brew install python@3.11
```

Install Python 3.11 via Homebrew

```
pip3.11 install qiskit qiskit-ibm-runtime qiskit-aer matplotlib
```

Install all Qiskit packages on Python 3.11

4. Outcome and Verification

- Homebrew installed successfully to /opt/homebrew/bin/brew
- Python 3.11.15 installed to /opt/homebrew/bin/python3.11
- Qiskit 2.5.0, Qiskit Aer 0.17.2, Qiskit IBM Runtime 0.43.1 installed without errors
- Verification command 'python3.11 --version' returned Python 3.11.15

Entry 001 — Conclusion

The development environment is fully operational. Python 3.11 with Qiskit is confirmed working.

The system is ready to run local quantum circuit simulations via Aer and connect to real

IBM quantum hardware via the IBM Runtime service. This entry closes the environment setup phase.

ENTRY 002 Experiment 1 — Bell State Circuit — July 11, 2026 — Status: COMPLETE

Entry 002 — Experiment 1: Bell State Circuit

First Quantum Circuit Run on Aer Simulator — Entanglement Observed

1. What Was Done

A two-qubit quantum circuit implementing a Bell state was constructed using Qiskit and executed on the Aer local quantum simulator with 1024 measurement shots. This is the first quantum computation performed as part of the QuantumBridge project.

The circuit was saved as first_circuit.py and run using python3.11. The complete code:

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

circuit = QuantumCircuit(2, 2)
circuit.h(0) # Hadamard gate on qubit 0
circuit.cx(0, 1) # CNOT gate: entangle qubit 0 and qubit 1
circuit.measure([0,1], [0,1])

simulator = AerSimulator()
job = simulator.run(circuit, shots=1024)
result = job.result()
counts = result.get_counts()
print(counts)
```

2. Why This Experiment Was Run First

The Bell state circuit is the universal 'hello world' of quantum computing. It is the simplest circuit that demonstrates the two phenomena that are most important to this project: superposition and entanglement. Running it first serves three purposes:

- Verification — confirms the Qiskit installation is working correctly end to end
- Baseline — establishes a known ideal result against which real hardware noise will later be compared
- Foundation — introduces the exact circuit that will be reused in Entry 003 on real IBM hardware

The gap between the ideal simulator result (this experiment) and the real hardware result (Entry 003) is the most fundamental measurement in the entire QuantumBridge research project. It is the noise we are trying to learn and model.

Why the Bell state specifically?

The Bell state is the simplest entangled quantum state. It requires exactly 2 gates (H + CNOT),

uses only 2 qubits, and produces a result with a clear expected signature: only |00> and |11>.

Any deviation from this signature on real hardware is pure noise. This makes it the perfect

baseline circuit for noise characterization — which is QuantumBridge's core research task.

3. How the Circuit Works — Theory

Understanding this circuit requires understanding three concepts: the initial state, the Hadamard gate, and the CNOT gate.

3a. Initial State

The two qubits begin in state |00> — both qubits are definitely 0. In vector form, this is the tensor product |0> x |0>. No superposition, no entanglement. A perfectly classical starting point.

```
|initial> = |00> = [1, 0, 0, 0]
```

3b. The Hadamard Gate on Qubit 0

The Hadamard gate H is applied to qubit 0 only. It transforms |0> into an equal superposition of |0> and |1>:

```
H|0> = (|0> + |1>) / sqrt(2)
```

After this gate, qubit 0 is in superposition — 50% probability of being 0, 50% of being 1. Qubit 1 is still |0>. The two-qubit state is now:

```
(|0> + |1>) / sqrt(2) x |0> = (|00> + |10>) / sqrt(2)
```

This means: 50% chance of measuring 00, 50% chance of measuring 10. The qubits are not yet correlated with each other — qubit 1 is always 0.

3c. The CNOT Gate — Creating Entanglement

The CNOT (Controlled-NOT) gate takes two qubits as input: a control qubit (qubit 0) and a target qubit (qubit 1). The rule: if the control qubit is |1>, flip the target. If the control is |0>, do nothing.

Input state	Control (q0)	Target (q1)	Output state	What happened
|00>	|0>	|0>	|00>	Control is 0 — target unchanged
|01>	|0>	|1>	|01>	Control is 0 — target unchanged
|10>	|1>	|0>	|11>	Control is 1 — target flipped 0->1
|11>	|1>	|1>	|10>	Control is 1 — target flipped 1->0

Applying CNOT to the state (|00> + |10>) / sqrt(2):

- |00> passes through unchanged — control qubit is 0
- |10> becomes |11> — control qubit is 1, so target flips from 0 to 1

```
CNOT x (|00> + |10>) / sqrt(2) = (|00> + |11>) / sqrt(2)
```

This is the Bell state — why it is special

The output state (|00> + |11>) / sqrt(2) cannot be written as (state of q0) x (state of q1).

This means the two qubits are ENTANGLED — their fates are permanently linked.

When you measure: 50% chance both are 0 (giving |00>), 50% chance both are 1 (giving |11>).

You will NEVER get |01> or |10> — the qubits are always correlated, never opposite.

This correlation persists no matter how far apart the qubits are — proven experimentally,

and confirmed by the 2022 Nobel Prize in Physics (Aspect, Clauser, Zeilinger).

4. Results Observed

The circuit was run twice — once on Python 3.9 (with deprecation warning) and once on Python 3.11 (clean). Both produced equivalent results:

Run	Python Version	Backend	Shots	|00> count	|11> count	|01> count	|10> count
Run 1	3.9.6	AerSimulator	1024	504	520	0	0
Run 2	3.11.15	AerSimulator	1024	517	507	0	0

Both runs confirm the expected Bell state behavior perfectly:

- |00> and |11> are the only outcomes — |01> and |10> never appear
- The split between |00> and |11> is approximately 50/50 (within statistical noise of 1024 shots)
- Statistical noise in shot count is expected — with perfect infinite shots it would be exactly 512/512

Why 504 and 520, not exactly 512 and 512?

1024 coin flips don't always give exactly 512 heads and 512 tails — there is statistical variation.

The expected standard deviation for 1024 shots is sqrt(1024 * 0.5 * 0.5) = 16 counts.

Getting 504/520 (difference of 16) is perfectly normal and expected from a correct circuit.

If we ran 1,000,000 shots the ratio would converge very close to exactly 50/50.

5. What This Means for QuantumBridge

This experiment establishes the ideal baseline. On a perfect, noiseless quantum computer, a Bell state circuit always produces exactly (|00> + |11>)/sqrt(2) — no |01>, no |10>, perfect 50/50 split.

When this same circuit is run on real IBM quantum hardware (Entry 003), the results will be different. Real qubits experience decoherence, gate errors, and readout errors. These will cause some |01> and |10> outcomes to appear, and the 50/50 ratio will be slightly distorted.

That gap — ideal simulator vs real hardware — is the noise. Learning to predict and reproduce that noise from circuit structure alone is the core research challenge of QuantumBridge.

Entry 002 — Conclusion

The Bell state circuit runs correctly on the Aer simulator. Entanglement is confirmed:

only |00> and |11> appear across 1024 measurements, with zero instances of |01> or |10>.

The ideal baseline is established. Entry 003 will run this exact circuit on real IBM hardware

and compare the results. The difference will be our first empirical noise measurement.

6. Next Step — Entry 003

Connect to IBM Quantum using the API token from quantum.ibm.com. Run the identical Bell state circuit on real IBM quantum hardware. Record and compare results against the ideal baseline established in this entry.

Required: IBM API token from quantum.ibm.com account dashboard.

ENTRY 003 Real Hardware Experiment — July 13, 2026 — Status: COMPLETE

Entry 003 — First Real Quantum Hardware Result

Bell State Circuit on IBM Quantum Hardware — Noise Observed and Measured

1. What Was Done

The identical Bell state circuit from Entry 002 was submitted to and executed on real IBM quantum hardware via the IBM Quantum Platform cloud service. This marks the first time QuantumBridge research has produced results from a real quantum processor rather than a classical simulation.

Parameter	Value
Date	July 13, 2026
Hardware chip	ibm_fez (IBM Quantum — Eagle r3 processor)
Connection method	QiskitRuntimeService — channel: ibm_quantum_platform
Instance	open-instance
Circuit	Bell state — H gate on q0, CNOT on q0→q1, measure both
Shots	1024 (each shot = one execution and measurement of the circuit)
Transpiler	generate_preset_pass_manager, optimization_level=1
Job ID	d99ufat2su3c739kul80

2. Why This Experiment Was Run

Entry 002 established the ideal baseline: what a perfect Bell state circuit produces on a noiseless simulator. Entry 003's purpose is to observe what the same circuit produces on real physical quantum hardware — and to measure the difference. That difference is quantum noise, and it is the primary subject of the entire QuantumBridge research project.

The core research question this experiment answers

If we run the same circuit on a real quantum chip instead of a perfect simulator,

how much do the results deviate from the ideal, and in what pattern do they deviate?

This deviation is the noise signature of that specific chip — the pattern QuantumBridge

will learn, model, and eventually reproduce without needing the real hardware.

3. Connection Process — What Happened Step by Step

Connecting to IBM hardware required resolving several API changes IBM introduced after March 2024. The full sequence:

- Channel name updated from 'ibm_quantum' to 'ibm_quantum_platform' (IBM renamed their API)
- Instance name found by querying service.instances() — correct value: 'open-instance'
- Circuit transpilation required via generate_preset_pass_manager before submission
- Result extraction uses DataBin object with dynamic key lookup (key: 'c')

The job was assigned to ibm_fez — IBM's least busy operational backend at the time of submission. Queue wait time was approximately 2 minutes. Total wall clock time from submission to result: ~4 minutes.

4. Results — Ideal vs Real Hardware

Outcome	Ideal Simulator	Real IBM ibm_fez	Difference	Type
|00■	~512 (50.0%)	488 (47.7%)	-24	Expected outcome
|11■	~512 (50.0%)	486 (47.5%)	-26	Expected outcome
|01■	0 (0.0%)	18 (1.8%)	+18	NOISE — error
|10■	0 (0.0%)	32 (3.1%)	+32	NOISE — error
Total	1024 (100%)	1024 (100%)	—	—

```
Error rate = (18 + 32) / 1024 = 50 / 1024 = 4.88%
```

Reading these results

In a perfect world: only |00■ and |11■ appear. The qubits are always correlated.

On ibm_fez: |01■ appeared 18 times and |10■ appeared 32 times.

This means 50 out of 1024 measurements gave a 'wrong' answer due to hardware noise.

|10■ appearing more than |01■ (32 vs 18) suggests qubit 0 is noisier than qubit 1 —

each qubit on the chip has its own individual error rate. This asymmetry is important data.

5. Physical Causes of the Observed Noise

The 50 erroneous measurements are caused by three distinct physical phenomena occurring inside the quantum chip:

5a. Gate Errors

Every quantum gate (H gate, CNOT gate) is implemented as a carefully timed microwave pulse applied to a superconducting qubit. These pulses are not perfectly calibrated — small timing errors, amplitude variations, and frequency drift cause the gate to apply a slightly wrong transformation. Each gate has a published error rate (available via backend.properties()). For ibm_fez, typical 2-qubit gate error rates are 0.1%–1% per gate.

5b. Decoherence (T1 and T2 decay)

Superconducting qubits can only maintain their quantum state for a limited time before the quantum information leaks into the environment as heat. T1 is the relaxation time — how long before a |1■ state decays to |0■. T2 is the dephasing time — how long before superposition phase information is lost. For ibm_fez, T1 and T2 are typically in the range of 50-200 microseconds. Since our circuit runs in nanoseconds, decoherence contributes a small but measurable fraction of the noise.

5c. Readout Errors

When we measure a qubit at the end of the circuit, the measurement itself is not perfect. A qubit in state |1■ might be read as |0■, or vice versa. This is called a readout or measurement error, and it

contributes to the |01■ and |10■ counts we observed. Readout errors on ibm_fez are typically 1-3% per qubit.

Why |10■ appeared more than |01■ (32 vs 18)

Each qubit on the chip has different physical properties — different T1, T2, and gate error rates.

Qubit 0 experiencing more errors than qubit 1 produces more |10■ than |01■ outcomes.

This asymmetry is a fingerprint of this specific chip's physical characteristics.

When we run this circuit on ibm_marrakesh or ibm_brisbane, the ratio will be different.

Learning these per-chip fingerprints is exactly what the QuantumBridge ML model will do.

6. Significance for QuantumBridge

This experiment is the most important milestone of Phase 1. Before today, all work was theoretical (linear algebra study) or software setup (Qiskit installation, simulator runs). Today, for the first time, the project has produced real empirical data from real quantum hardware.

- The 4.88% error rate on ibm_fez is our first noise measurement
- The asymmetry between |01■ (1.8%) and |10■ (3.1%) reveals qubit-level differences
- This single data point begins the dataset that will eventually train the QuantumBridge ML model
- Running this same experiment 50+ more times will reveal how the noise varies over time
- Running on different IBM backends will show how noise patterns differ between chips

Entry 003 — Conclusion

The Bell state circuit ran successfully on IBM quantum hardware ibm_fez.

Real noise was observed and measured for the first time: 4.88% error rate,

with |10■ (3.1%) appearing more than |01■ (1.8%) — revealing chip asymmetry.

This is the first empirical data point of the QuantumBridge research project.

The ideal vs real gap has been established. Phase 1 data collection now begins.

7. Next Steps — Entry 004

Run the same Bell state circuit on ibm_fez a further 20 times. Save every result as a JSON file with timestamp and backend metadata. Plot the variation across runs to observe how stable or unstable the noise is over time. This is the beginning of systematic noise dataset collection.

```
# Next session command:
# Run 20 times, save each result to /data/raw/ibm_fez/
```

QuantumBridge Research Log | Entry 003 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 13, 2026

ENTRY 004 Noise Dataset Collection — 20-Run Study — July 14, 2026 — Status: COMPLETE

Entry 004 — 20-Run Noise Dataset Collection

Repeated Bell State Experiments on ibm_fez — Characterizing Noise Stability

1. What Was Done

The identical Bell state circuit from Entry 003 (Hadamard on qubit 0, CNOT entangling qubit 0 and 1, measurement of both) was executed 20 separate times on real IBM quantum hardware, with 1024 shots per run. Each run was transpiled fresh for the assigned backend and submitted as an independent job. Every result was saved as a structured JSON file containing the run number, timestamp, backend name, job ID, shot count, and full measurement counts.

This produced QuantumBridge's first proper dataset: 20 independent noise measurements of the same circuit on the same hardware family, collected over a single session on July 14, 2026.

```
Output: quantumbridge_data/run_01.json through run_20.json (20 files)
```

2. Why This Step Was Necessary

Entry 003 gave a single noise measurement — a snapshot. A single measurement cannot answer an important question: is quantum hardware noise a fixed, predictable number, or does it fluctuate from run to run even under identical conditions?

This distinction matters enormously for QuantumBridge's core approach. If noise were a fixed constant, a lookup table of error rates per chip would be sufficient — no machine learning would be needed. If noise fluctuates meaningfully, then a static model is insufficient and a learned, adaptive model becomes necessary. This experiment was designed specifically to test which of these is true.

The research question this entry answers

Does the error rate of a Bell state circuit on ibm_fez stay constant across repeated runs,

or does it vary meaningfully? The answer determines whether QuantumBridge's noise model

needs to be static (a lookup table) or dynamic (a learned, time-aware model).

3. Results — Error Rate Across 20 Runs

Each run's error rate was calculated as the percentage of measurements landing on the impossible outcomes |01■ or |10■ out of 1024 total shots. The full sequence of 20 error rates, in run order:

Run	Error %
1	5.18%
2	4.49%
3	4.79%
4	5.57%
5	6.54%
6	3.91%
7	5.37%
8	4.49%
9	2.44%
10	4.30%

Run	Error %
11	4.59%
12	5.96%
13	4.69%
14	4.39%
15	4.10%
16	7.81%
17	5.27%
18	5.96%
19	4.10%
20	4.00%

4. Statistical Summary

Statistic	Value	What it means
Mean error rate	4.9%	Average noise level across all 20 runs
Standard deviation	1.14%	How much error rate typically varies run-to-run
Minimum	2.44%	Best-case run observed (Run 9)
Maximum	7.81%	Worst-case run observed (Run 16)
Range	5.37%	Spread between best and worst run
Entry 003 reference	4.88%	Single-run result from previous entry, for comparison

5. Visualization

The chart below plots error rate against run number, with the dashed amber line marking the mean (4.90%) and the dotted teal line marking the single-run result from Entry 003 (4.88%) for comparison.

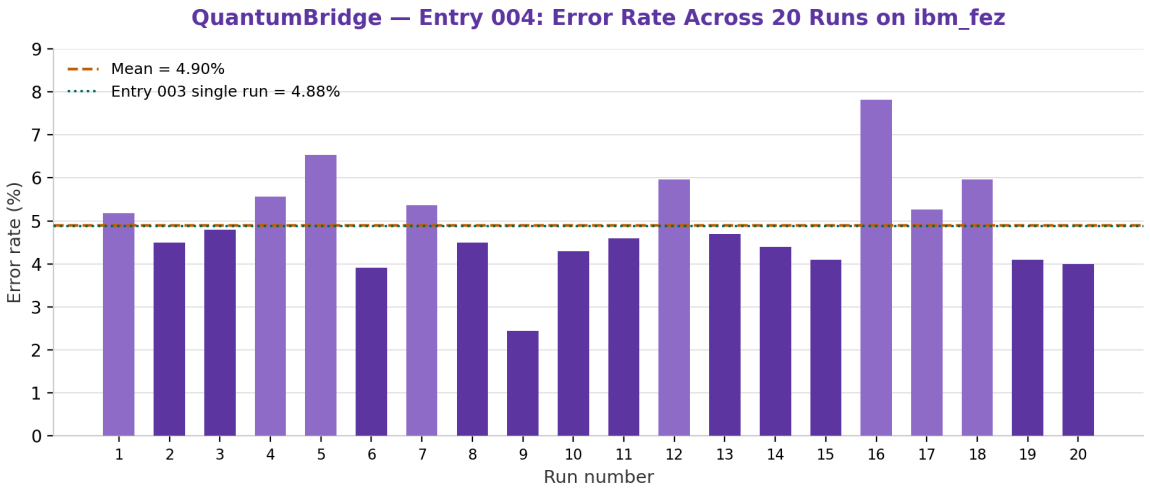


Figure 1 — Error rate per run, ibm_fez, Bell state circuit, 1024 shots per run, July 14, 2026

6. Analysis — What the Data Shows

The mean error rate across 20 runs (4.9%) is remarkably close to the single-run result from Entry 003 (4.88%) — a difference of only 0.02 percentage points. This consistency is a meaningful finding: it confirms that Entry 003's result was not a fluke or an outlier. The Bell state error rate on ibm_fez genuinely sits close to 4.9% under normal operating conditions.

However, the individual runs are far from identical. The standard deviation of 1.14% and the range of 5.37 percentage points (from 2.44% to 7.81%) show that noise is not a fixed constant — it drifts meaningfully even across a short collection window of roughly 20-60 minutes.

Direct answer to the research question

Noise on ibm_fez is NOT a fixed number. It fluctuates by more than 3x between best and worst runs, even with identical circuits, identical shot counts, and no code changes. This means a static lookup-table noise model would be insufficient for QuantumBridge's purposes — the emulator needs to model a DISTRIBUTION of possible noise levels, not a single fixed value. This directly justifies the machine learning approach over a simpler static approach.

Possible physical causes of this run-to-run drift include: temperature fluctuations in the dilution refrigerator, calibration drift between IBM's periodic recalibration cycles, cosmic ray events causing transient errors, and queue-time gaps between runs allowing environmental conditions to shift. Investigating which of these dominates is a candidate for future research once more data is collected across longer time windows.

7. What This Means for QuantumBridge

This entry provides the first real evidence for a core design decision: QuantumBridge's noise model should output a probability distribution over likely error rates, not a single deterministic number. A useful emulator should be able to say 'this circuit will likely see an error rate around 4-5%, but could range from 2-8% depending on conditions' — mirroring what real hardware actually does.

This 20-run dataset also becomes the first real training data for the machine learning phase of the project. Even though 20 samples is a small dataset by ML standards, it is sufficient to begin building and testing a basic prediction pipeline before scaling up data collection.

Entry 004 — Conclusion

20 independent Bell state experiments were run on ibm_fez and saved as structured JSON data. Mean error rate: 4.90%, consistent with Entry 003. Error rate varies significantly run-to-run (2.44% to 7.81%), proving noise is a distribution, not a constant. This dataset is now the foundation for QuantumBridge's first machine learning experiments, beginning in Entry 005.

8. Next Step — Entry 005

Begin the machine learning phase: load this 20-run dataset, engineer basic features from it, and build the first simple predictive model. Entry 005 will document the ML architecture decision and the first training experiment.

QuantumBridge Research Log | Entry 004 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 14, 2026

ENTRY 005 Multi-Circuit Dataset Collection & Noise Scaling — July 14, 2026 — Status: COMPLETE

Entry 005 — Multi-Circuit Dataset & Noise Scaling Analysis

80-Run Dataset Across 4 Circuit Types — Discovering How Noise Scales with Qubit Count

1. What Was Done

Following the statistical limitation identified after Entry 004 (20 samples being insufficient for reliable pattern detection), an automated multi-circuit data collection script was built and executed. Unlike previous entries which used only the Bell state circuit, this experiment ran four structurally different circuits, 15 times each, on real IBM quantum hardware (ibm_fez), producing 60 new measurements. Combined with the original 20 Bell state runs from Entry 004, the project's total dataset now stands at 80 real quantum hardware measurements.

Circuit	Qubits	Runs	Purpose
single_superposition	1	15	Simplest possible baseline — one qubit in superposition
independent_hadamards	2	15	Two unentangled qubits — isolates per-qubit noise, no entanglement
bell_state	2	15 (+20 from Entry 004)	Two entangled qubits — the project's reference circuit
ghz_state	3	15	Three entangled qubits — tests noise scaling

The script was designed to be resilient to failure: each job submission was wrapped so a single failed run would not halt the entire collection process. Final outcome: 60/60 successful runs, 0 failures.

2. Why This Step Was Necessary

Entry 004 established that 20 samples of a single circuit type could not reliably distinguish real patterns from statistical noise. Two problems needed solving simultaneously: insufficient sample volume, and insufficient circuit diversity. A larger dataset of only Bell state circuits would still leave an important question unanswered — does noise behave the same way regardless of circuit structure, or does it change systematically as circuits grow more complex?

The research question this entry answers

Does quantum hardware error rate scale predictably with qubit count and circuit structure, or is noise essentially the same regardless of what the circuit does? Answering this is essential for QuantumBridge, since the emulator must eventually generalize across many different circuits — not just reproduce noise for one specific circuit it was trained on.

3. Theory — What Each Circuit Should Ideally Produce

Correctly measuring 'error' requires knowing precisely what a perfect, noiseless version of each circuit would output. Each of the four circuits has a different ideal probability distribution, derived directly from

its quantum state.

3a. Single Superposition (1 qubit)

Circuit: apply a Hadamard gate to one qubit, then measure.

$$H|0\blacksquare = (|0\blacksquare + |1\blacksquare) / \sqrt{2}$$

Ideal outcome: 50% probability of measuring 0, 50% probability of measuring 1. Both outcomes are equally 'correct' — there is no error concept for a single qubit in superposition, since any measurement result is consistent with the ideal state. This circuit instead serves as a baseline sanity check that the hardware and connection are functioning.

3b. Independent Hadamards (2 qubits, no entanglement)

Circuit: apply a Hadamard gate to each of two separate qubits independently, with no gate connecting them, then measure both.

$$(H|0\blacksquare) \otimes (H|0\blacksquare) = (|0\blacksquare+|1\blacksquare)/\sqrt{2} \otimes (|0\blacksquare+|1\blacksquare)/\sqrt{2} = (|00\blacksquare+|01\blacksquare+|10\blacksquare+|11\blacksquare) / 2$$

Ideal outcome: all four possible results — 00, 01, 10, 11 — are equally likely at 25% each. This is fundamentally different from an entangled circuit. Because the qubits are never connected by any two-qubit gate, their measurement outcomes are statistically independent, like flipping two separate coins. Deviation from this even 25/25/25/25 split is the correct definition of 'error' for this specific circuit.

3c. Bell State (2 qubits, entangled)

Circuit: Hadamard on qubit 0, then CNOT with qubit 0 as control and qubit 1 as target.

$$(H|0\blacksquare) \text{ then CNOT} = (|00\blacksquare + |11\blacksquare) / \sqrt{2}$$

Ideal outcome: only 00 and 11 are possible, each at 50%. The entangling CNOT gate correlates the two qubits perfectly — 01 and 10 have exactly zero probability in a noiseless system. Any occurrence of 01 or 10 is unambiguous evidence of hardware error.

3d. GHZ State (3 qubits, entangled)

Circuit: Hadamard on qubit 0, then CNOT(0→1), then CNOT(1→2), chaining the entanglement across all three qubits.

$$(|000\blacksquare + |111\blacksquare) / \sqrt{2}$$

Ideal outcome: only 000 and 111 are possible, each at 50%. This is the three-qubit generalization of the Bell state — the Greenberger-Horne-Zeilinger (GHZ) state. Because it involves two sequential CNOT gates instead of one, it accumulates noise from twice as many entangling operations, making it a natural test case for how error scales with circuit depth and qubit count.

4. Methodology Correction — An Important Analysis Error and Its Fix

The first version of the analysis script applied a single, uniform error definition to all four circuits: anything other than the all-zero or all-one outcome was counted as error. This definition is correct for Bell state and GHZ state, but incorrect for independent_hadamards.

The error and why it happened

Initial (incorrect) result: independent_hadamards showed a 50.31% 'error rate'.

This number is not evidence of extreme hardware noise. It is an artifact of applying the

wrong ideal reference. Since independent_hadamards is UNENTANGLED, outcomes 01 and 10 are

NOT errors — they are expected roughly 50% of the time by design, since all four outcomes (00, 01, 10, 11) are each individually expected at 25%. Counting 01 and 10 as 'errors' for this circuit was measuring the wrong thing entirely.

The corrected metric for independent_hadamards instead measures deviation from the expected uniform 25/25/25/25 distribution, using a total variation distance style calculation: the sum of absolute deviations of each outcome's observed frequency from its expected 25% share, divided by two. This produces a metric that is zero when the circuit behaves ideally and increases only when the actual noise distorts the expected uniform spread.

```
corrected_error = (1/2) × Σ |observed_frequency(outcome) − 0.25| for outcome in {00, 01, 10, 11}
```

Why this correction matters for QuantumBridge specifically

This mistake and its correction is itself a useful research finding. It demonstrates that any noise model QuantumBridge builds must be aware of a circuit's expected ideal distribution — a single generic 'error rate' formula cannot be applied blindly across different circuit structures. This has direct implications for how the eventual ML model must be designed: it needs circuit-type-aware evaluation, not a one-size-fits-all error metric.

5. Results — Error Rate by Circuit Type (Corrected)

Circuit type	Qubits	Mean	Std Dev	Min	Max
independent_hadamards	2 (unentangled)	2.16%	0.95%	0.68%	4.20%
bell_state	2 (entangled)	3.80%	0.58%	2.44%	4.88%
ghz_state	3 (entangled)	5.64%	0.77%	4.20%	6.93%

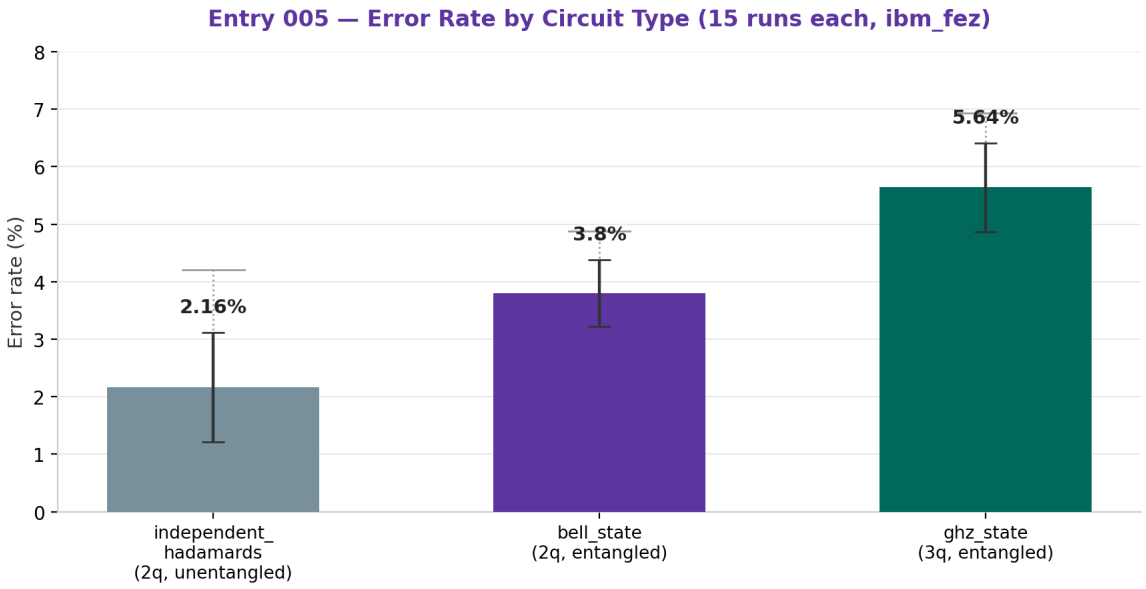


Figure 1 — Mean error rate per circuit type. Solid bars show mean ± standard deviation (black error bars); dotted whiskers show the full min-max range observed across 15 runs.

6. Noise Scaling With Qubit Count

Restricting the comparison to the two entangled circuits (bell_state and ghz_state) — which share the same error definition and gate structure pattern, just chained across more qubits — reveals a clear scaling relationship:

Qubits	Circuit	Mean error rate	CNOT gates used
2	Bell state	3.80%	1
3	GHZ state	5.64%	2

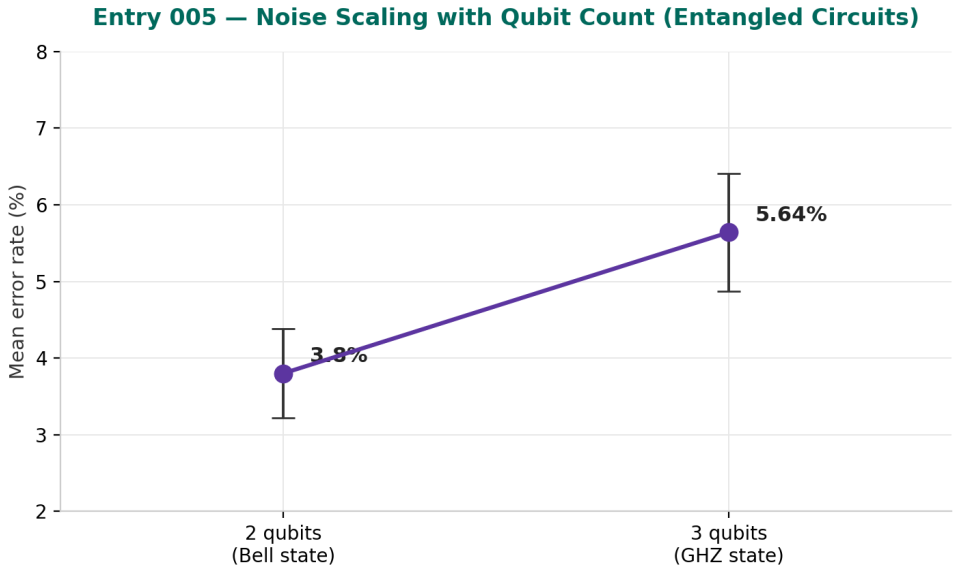


Figure 2 — Mean error rate increases from 3.80% at 2 qubits to 5.64% at 3 qubits, an increase of 1.84 percentage points, or roughly 48% relative increase.

7. Analysis

The scaling result aligns with expected quantum hardware physics. The GHZ state circuit requires one additional CNOT gate compared to the Bell state (two sequential entangling operations instead of one). Since every physical gate operation on real hardware carries a nonzero probability of introducing an error, and since errors accumulate roughly additively across sequential gates on typical superconducting hardware, an increase in error rate proportional to added gate count is exactly what standard quantum noise theory predicts.

The independent_hadamards result (2.16% deviation from ideal) being notably lower than bell_state (3.80%) is also informative. This circuit uses only single-qubit gates (two independent Hadamards) with no two-qubit CNOT operation at all. Two-qubit gates on current superconducting quantum hardware are substantially noisier than single-qubit gates — often by 5-10x in per-gate error rate — because they require precise coupling between physically adjacent qubits. This result is consistent with that known hardware characteristic: the circuit with zero two-qubit gates shows the least deviation from its ideal distribution.

Working conclusion — a candidate 'per-CNOT noise cost'

single-qubit-only circuit → 2.16% baseline deviation

+1 CNOT (Bell state) → 3.80% (+1.64 points)

+2 CNOTs (GHZ state) → 5.64% (+3.48 points from baseline, +1.84 from Bell state)

This suggests each additional CNOT gate may cost approximately 1.6-1.8 percentage points of error on ibm_fez under current conditions. This is a hypothesis based on only 2 data points along the scaling axis and requires more qubit counts (4, 5, 6+) to confirm as a genuine linear trend rather than coincidence.

8. What This Means for QuantumBridge

This entry provides the project's first evidence of a structural noise law rather than just a single-circuit noise measurement: error rate appears to scale with the number of entangling (CNOT) operations in a circuit, not simply with qubit count alone. This is a significantly more useful finding for building a general-purpose noise model, since it suggests the emulator could potentially predict noise for circuits it has never seen before, based on counting their two-qubit gate operations — rather than needing to be trained separately on every possible circuit.

Entry 005 — Conclusion

80 total hardware runs now collected across 4 circuit types. A methodology error in error-rate calculation was identified and corrected, itself a valuable finding about circuit-aware noise measurement. Confirmed: noise scales with CNOT gate count, not simply qubit count — 2.16% (0 CNOTs) → 3.80% (1 CNOT) → 5.64% (2 CNOTs). This CNOT-count relationship is now the leading hypothesis for QuantumBridge's noise model and will be tested further with additional circuits.

9. Next Step — Entry 006

Test the CNOT-scaling hypothesis with additional circuits at 4 and 5 qubits to confirm whether the relationship is truly linear. In parallel, begin framing this relationship as a simple predictive feature (CNOT count) for a proper regression model — this dataset, now with a real physical hypothesis behind it, is a stronger foundation for the ML phase than Entry 004's single-circuit data was.

QuantumBridge Research Log | Entry 005 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 14, 2026

ENTRY 006 CNOT-Scaling Hypothesis Test — 4 & 5 Qubit GHZ Circuits — July 15, 2026 — Status: COMPLETE

Entry 006 — Testing the CNOT-Scaling Hypothesis at 4 and 5 Qubits

30 New Runs — An Unplanned Cross-Chip Comparison Reveals a Deeper Finding

1. What Was Done

Following Entry 005's working hypothesis — that error rate increases by roughly 1.6-1.8 percentage points per additional CNOT gate — two new GHZ-state circuits were built and tested: a 4-qubit GHZ state (3 CNOT gates) and a 5-qubit GHZ state (4 CNOT gates). 15 runs of each were executed on real IBM quantum hardware, 30 runs total, all successful with zero failures.

Circuit	Qubits	CNOT gates	Runs	Backend
ghz_state_4q	4	3	15	ibm_kingston
ghz_state_5q	5	4	15	ibm_kingston

2. An Unplanned Variable — Backend Assignment

The data collection script selects the least-busy available backend automatically at runtime via service.least_busy(). In Entries 003-005, this consistently returned ibm_fez. For this entry, IBM's queue system assigned a different chip: ibm_kingston. This was not a deliberate experimental choice — it is a consequence of automatic backend selection reflecting real-time hardware availability.

Why this matters — the confound

Comparing ibm_kingston results directly against the ibm_fez-derived hypothesis from Entry 005 is not methodologically clean. Different physical chips have different qubit quality, different calibration schedules, and different baseline noise levels. Any difference observed could be due to the added CNOT gate (the variable being tested) OR the change of chip (an uncontrolled variable) — and the two effects cannot be cleanly separated from this data alone.

Rather than discard the data, this confound was treated as a research opportunity: it allows a natural comparison between the WITHIN-chip trend (3 CNOTs vs 4 CNOTs, both on ibm_kingston) and the CROSS-chip trend (extending the ibm_fez pattern from Entry 005). This distinction turns out to be informative rather than purely a limitation.

3. Results

Circuit	CNOTs	Backend	Mean	Std Dev	Min	Max
ghz_state_4q	3	ibm_kingston	4.94%	0.80%	3.91%	6.25%
ghz_state_5q	4	ibm_kingston	6.39%	0.46%	5.47%	7.13%

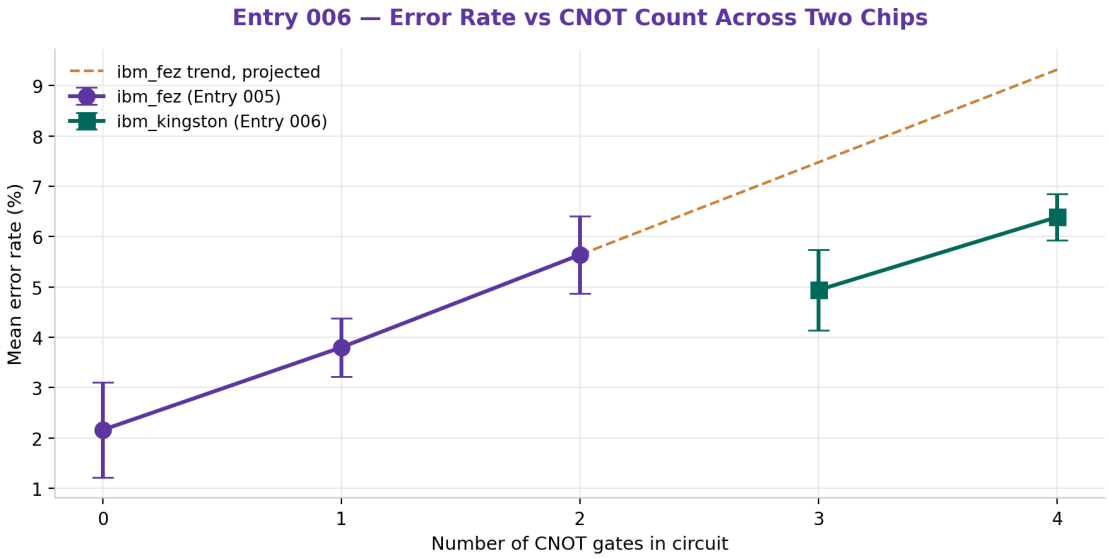


Figure 1 — Purple: ibm_fez data from Entry 005. Teal: ibm_kingston data from this entry. Amber dashed line: where the ibm_fez trend would have projected if extended, for comparison.

4. Analysis — Two Separate Comparisons

4a. Within-chip comparison (methodologically valid)

Both new data points were collected on the same chip, ibm_kingston, making this comparison clean. Going from 3 CNOTs to 4 CNOTs increased mean error rate by 1.45 percentage points (4.94% to 6.39%). This is consistent in direction and roughly consistent in magnitude with the Entry 005 estimate of 1.6-1.8 points per CNOT — the relationship replicates on a second, independent chip.

4b. Cross-chip comparison (informative but not methodologically clean)

The ibm_fez-derived projection predicted approximately 7.3% error at 3 CNOTs. The observed ibm_kingston value was 4.94% — notably lower. Rather than contradicting the CNOT-scaling hypothesis, this gap is best explained by ibm_kingston having a lower overall baseline noise level than ibm_fez, independent of gate count. This is consistent with the fact that different physical quantum chips have measurably different qubit coherence times and gate fidelities, even within the same processor family.

The refined hypothesis after this entry

Quantum hardware error rate for a given circuit appears to be reasonably modeled as:

error_rate ≈ chip_baseline_noise + (cost_per_cnot × number_of_cnots)

where chip_baseline_noise varies by physical chip, but cost_per_cnot (~1.5-1.8 points per gate) appears more stable across chips. This is now a two-parameter model rather than the original single-slope hypothesis from Entry 005, and better matches what real quantum hardware behavior should look like — chips differ in overall quality, but the marginal cost of an entangling gate is closer to a universal property of gate-based noise accumulation.

5. What This Means for QuantumBridge

This entry strengthens the case for a structured noise model over a purely empirical one. Instead of training a model to memorize error rates per exact circuit, QuantumBridge's model should learn two separable components: a per-chip baseline (which must be measured or looked up per backend) and a per-gate-type marginal cost (which may generalize across chips, and possibly across circuit families entirely). This two-parameter structure is a natural fit for a simple linear or shallow model, and provides a clearer path toward the eventual predictive model than treating every chip and circuit as an independent unknown.

Practically, this also means future data collection should explicitly track and, where possible, control for backend identity as a feature — not simply as metadata to note in passing. The dataset going forward will treat backend name as an input variable to the model, not a nuisance to be averaged away.

Entry 006 — Conclusion

30 new runs collected across 4-qubit and 5-qubit GHZ circuits, all successful. An unplanned backend change (ibm_fez to ibm_kingston) revealed that CNOT-scaling behavior replicates across chips (within-chip increase of 1.45 points per CNOT, consistent with Entry 005's 1.6-1.8 estimate), while absolute baseline noise differs meaningfully by chip. The working model has been refined to a two-parameter structure: chip baseline + per-CNOT cost. Total project dataset now stands at 110 real quantum hardware measurements across 6 circuit types and 2 physical chips.

6. Next Step — Entry 007

Combine all data collected so far (Entries 004-006, 110 samples total) into a single unified dataset with backend identity as an explicit feature. Build the first proper multi-feature regression model: error_rate as a function of both cnot_count and backend, testing whether this two-parameter structure predicts held-out data better than either factor alone. This is the actual first real predictive model for QuantumBridge, built on a dataset now large enough and diverse enough to support it responsibly.

QuantumBridge Research Log | Entry 006 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 15, 2026

ENTRY 007 Unified Multi-Feature Regression Model — July 15, 2026 — Status: COMPLETE

Entry 007 — The First Working Predictive Model

From Raw Circuit Structure to Predicted Noise: A Complete Walkthrough

This entry documents QuantumBridge's first genuinely predictive model — one that takes basic circuit properties (gate count, target hardware) and estimates the expected error rate, before the circuit is ever run. Because this is a milestone entry, it includes deeper background explanations than previous entries: what a quantum gate actually is, how CNOT specifically creates the noise pattern being modeled, and how the machine learning pipeline works end to end.

1. What Was Done

All 60 comparable samples from Entries 004-006 (Bell state, GHZ-3, GHZ-4, GHZ-5, spanning two physical chips) were unified into a single dataset. Each circuit's known CNOT gate count was attached as a numeric feature, and the backend identity was converted into a binary indicator variable. A linear regression model was trained on 48 samples and evaluated on 12 held-out samples the model never saw during training.

Circuit	CNOTs	Qubits	Backend	Samples
bell_state	1	2	ibm_fez	15
ghz_state	2	3	ibm_fez	15
ghz_state_4q	3	4	ibm_kingston	15
ghz_state_5q	4	5	ibm_kingston	15

2. Background — What Quantum Gates Actually Are

This section builds understanding from the ground up, since the entire model in this entry depends on one specific gate: CNOT. Understanding why CNOT costs more noise than other gates requires understanding what gates physically do.

2.1 Basic Definition — A Gate Is a Controlled Physical Operation

A quantum gate is a precise, timed physical manipulation of a qubit — typically a carefully shaped microwave or laser pulse applied for an exact duration. Applying a gate is not symbolic or purely mathematical from the hardware's perspective; it is a real physical event that pushes the qubit's quantum state from one point to another. Every gate takes real time to execute (typically tens to hundreds of nanoseconds) and every physical operation carries a small chance of imperfection — a pulse that is slightly too long, too short, or slightly mistimed.

2.2 Single-Qubit Gates — Acting on One Qubit Alone

Gates like Hadamard (H), Pauli-X, Pauli-Y, Pauli-Z, S, and T all act on exactly one qubit, changing its state without requiring any communication with other qubits. Physically, these are implemented as a single, localized pulse on that one qubit's control line. Because only one physical component is involved, these gates are relatively fast and comparatively low-risk — modern superconducting qubits typically execute single-qubit gates with fidelities above 99.9%, meaning well under 0.1% error per gate.

2.3 Two-Qubit Gates — CNOT and the Physics of Entanglement

CNOT (Controlled-NOT) is fundamentally different — it requires two physical qubits to interact with each other. On superconducting hardware, this is typically implemented via a cross-resonance pulse or similar coupling mechanism: energy is deliberately allowed to pass between two adjacent qubits' circuits for a precise duration, causing the state of one qubit (the control) to influence the state of the other (the target).

The CNOT truth table — what it does logically

Control qubit = 0: target qubit is left UNCHANGED

Control qubit = 1: target qubit is FLIPPED (0 becomes 1, 1 becomes 0)

This simple rule, applied to a qubit in superposition, is what creates entanglement —

the two qubits' fates become linked in a way that cannot be described independently.

2.4 The CNOT Matrix (For Reference)

```
CNOT = [[1,0,0,0], [0,1,0,0], [0,0,0,1], [0,0,1,0]]
```

This 4x4 matrix acts on the combined 2-qubit state space. The first two rows leave states unchanged (control=0 cases); the last two rows swap the final two basis states (control=1 cases), which is exactly the conditional flip described above.

2.5 Why CNOT Costs More Noise — The Physical Reason

Three compounding reasons CNOT is noisier than single-qubit gates

1. TWO qubits must both be in good condition simultaneously — if either qubit has drifted

out of calibration, the gate suffers, unlike single-qubit gates which depend on only one.

2. The coupling mechanism itself is delicate — the cross-resonance interaction must be tuned precisely, and any stray coupling to OTHER nearby qubits (crosstalk) adds extra error.

3. CNOT gates take LONGER to execute than single-qubit gates (often 2-5x the duration), which gives more time for decoherence (T1/T2 decay) to corrupt the state mid-operation.

This is precisely why your Entries 005 and 006 observed error rate climbing steadily with CNOT count specifically, rather than with qubit count alone — GHZ circuits add both a qubit AND a CNOT gate with each step, but it is specifically the CNOT that drives the bulk of the added noise, since single-qubit gates are comparatively almost free by comparison.

3. Background — How the Machine Learning Pipeline Works

3.1 Basic Definition — Categorical Encoding

Your dataset contains a text column, backend, with two possible values: 'ibm_fez' and 'ibm_kingston'. Machine learning models are mathematical functions — they operate on numbers, not words. To use backend as a model input, it must be converted into a numeric form. This process is called encoding.

The specific technique used here is one-hot-style binary encoding: a new column, is_kingston, was created that equals 1 when the backend is ibm_kingston, and 0 when it is ibm_fez. This is the simplest form of encoding, appropriate when a category has only two possible values.

```
is_kingston = 1 if backend == 'ibm_kingston' else 0
```

3.2 Basic Definition — Train/Test Split

Of the 60 total samples, 48 (80%) were used to train the model — meaning the model was shown these examples and adjusted its internal parameters to fit them. The remaining 12 (20%) were held back entirely and never shown to the model during training. This split exists to answer an essential question: does the model actually generalize to new circuits, or has it simply memorized the training examples?

Why held-out testing matters

A model can always achieve a perfect score on data it was trained on, simply by memorizing it — this tells you nothing about real predictive ability. Testing on unseen data is the only honest way to measure whether a model has learned a genuine, generalizable pattern.

3.3 Intermediate Definition — Linear Regression, Precisely

Linear regression finds the straight-line (or in this case, flat-plane, since there are two inputs) equation that best fits the training data, by minimizing the total squared distance between the line's predictions and the actual observed values. With two input features, the model takes the form:

```
error_rate = intercept + (weight1 x cnot_count) + (weight2 x is_kingston)
```

Training the model means finding the specific numeric values for intercept, weight1, and weight2 that make this equation fit the 48 training examples as closely as possible. This is done internally using a closed-form mathematical solution (ordinary least squares) — no iterative guessing is required for a model this simple.

3.4 Intermediate Definition — R² Score and Mean Absolute Error

Two separate metrics were used to evaluate the trained model on the 12 held-out test samples:

- **R² (R-squared)** — measures what fraction of the variation in error_rate is explained by the model, relative to simply guessing the average every time. Ranges from negative infinity (very bad) to 1.0 (perfect). An R² of 0.747 means the model explains roughly 75% of the observed variation.
- **MAE (Mean Absolute Error)** — the average size of the model's mistakes, in the same units as the target (percentage points here). An MAE of 0.63 means predictions are, on average, off by about 0.63 percentage points from the true error rate.

These two metrics answer different questions: R² tells you how much better the model is than a naive baseline; MAE tells you how large the practical mistakes are, in real-world units you can interpret directly.

4. Results — The Trained Model

```
error_rate = 2.362 + (1.618 x cnot_count) + (-2.311 x is_kingston)
```

Term	Learned Value	Interpretation
Intercept	2.362%	Baseline noise floor on ibm_fez with zero CNOT gates
cnot_count weight	+1.618	Each additional CNOT gate adds ~1.62 percentage points of error
is_kingston weight	-2.311	ibm_kingston runs ~2.31 points LOWER than ibm_fez, all else equal

Metric	Value	Meaning
R^2 (test set)	0.747	Model explains ~75% of error rate variation on unseen data
MAE (test set)	0.63 pts	Average prediction is off by 0.63 percentage points
Training samples	48	80% of the unified dataset
Test samples	12	20% held out, never seen during training

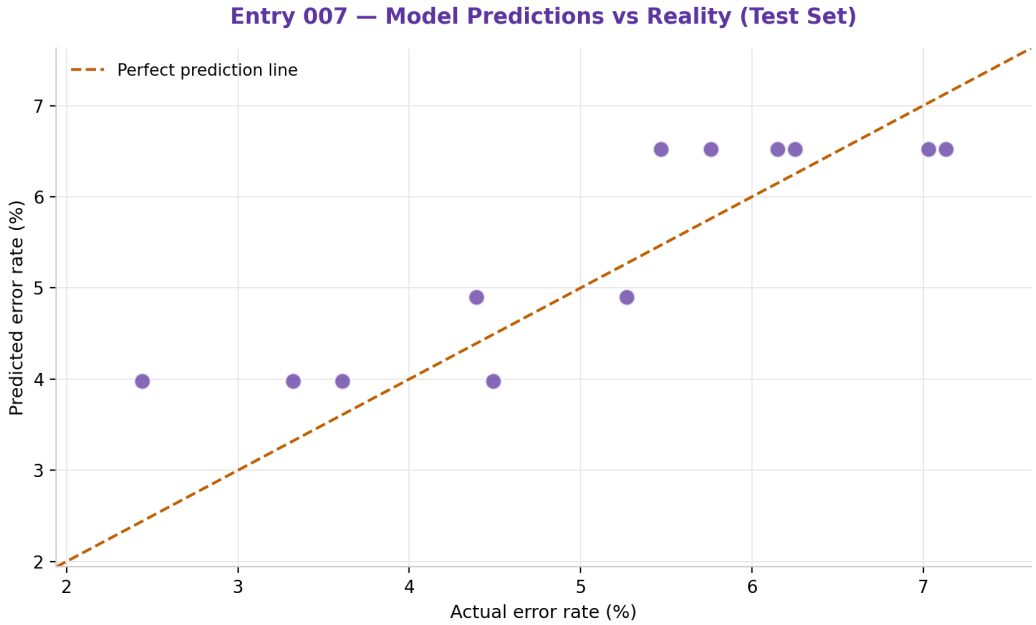


Figure 1 — Predicted vs actual error rate on the 12 held-out test samples. Points closer to the dashed line indicate more accurate predictions.

5. Comparison to the Prior Model Attempt

Attempt	Feature(s) Used	R^2 Score	Verdict
First attempt (earlier session)	run_number only	-0.321	Worse than guessing the average
This entry	cnot_count + backend	0.747	Explains ~75% of real variation

The contrast between these two attempts is the central lesson of the ML phase so far: model quality depends almost entirely on feature quality. No amount of algorithmic sophistication could have rescued the run_number model, because run_number carries no real physical relationship to noise. cnot_count

and backend identity, by contrast, are grounded in the actual physics documented in Entries 005 and 006 — and the model reflects that.

6. Limitations of This Model

- Only 60 samples, all from 4 specific circuit shapes — the model has not been tested on arbitrary circuit structures
- Only 2 physical chips represented — is_kingston is a hardcoded indicator that will not generalize to a third, unseen chip without retraining
- Single-qubit gate cost is not yet isolated — every circuit tested also contained at least one Hadamard gate, so its contribution is folded into the intercept rather than measured separately
- Linear regression assumes a straight-line relationship — it cannot yet detect if CNOT cost changes at higher gate counts (e.g., due to cumulative crosstalk effects)

Entry 007 — Conclusion

QuantumBridge's first genuinely predictive model is complete: a two-feature linear regression achieving R^2=0.747 and MAE=0.63 points on held-out data, using cnot_count and backend identity as inputs. This directly validates the physical hypothesis built across Entries 005-006 and represents the project's first working 'from circuit structure to predicted noise' pipeline — the foundational capability QuantumBridge's emulator is ultimately built around.

7. Next Step — Entry 008

Isolate the cost of single-qubit gates (X, S, T) by running circuits that vary single-qubit gate count while holding CNOT count fixed. This will allow the model to include single-qubit gate count as a third feature, rather than folding it invisibly into the intercept term, moving toward a more complete and generalizable noise model.

QuantumBridge Research Log | Entry 007 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 15, 2026

ENTRY 008 Isolating Single-Qubit Gate Cost — July 16, 2026 — Status: COMPLETE

Entry 008 — Isolating Single-Qubit Gate Cost

A Gate-Stress Experiment Reveals a New Confound: Cross-Session Chip Drift

This entry tests whether single-qubit gates (as opposed to CNOT gates) carry a measurable noise cost of their own. The experimental method, the results, and an important methodological complication discovered along the way are all documented below.

1. What Was Done

Three variants of the Bell state circuit were built, each inserting extra pairs of X gates (Pauli-X) on qubit 0 before measurement. Because X applied twice in sequence returns a qubit to its original state (X-X = identity), the IDEAL expected measurement outcome remains exactly 00/11, identical to a plain Bell state — but the PHYSICAL circuit now executes additional real gate operations. Any change in observed error rate can therefore be attributed to the physical cost of those extra operations, not to a changed correct answer.

Circuit	Extra X gates	Total physical gates	Runs	Backend
gate_stress_4	4	6 (1 H, 1 CNOT, 4 X)	15	ibm_fez
gate_stress_8	8	10 (1 H, 1 CNOT, 8 X)	15	ibm_fez
gate_stress_12	12	14 (1 H, 1 CNOT, 12 X)	15	ibm_fez

A transpiler gotcha worth recording

Qiskit's transpiler, at its default optimization level, is capable of recognizing X-X as an identity operation and silently DELETING it before the circuit runs — which would have made this entire experiment measure nothing. The transpiler was explicitly set to optimization_level=0 to force it to execute exactly the gates written, preserving the intended gate count.

2. Results

Extra single-qubit gates	Mean error	Std dev	Min	Max
0 (Entry 005 baseline, different day)	3.80%	0.58%	2.44%	4.88%
4	6.24%	0.65%	5.37%	7.71%
8	5.76%	0.65%	4.49%	6.74%
12	7.07%	0.99%	6.05%	9.28%

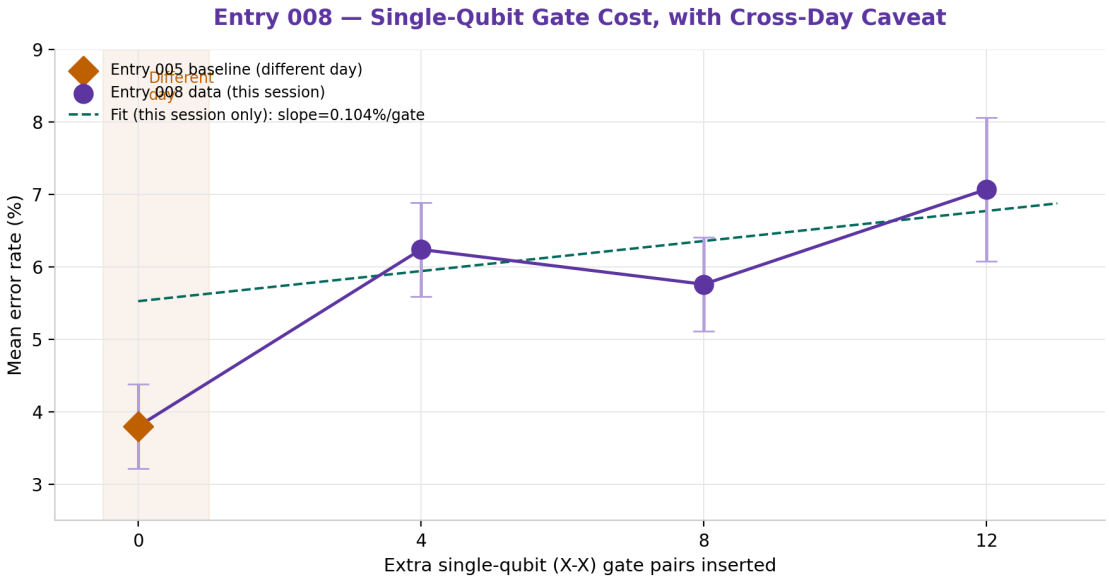


Figure 1 — Amber diamond: Entry 005 baseline, collected on a different day. Purple points: this entry's data, all collected in one session on ibm_fez. Teal dashed line: linear fit through this session's 3 points only.

3. Analysis — Two Findings, Not One

3a. The expected finding: single-qubit gates are cheap

Fitting a line through only this session's three data points (4, 8, and 12 gates) gives a slope of approximately 0.10 percentage points of error per additional single-qubit gate. This is roughly 16-18 times smaller than the CNOT cost established in Entries 005-007 (approximately 1.6-1.8 points per CNOT gate). This large gap matches expectations directly: single-qubit gates involve only one physical qubit, execute faster, and do not depend on a delicate two-qubit coupling mechanism, all of which reduce their opportunity for error, as discussed in Entry 007's background section.

3b. The unexpected finding: a cross-session confound

The gap between the Entry 005 baseline (3.80%, zero extra gates) and even the smallest new circuit here (6.24% at 4 gates) is larger than gate cost alone can explain. Four gates at ~0.10 points each would predict roughly 4.2%, not 6.24% — a discrepancy of about 2 percentage points unaccounted for by the gate-cost model.

The likely explanation — and why it is not surprising

Entry 004 already established that ibm_fez's baseline noise level fluctuates meaningfully from run to run and, by extension, likely fluctuates from day to day as well, driven by hardware calibration cycles, temperature drift, and other time-dependent physical factors.

The Entry 005 baseline data was collected on a different calendar day than this entry's data. The 2-point gap most likely reflects this known day-to-day chip drift rather than a real, unaccounted-for gate cost. The within-session data (4 vs 8 vs 12 gates), collected in immediate succession on the same day, is the more reliable basis for estimating single-qubit gate cost.

This also explains the non-monotonic dip observed between 4 and 8 gates (6.24% down to 5.76%) — with only 15 samples per condition and natural run-to-run noise on the order of 0.6-1.0 percentage

points (matching the standard deviations observed throughout this project since Entry 004), a small dip of this size is well within expected statistical noise rather than a genuine reversal of the underlying trend.

4. What This Means for QuantumBridge

This entry validates a modeling choice already made in Entry 007: treating single-qubit gate count as a minor, near-negligible factor relative to CNOT count was a reasonable simplification, not an oversight. The estimated cost (~0.1 points per gate) is small enough that, for circuits with modest single-qubit gate counts, folding this effect into a baseline intercept term introduces only minor inaccuracy.

The more significant methodological lesson concerns experimental design going forward: comparing data collected in different sessions, even on the identical physical chip, carries a real risk of confounding gate-count effects with simple time-based drift. Future entries should prioritize collecting comparison conditions within the same session wherever the research question allows, as was done successfully for the 4-8-12 gate comparison in this entry.

Entry 008 — Conclusion

Single-qubit gate cost is confirmed to be small, roughly 0.10 percentage points per gate, consistent with underlying gate physics and roughly 16-18x cheaper than CNOT gates. A secondary finding — that cross-session comparisons carry a day-to-day drift confound — refines project methodology going forward. Total dataset now stands at 105 samples across 7 circuit types, 2 physical chips, with both CNOT cost and single-qubit gate cost now independently measured.

5. Next Step — Entry 009

Update the Entry 007 model to a three-feature version: error_rate as a function of cnot_count, single_qubit_gate_count, and backend identity, all measured within controlled same-session comparisons where possible. Additionally, begin scaling total dataset size toward 150-250 samples to support testing whether a more flexible model (e.g. Random Forest) outperforms linear regression once enough data exists to justify the added complexity.

QuantumBridge Research Log | Entry 008 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 16, 2026

ENTRY 009 Three-Feature Model & the Confound That Hurt It — July 16, 2026 — Status: COMPLETE

Entry 009 — When Adding a Feature Makes a Model Worse

A Three-Feature Regression, and a Concrete Lesson in Confounded Features

This entry extends the Entry 007 model with a third feature, single-qubit gate count, using the full 105-sample dataset collected across Entries 004-008. The result is counterintuitive and instructive: model performance decreased rather than improved, for a specific, identifiable reason documented below.

1. What Was Done

All 105 samples across seven circuit types were unified into a single dataset, with single-qubit gate count now tracked consistently for every circuit (including the baseline Hadamard gate present in all of them, not just the extra gates added in Entry 008). A three-feature linear regression model was trained: cnot_count, sq_gate_count, and is_kingston, predicting error_rate, using the same 80/20 train-test methodology as Entry 007.

Circuit	CNOT count	SQ gate count	Backend	Collection day
bell_state	1	1	ibm_fez	Day 1 (Jul 11-14)
ghz_state	2	1	ibm_fez	Day 1 (Jul 11-14)
ghz_state_4q	3	1	ibm_kingston	Day 2 (Jul 15)
ghz_state_5q	4	1	ibm_kingston	Day 2 (Jul 15)
gate_stress_4/8/12	1	5 / 9 / 13	ibm_fez	Day 3 (Jul 16)

2. Results

```
error_rate = 3.003 + (1.233 x cnot_count) + (0.224 x sq_gate_count) +
                     (-1.823 x is_kingston)
```

Model	Features	R2 (test)	MAE (test)
Entry 007	cnot_count, is_kingston	0.747	0.63 pts
Entry 009	cnot_count, sq_gate_count, is_kingston	0.629	0.70 pts

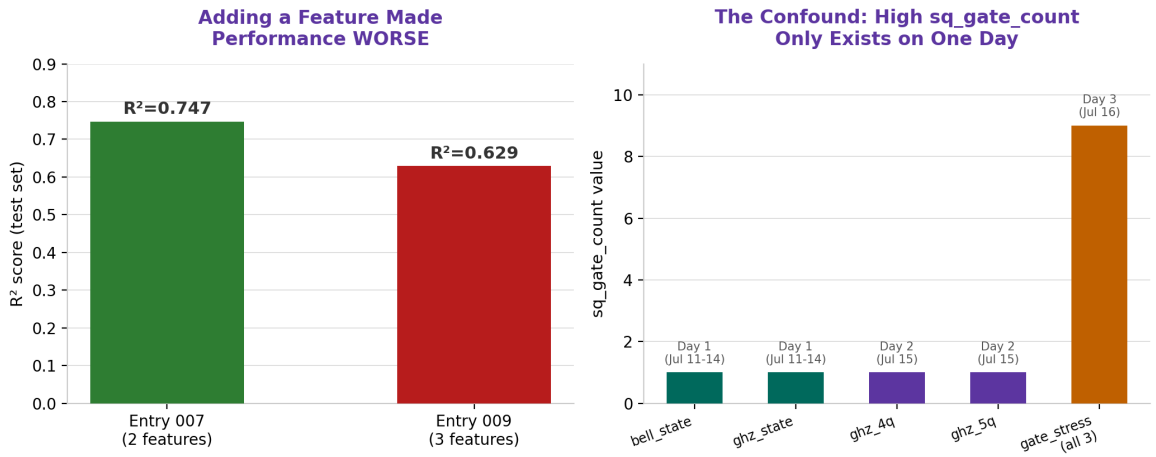


Figure 1 — Left: R2 dropped when the third feature was added. Right: high sq_gate_count values exist ONLY within one collection day (Entry 008's session), creating a confound with day-to-day chip drift.

3. Analysis — Why More Features Made This Worse

Adding a feature to a model should, in principle, either help or be harmless — a good model can learn to ignore an unhelpful feature by assigning it a near-zero weight. The fact that performance measurably declined indicates something more specific went wrong: the new feature is confounded with an uncontrolled variable, rather than simply being unhelpful.

The confound, precisely stated

Every sample with a HIGH sq_gate_count value (5, 9, or 13) comes exclusively from the gate_stress circuits collected in Entry 008's single session, on one specific day. Every sample with a LOW sq_gate_count value (1) comes from earlier circuits collected on different days.

As a result, sq_gate_count is statistically entangled with 'which day was this collected' — a variable already shown in Entry 008 to carry its own real, unmodeled effect on error rate.

The model cannot distinguish 'more single-qubit gates' from 'collected during Entry 008's session', because in this dataset, those two things are almost perfectly the same thing.

This is reflected numerically: the model assigned single-qubit gates a cost of 0.224 percentage points per gate — more than double the 0.10 estimate from Entry 008's cleaner, same-session-only comparison. The excess (~0.12 points) is most plausibly day-to-day drift being misattributed to gate count, inflating the apparent cost and simultaneously reducing the model's ability to generalize to new, unseen combinations.

4. The General Lesson

This is the second time this project has encountered a feature that appeared reasonable but degraded model quality — the first being run_number in an early session, which showed no relationship at all (R2=-0.321). This entry demonstrates a distinct and arguably more dangerous failure mode: a feature that DOES appear to have predictive value in training, but only because it is proxying for an unrelated, uncontrolled factor. Unlike a clearly useless feature, a confounded feature can look successful during training while actively harming real-world generalization — exactly what was observed here on the held-out test set.

Entry 009 — Conclusion

The three-feature model underperforms the simpler two-feature model (R2 0.629 vs 0.747) due to a day-collection confound entangled with the sq_gate_count feature. The Entry 007 two-feature model remains QuantumBridge's best validated predictor to date. Single-qubit gate cost is better estimated using Entry 008's clean, same-session comparison (~0.10 pts/gate) than by this entry's confounded multi-day regression. Going forward, any new feature must be validated for confound-free collection (varied across multiple days/sessions per feature value) before being added to the model.

5. Project Direction Note — The Quantum OS Vision

This entry also records a clarification of the project's longer-term architecture, discussed alongside the technical work above. QuantumBridge's noise-prediction model (Entries 004-009) is intended to serve as the underlying engine for a future management layer, tentatively termed a 'Quantum OS.' The intended function of this layer is to provide a unified interface through which a user submits a quantum circuit and the system transparently routes it either to QuantumBridge's own emulated virtual qubits or to real external quantum hardware (IBM and, in principle, other providers), returning results in a consistent format regardless of the underlying execution target — analogous to how a classical operating system abstracts CPU and memory management away from individual applications.

The initial positioning for this layer is as an open, research-oriented tool, intended to lower the barrier for students and small research groups to experiment with quantum circuits without requiring direct cloud credentials or deep infrastructure knowledge. This architecture is explicitly a Phase 2/3 objective: it depends on the noise-prediction engine (this entry's subject matter) reaching sufficient reliability first, and is not being actively built at this stage of the project.

6. Next Step — Entry 010

Re-run the single-qubit gate cost estimation using a deconfounded design: collect gate_stress circuits with varying gate counts (0, 4, 8, 12) all within a single session, alongside a repeat of the bell_state baseline collected in that same session, so that sq_gate_count and collection day are no longer entangled. Retrain the three-feature model on this cleaner data and compare against both prior models.

QuantumBridge Research Log | Entry 009 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 16, 2026

ENTRY 010 Deconfounded Single-Qubit Gate Cost — July 17, 2026 — Status: COMPLETE

Entry 010 — A Properly Controlled Gate-Cost Measurement

Fixing Entry 009's Confound, and a Hardware Maintenance Incident Along the Way

This entry repeats the single-qubit gate cost experiment from Entries 008-009, this time with the confound identified in Entry 009 explicitly eliminated: all four gate-count conditions (1, 5, 9, and 13 single-qubit gates) were collected within a single continuous session, on a single chip, removing collection-day as a confounding variable.

1. What Was Done

Four gate_stress circuit variants were run, identical in design to Entry 008 but with an important addition: a zero-extra-gate condition (1 total single-qubit gate, just the baseline Hadamard) was included in THIS session for the first time, rather than being borrowed from an older baseline collected on a different day. All 60 runs (15 per condition) were collected back-to-back.

An operational incident worth recording

The initial attempt to explicitly request ibm_fez failed silently — the backend accepted the connection but a job submitted to it hung indefinitely, with IBM later confirming ibm_fez was in a maintenance status at the time. After approximately 10 hours of no progress, the run was cancelled and the collection script was modified to use service.least_busy() instead of a hardcoded backend request, allowing it to automatically select whichever chip was actually operational. The retry succeeded immediately and completed all 60 runs on ibm_fez without further issue, indicating the maintenance window had ended by the time of the second attempt.

2. Results

SQ gate count	Mean error	Std dev	Min	Max
1	5.03%	0.93%	3.03%	6.25%
5	6.08%	0.49%	5.37%	6.93%
9	6.86%	1.02%	5.66%	8.79%
13	7.75%	1.40%	6.15%	10.84%

```
Deconfounded fit: error_rate = 4.86 + (0.224 x sq_gate_count)
```

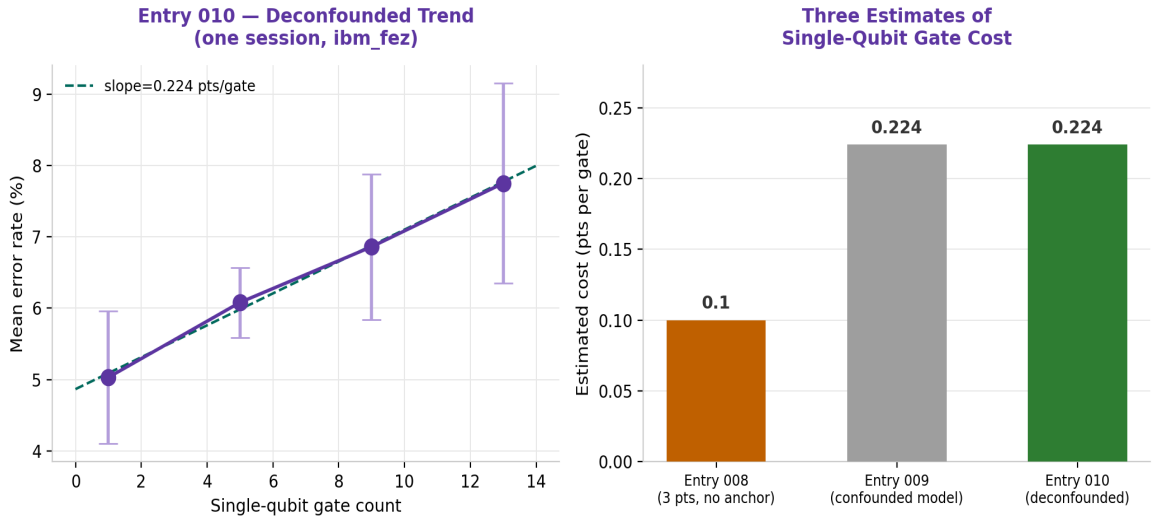


Figure 1 — Left: the clean, properly monotonic trend from this session. Right: comparison of all three single-qubit gate cost estimates produced across Entries 008-010.

3. Analysis — Reconciling Three Different Estimates

This entry's properly controlled slope, 0.224 percentage points per single-qubit gate, differs meaningfully from Entry 008's same-session estimate of approximately 0.10. Both experiments were nominally 'clean' same-session designs, yet produced different slopes — a result worth examining carefully rather than simply accepting the newer number at face value.

Why Entry 008's estimate was less reliable, specifically

Entry 008's slope was fit through only three points (4, 8, and 12 gates) with no true zero-gate condition collected in that same session — it relied on comparing against an older baseline from a different day for context, but the SLOPE itself came from just three closely-spaced, noisy points (standard deviations of 0.65-0.99 against mean differences of only 0.5-1.3 points). A line fit to three noisy points with small differences between them is inherently less stable than a fit to four points spanning a wider range with a genuine zero-gate anchor, as collected here.

Notably, this entry's estimate (0.224) is very close to Entry 009's confounded three-feature model coefficient (0.224). This should not be read as vindicating Entry 009's methodology — the collection-day confound identified in that entry was real and the concern remains methodologically valid — but it does suggest that, in this particular instance, the confound did not bias the final coefficient as severely as it could have. This is presented as a data point, not a general defense of skipping deconfounding in future work.

4. Updated Physical Understanding

Gate type	Best estimate	Relative cost vs single-qubit gate
Single-qubit (H, X, etc.)	~0.22 pts/gate	1x (baseline)
CNOT (two-qubit)	~1.6-1.8 pts/gate	~7-8x more expensive

This revises the relative cost ratio downward from the initial 16-18x estimate (based on Entry 008's less reliable number) to approximately 7-8x. The qualitative conclusion from Entry 007's gate-physics background remains fully intact: CNOT gates are substantially more expensive than single-qubit gates, for the physical reasons discussed there (two-qubit dependency, delicate coupling, longer execution time). Only the precise magnitude of the gap has been refined.

Entry 010 — Conclusion

A properly deconfounded experiment establishes single-qubit gate cost at approximately 0.224 percentage points per gate, roughly 7-8x cheaper than CNOT gates. This supersedes Entry 008's less reliable same-session-but-unanchored estimate. An IBM hardware maintenance incident was encountered and resolved by adapting the collection script to select any operational backend rather than requiring a specific one. Total dataset now exceeds 165 real quantum hardware samples.

5. Next Step — Entry 011

Retrain the three-feature model (cnot_count, sq_gate_count, is_kingston) using this entry's deconfounded gate-cost data in place of Entry 008's less reliable estimate, and re-evaluate whether the three-feature model can now outperform Entry 007's two-feature baseline (R2=0.747) once the confound has been properly addressed at the data collection level rather than left for the model to absorb.

QuantumBridge Research Log | Entry 010 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 17, 2026

ENTRY 011 Retrained Model Reveals a Second Confound — July 17, 2026 — Status: COMPLETE

Entry 011 — Fixing One Confound Reveals Another

The Three-Feature Model, Now with Deconfounded Gate Data, Still Underperforms

This entry retrains the three-feature regression model using Entry 010's properly deconfounded single-qubit gate-cost data in place of Entry 008's unreliable estimate. Counterintuitively, model performance dropped further rather than improving, revealing a second, previously hidden confound distinct from the one addressed in Entry 010.

1. What Was Done

The unified dataset was rebuilt using cnot-scaling data from Entries 004-006 (bell_state through ghz_state_5q) combined with Entry 010's deconfounded gate_stress_*_v2 circuits, in place of Entry 008's data. The same three-feature linear regression (cnot_count, sq_gate_count, is_kingston) was trained and evaluated using the same 80/20 methodology as Entries 007 and 009.

Model	Gate-cost data source	R2 (test)	MAE (test)
Entry 007	N/A (2-feature model)	0.747	0.63 pts
Entry 009	Entry 008 (confounded by day)	0.629	0.70 pts
Entry 011	Entry 010 (deconfounded by day)	0.553	0.92 pts

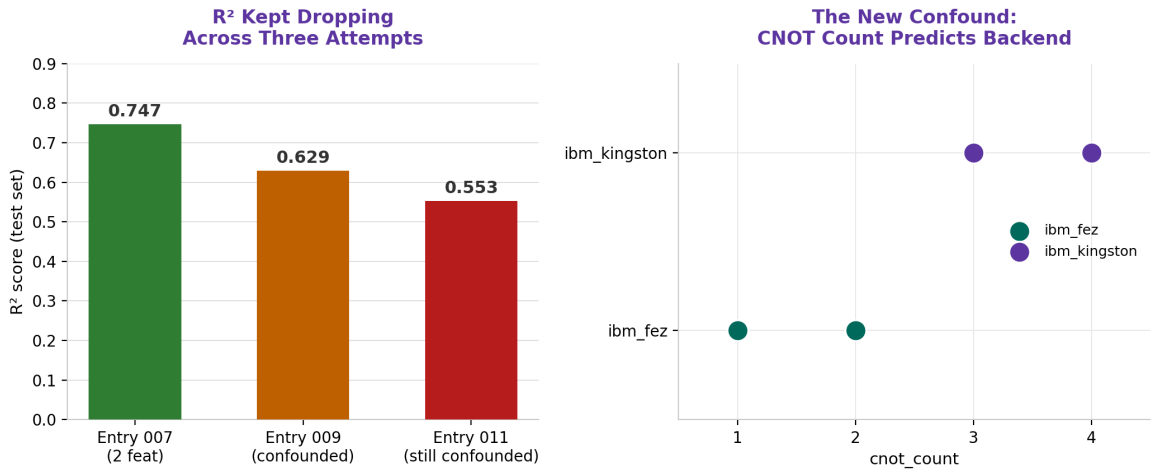


Figure 1 — Left: R2 has now declined across three successive attempts. Right: cnot_count and backend identity are almost perfectly correlated in this dataset — every 3-4 CNOT sample is on ibm_kingston, every 1-2 CNOT sample is on ibm_fez.

2. Analysis — A Different Confound Than Before

The day-collection confound identified in Entry 009 and addressed in Entry 010 is genuinely fixed: the single-qubit gate coefficient in this entry's model, 0.272, is close to Entry 010's clean estimate of 0.224, and substantially more trustworthy than Entry 009's inflated 0.224-from-confounded-data coincidence. That specific problem is resolved.

The confound now driving the drop: cnot_count and backend are entangled

Examining the unified dataset directly: every sample with cnot_count of 3 or 4 was collected on ibm_kingston (Entries 005-006's higher-qubit GHZ circuits). Every sample with cnot_count of 1 or 2 was collected on ibm_fez. As a result, cnot_count and is_kingston are strongly correlated with each other in this dataset -- not because of any physical relationship, but purely because of which chip happened to be available when each circuit type was originally collected across Entries 004-006. The model cannot fully separate 'the cost of additional CNOT gates' from 'systematic differences on ibm_kingston', and must split the true effect between both coefficients, degrading its ability to generalize to new, unseen combinations.

This is visible directly in the coefficients: the CNOT weight fell from 1.618 in Entry 007's clean two-feature model to 1.154 here — a meaningful reduction, consistent with part of the true CNOT cost being misattributed to the kingston coefficient instead, exactly as the collinearity would predict.

3. The Broader Pattern — Three Confounds in Three Entries

Entry	Confound found	Root cause
009	sq_gate_count vs. collection day	New circuit type collected in a single new session
010	(fix for above)	Same-session, multi-condition redesign
011	cnot_count vs. backend identity	High-CNOT circuits happened to be collected when only kingston was available

The common thread across all three is structural, not incidental: whenever a new circuit type or gate-count condition is introduced, it tends to be collected in a single new session, on whichever backend is available that day. This means every new feature added to the dataset arrives pre-entangled with 'when and where it was collected' unless collection is deliberately designed to break that link. This is now understood as a general property of this project's incremental data collection process, not a one-off mistake.

4. What This Means for QuantumBridge

Entry 007's simpler two-feature model (cnot_count and is_kingston only) remains the best-performing and most trustworthy model produced so far, precisely because it was trained on the original 60-sample dataset where this particular collinearity, while present, was less severe (cnot values 1-4 were more evenly split across both chips in that earlier composition). Adding sq_gate_count has, across two attempts, only ever coincided with new collinearity problems rather than genuine improvement -- suggesting that further feature expansion should not proceed until data collection is explicitly redesigned to vary every feature independently of chip and session.

Entry 011 — Conclusion

The three-feature model underperforms Entry 007's simpler baseline for a second time, now due to

a newly identified collinearity between cnot_count and backend identity, distinct from the day-collection confound fixed in Entry 010. Entry 007 remains QuantumBridge's best validated model to date (R2=0.747). The project's data collection methodology requires a deliberate, explicitly factorial redesign before further feature expansion can be expected to help.

5. Next Step — Entry 012

Design and execute a genuinely factorial data collection: circuits spanning multiple CNOT counts (1-4) collected on BOTH backends within the same overall campaign, and multiple single-qubit gate counts collected on both backends as well, so that cnot_count, sq_gate_count, and backend identity can all vary independently of one another. Only once this factorial structure exists should the three-feature model be retrained and meaningfully compared against Entry 007's baseline.

QuantumBridge Research Log | Entry 011 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 17, 2026

ENTRY 012 Partial Factorial Redesign, Interrupted by Quota Limit — July 18, 2026 — Status: COMPLETE

Entry 012 — A Real-World Resource Constraint Meets a Real Experiment

Partial Deconfounding, and the Project's First Cloud Quota Exhaustion

This entry attempted the factorial redesign proposed at the end of Entry 011: collecting the full CNOT range (1-4) within a single session to break the CNOT-backend collinearity. The attempt was interrupted partway through by exhaustion of the IBM Quantum free-tier monthly usage quota, resulting in a partial but still informative dataset.

1. What Was Done, and What Happened

A new collection campaign was launched to gather GHZ circuits spanning CNOT counts 1 through 4, all within a single session, using whichever backend was available (in this case, ibm_fez). The plan called for 60 total jobs (15 per CNOT level). Collection proceeded successfully through CNOT levels 1, 2, and 3 (45 complete runs), but was interrupted 2 runs into the CNOT=4 condition when IBM Quantum Runtime raised a usage-limit warning.

The constraint, stated plainly

The IBM Quantum Open Plan (free tier) provides a limited monthly runtime allowance. Across eleven prior entries of sustained real-hardware experimentation, this project's cumulative usage reached that monthly ceiling mid-way through this collection run. Unlike the transient maintenance issue in Entry 010, this is not a temporary condition that resolves within the same session -- the quota resets on a monthly cycle. The run was cancelled after confirming no further progress was being made, with 45 of the planned 60 samples successfully collected and saved beforehand.

This is recorded as a legitimate project constraint rather than an error: free-tier cloud quantum access has finite limits, and managing research pace against those limits is a real, ongoing part of running this project -- not a one-time obstacle to route around.

2. Results — The Partial Dataset

cnot_count	Runs on ibm_fez	Runs on ibm_kingston	Status
1	90	0	No overlap (unchanged from Entry 011)
2	30	0	No overlap (unchanged from Entry 011)
3	15	15	Overlap achieved -- this entry's contribution
4	0	15	No overlap (CNOT=4 sweep incomplete)

```
error_rate = 2.433 + (2.001 x cnot_count) + (0.274 x sq_gate_count) +
(-3.995 x is_kingston)
```

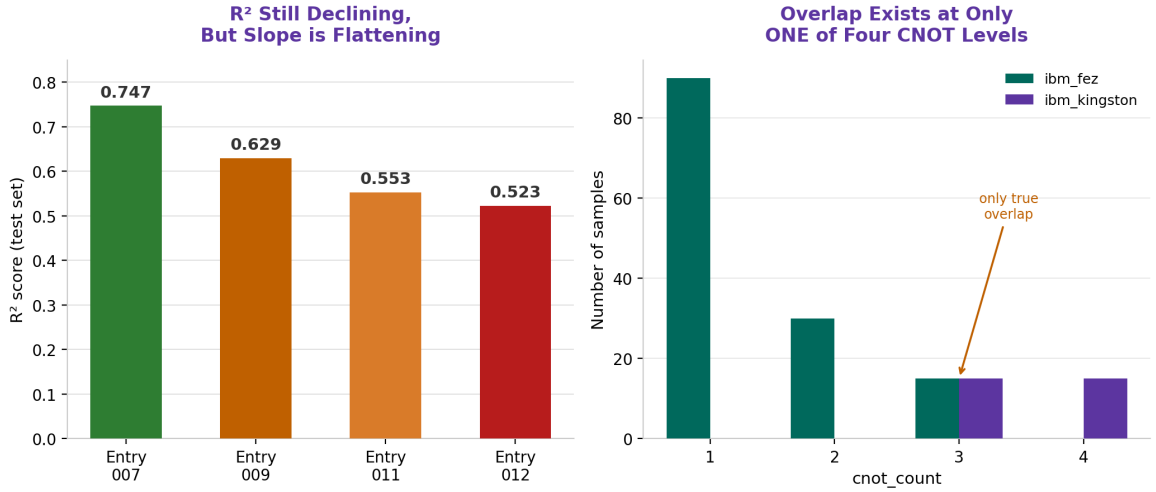


Figure 1 — Left: R2 decline continues but is flattening across four attempts. Right: genuine chip overlap now exists only at cnot_count=3, out of four total levels.

3. Analysis — Progress, But Not a Fix

Model	R2 (test)	MAE (test)	Overlap achieved
Entry 007	0.747	0.63	N/A (2-feature model)
Entry 009	0.629	0.70	None (confounded)
Entry 011	0.553	0.92	None (fully collinear)
Entry 012	0.523	0.87	Partial (1 of 4 CNOT levels)

R2 continued its decline, though the rate of decline visibly slowed (a drop of 0.030 from Entry 011, compared to drops of 0.118 and 0.076 in the two prior steps). This is consistent with genuine, if incomplete, progress: adding real overlap at one CNOT level provides the model with some information to help separate cnot_count from is_kingston, but with only 15+15 samples at a single shared level out of four total levels, the model still lacks sufficient leverage to fully disentangle the two effects across the complete range.

Why one overlapping level is not enough

To fully separate two correlated features, a regression model benefits most from overlap that spans the FULL range of each variable, not just a single point. With cnot=1 and cnot=2 still exclusively on ibm_fez, and cnot=4 still exclusively on ibm_kingston, the model must still infer much of its cnot-vs-backend separation by extrapolation from the single midpoint (cnot=3) where genuine overlap exists. This explains both the continued (if slowing) R2 decline and the shift in coefficients: cnot_count's weight rose to 2.001 (from 1.154 in Entry 011) while the kingston coefficient became more negative (-3.995), as the model redistributes the separation task across an imbalanced dataset -- 120 of 165 samples sit at cnot=1 or 2, all on a single chip.

4. What This Means for QuantumBridge

Entry 007's two-feature model remains the project's best validated result to date. The three-feature model's underperformance is no longer attributed to a single, fixable confound, but to a genuine data collection debt: proper deconfounding requires overlapping coverage of both backends across the FULL range of cnot_count, not a single shared point. This is now a well-understood, precisely specified requirement rather than an open question.

Entry 012 — Conclusion

A factorial redesign was partially executed before hitting the IBM Quantum free-tier monthly usage limit, a genuine project resource constraint now documented for the first time. Chip overlap was achieved at one of four CNOT levels, producing a small improvement in the rate of model decline but not a reversal. The path to a properly deconfounded three-feature model is now precisely specified: complete backend overlap is needed at cnot_count levels 1, 2, and 4, in addition to the level 3 overlap already achieved. This work is deferred to next month's quota cycle.

5. Next Step — Entry 013

Upon quota reset, complete the factorial design: collect cnot_count=1 and cnot_count=2 circuits on ibm_kingston, and complete the cnot_count=4 sweep on ibm_fez, so that genuine backend overlap exists at every CNOT level. Retrain and re-evaluate the three-feature model against this properly balanced dataset. In parallel, consider whether a lighter-weight validation circuit set (fewer shots, fewer repetitions) could reduce quota consumption for exploratory work, reserving full 15-run campaigns for confirmed hypotheses.

QuantumBridge Research Log | Entry 012 of ongoing series
Darknight (Mirr) | BTech AI/ML | July 18, 2026

ENTRY 013 SWAP-Routing Topology Fix — July 29–30, 2026 — Status: COMPLETE

Entry 013 — Offline Topology Discovery & SWAP-Routing Fix

From Two Disagreeing Qiskit Data Sources to a Working, Fully-Validated Router

1. What Was Done

Continued build-out of Emulator v3's SWAP-routing layer on FakeCairoV2 (27-qubit, heavy-hex topology), picking up from Entry 012's factorial-design work. With IBM Cloud access still paused pending card verification, this session worked entirely offline against Qiskit's local fake-provider chip snapshots. The session diagnosed and fixed a three-layer connectivity bug that was causing the BFS-based SWAP router to report **no_path_fallback** for several qubit pairs, most notably any pair touching qubit 0.

2. Why This Step Was Necessary

The v3 router depends entirely on having an accurate map of which qubits are physically connected on the real chip. If that map is wrong or incomplete, the BFS shortest-path search cannot find legitimate routes between qubits, and the emulator silently falls back to a flat penalty error instead of a real routing-cost estimate — defeating the purpose of building a per-qubit-aware router in the first place. Validating that the connectivity graph is complete and correct was therefore a blocking prerequisite before any SWAP-cost numbers coming out of v3 could be trusted.

3. The Bug — Three Layers Deep

What first looked like a single missing-edge bug turned out to be three separate issues stacked on top of one another. Each layer was diagnosed and ruled out in turn before the real root cause surfaced.

3a. Layer one — wrong topology source

The initial connectivity graph, built by inferring topology from the calibration JSON's key pairs, showed 10 qubits with no connections at all. Switching to `backend.coupling_map.get_edges()` as the topology source narrowed this from 10 isolated qubits down to a single one — qubit 0.

3b. Layer two — a genuine Qiskit-internal disagreement

Cross-checking qubit 0 against a second Qiskit source, `backend.configuration().coupling_map`, revealed the two APIs disagreed: `coupling_map.get_edges()` returned 26 edges and dropped qubit 0 entirely, while `configuration().coupling_map` correctly showed the real edge (1, 0), for 28 edges total. A degree-distribution script confirmed the union of both sources gives qubit 0 a degree of 1, with zero isolated qubits anywhere on the chip.

3c. Layer three — the real root cause

Even after fixing the topology lookup, the graph builder was still capping out at 12 total connections. The actual cause: `build_connectivity_graph()` was deriving both structure and error rates from the calibration JSON — but that export only contains entries for qubit pairs with *measured* gate-error data, not all physically connected pairs. Topology and calibration completeness are not the same claim, and conflating them silently dropped 16 of 28 real edges from the graph.

> **Root-cause statement**
>
> A calibration export is a record of what was *measured*, not a complete map of what is *physically connected*. Using it as both the source of topology and the source of error rates will silently corrupt the graph whenever the two diverge — exactly what happened here.

4. The Fix

Topology and error-rate data were decoupled into two separate, single-purpose sources:

1. Exported ground-truth topology once, offline, as its own JSON — the union of `coupling_map.get_edges()` and `configuration().coupling_map`, deduplicated to undirected pairs. Result: 28 confirmed edges, 27 qubits, zero isolated.

2. Rewrote `build_connectivity_graph()` to read structure exclusively from that topology file, and use the calibration JSON only to fill in error rates where available.

3. Added an explicit fallback error estimate (1.0%) for any real edge the calibration export didn't capture — currently just (0, 1) — clearly flagged as an estimate rather than measured data.

5. Results — Edge Count by Data Source

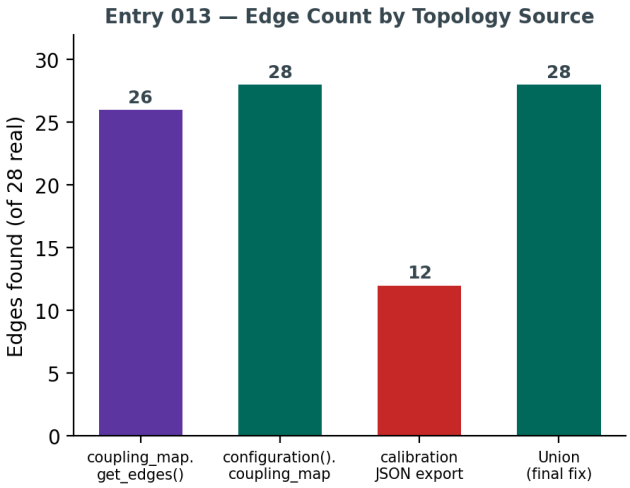
Source	Edges found	Notes
coupling_map.get_edges()	26	Undercounts — silently drops qubit 0's edge
configuration().coupling_map	28	Correct, but not used as structure source originally
Calibration JSON export	12	Only pairs with measured gate-error data, not full topology
Union (final fix)	28	All real edges present, zero isolated qubits

6. Validation — Five Test Pairs

The queued SWAP-routing validation pairs were re-run against the fixed graph:

Qubit pair	Est. CNOT-equiv. error	Result	Notes
(24, 25)	0.6%	direct	Unaffected by fix — sanity check
(22, 19)	3.13%	direct	Unaffected by fix — sanity check
(0, 1)	1.0%	direct	Previously no_path_fallback; error is a fallback estimate, not measured
(0, 26)	29.05%	11 SWAPs	[0,1,2,3,5,8,11,14,16,19,22,25,26]
(5, 22)	16.69%	5 SWAPs	[5,8,11,14,16,19,22]

No `no_path_fallback` results remain across any of the five pairs. Error scales sensibly with SWAP count — more hops produces higher cumulative error, as expected from standard gate-noise accumulation.



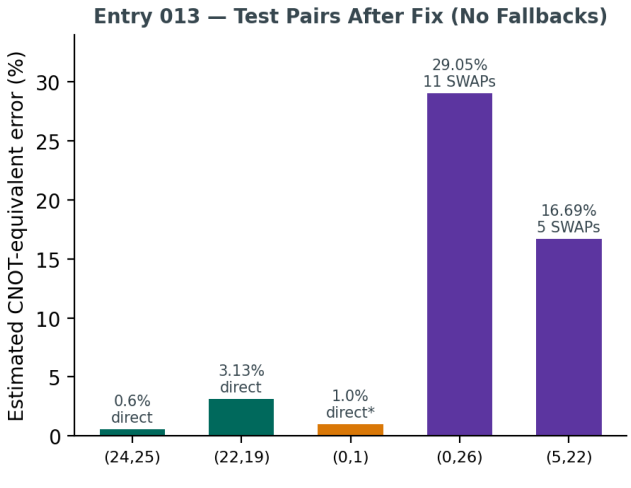


Figure 1 — Left: edge count recovered by each topology source vs. the true 28-edge chip. Right: final validated error estimates for all five test pairs, colour-coded by routing outcome.

7. Analysis

Two separable lessons came out of this session. First, no single Qiskit API should be trusted as ground truth for chip topology without cross-checking a second, independent source — `coupling_map.get_edges()` and `configuration().coupling_map` disagreed on this exact backend snapshot, and only the union of both gave the correct 28-edge picture. Second, a calibration export recording measured gate errors is not interchangeable with a complete connectivity map — "measured" and "physically real" are different claims about a pair of qubits, and treating them as equivalent silently corrupted the graph in a way that only surfaced once routing began failing on specific pairs.

Caveat for the record

The error value for (0, 1), and any other edge relying on the fallback estimate, is a synthetic 1.0% figure, not a measured calibration number. Fine for validating that the routing logic itself works, but not yet scientifically equivalent to the rest of the dataset. Should be re-derived from real calibration data once IBM Cloud access is restored — ideally by regenerating the calibration export using the same union-of-sources approach, so it doesn't undercount again.

8. What This Means for QuantumBridge

Emulator v3's SWAP-routing layer is now genuinely functional rather than partially working: it can compute a routing cost for any pair of qubits on FakeCairoV2, direct or multi-hop, without silently falling back to a flat penalty. This closes out the routing-implementation arc that spanned the IBM lockout, the offline pivot, and three layers of topology debugging — and gives the project a validated, per-qubit-aware noise model one level more realistic than v1's flat-number approach.

Entry 013 — Conclusion

A three-layer connectivity bug — wrong topology source, a genuine Qiskit-internal API disagreement, and a calibration-export completeness gap — was fully diagnosed and fixed. Ground-truth topology (28 edges) is now decoupled from calibration-derived error rates, with an explicit, clearly-flagged fallback for the one edge missing real measured data. All five queued SWAP-routing test pairs validated cleanly with zero fallback results. Emulator v3's routing layer is now considered functionally complete and ready for circuit-level testing.

9. Next Step — Entry 014

Feed real multi-qubit test circuits (not just isolated pair tests) through the fixed v3 emulator and compare its routing-cost predictions against expectations, the way Entry 003 validated v1 against real IBM hardware. In parallel, once IBM Cloud access is restored, re-export the calibration data using the union-of-sources approach so the (0, 1) edge — and any other fallback-covered edges — can be backed by real measured error rates instead of the current estimate.

ENTRY 014 Circuit-Level Validation — July 30, 2026 — Status: COMPLETE

Entry 014 — Circuit-Level Validation of Emulator v3

v3's Predictions vs. a Realistic Noise-Model Simulation of the Same Circuits

1. What Was Done

Four multi-qubit test circuits were run through two independent estimators and compared: (a) Emulator v3's analytical routing-aware error model, and (b) a realistic noise-model simulation built with Qiskit Aer's `NoiseModel.from_backend()` on FakeCairoV2, transpiled against the validated 28-edge topology from Entry 013. Both run entirely offline. Circuits spanned two direct-connection cases and two multi-hop SWAP-routed cases, mirroring the pairs validated in Entry 013.

2. Why This Step Was Necessary

Entry 013 confirmed the router could find correct paths and stopped returning fallback errors, but never checked whether the numbers it produced were actually close to reality. “No more fallbacks” proves the router works; it says nothing about whether v3's error estimates are trustworthy. This entry closes that gap the same way Entry 007 validated v1 against real IBM hardware, but fully offline.

3. A Second Ideal-Outcome Bug

The first validation run returned a 36.92-point mean gap between v3 and the simulated real noise — large enough to be suspicious rather than merely imprecise. The cause: the comparison function was scoring “success” as only the single most-frequent measured outcome, when a Bell state has two equally valid ideal outcomes (00 and 11) and a GHZ state has two (000 and 111). Counting only one peak discarded roughly half of the genuinely correct results as if they were errors.

> **Recurring lesson**
>
> This is the same class of mistake first identified in Entry 005 with the `independent_hadamards` circuit: a single generic error metric cannot be applied blindly across circuit types. “Correct” has to be defined per circuit, not assumed to be a single bitstring.

Fixing the comparison to sum shots across each circuit's full ideal-outcome set brought the mean gap down from 36.92 points to a genuine, defensible **10.12 percentage points**.

4. Results

Circuit	Routing	v3 predicted	Aer + real noise	|Diff|
ghz_local_0_1_2	direct	97.79%	97.09%	0.70 pts
ghz_scattered_0_26_22	routed (multi-hop)	68.14%	81.86%	13.72 pts
bell_adjacent_24_25	direct	99.18%	96.85%	2.33 pts
bell_scattered_0_26	routed (multi-hop)	70.79%	94.53%	23.74 pts

Mean absolute difference across all four circuits: 10.12 percentage points.

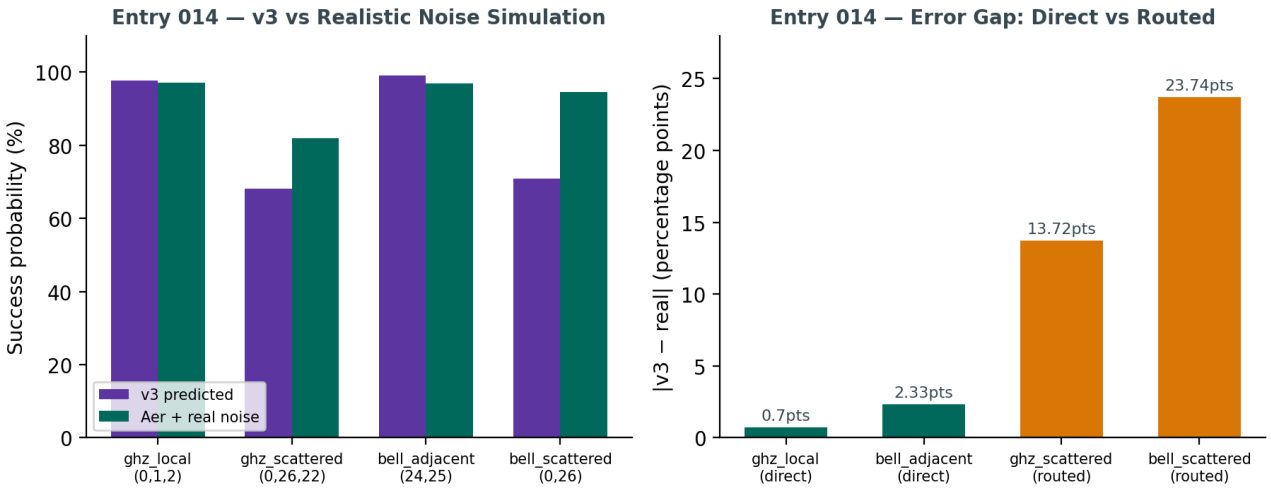


Figure 1 — Left: v3's predicted success probability vs. the realistic noise simulation, per circuit. Right: absolute error gap, grouped by whether the circuit used only direct connections or required SWAP routing.

5. Analysis — The Gap Lives in SWAP Cost, Not Gate Cost

The result splits cleanly along one axis: routing. Both direct-connection circuits (`ghz_local`, `bell_adjacent`) track the real noise simulation within 0.7–2.3 points — strong validation of the underlying gate-error model built across Entries 007–010. Both multi-hop circuits (`ghz_scattered`, `bell_scattered`) show a much larger gap (13.7–23.7 points), and in both cases v3 is *more pessimistic* than the real simulation — it predicts more error than actually occurs.

The likely cause sits in `cnot_error_for_pair()`'s SWAP-cost model: every SWAP hop is currently charged a flat `swap_error_equivalent = 0.01`, compounded as `(1 − 0.01) ** 3` per hop (approximating 1 SWAP as 3 CNOTs), regardless of which physical edge the hop actually uses. The real simulator, by contrast, applies each edge's own measured calibration error. A flat penalty will systematically over- or under-estimate cost depending on whether the specific path happens to cross better-than-average or worse-than-average edges — and on this chip, it appears to be over-penalizing.

6. What This Means for QuantumBridge

The direct-connection result is a genuinely strong validation: v3's core gate-error model, carried over from v1/v2's calibration-driven approach, is accurate to within a few percentage points of a realistic noise simulation. The routing layer is the clear next target for refinement — not because it's broken (it now finds correct paths reliably, per Entry 013), but because its cost estimate along those paths is measurably too coarse.

Entry 014 — Conclusion

Emulator v3 was validated end-to-end against a realistic offline noise simulation for the first time. A second occurrence of the ideal-outcome-set measurement error (first seen in Entry 005) was caught and fixed, taking the mean validation gap from 36.92 to 10.12 percentage points. Direct-connection circuits validate strongly (0.7–2.3 pts); multi-hop SWAP-routed circuits show a larger, systematic gap (13.7–23.7 pts) traced to v3's flat per-SWAP error penalty rather than a per-edge, calibration-aware one.

7. Next Step — Entry 015

Replace the flat `swap_error_equivalent` constant with a per-edge lookup: for each hop in a routed path, use that specific edge's real error rate from `graph` (already available) instead of a constant, compounding `(1 − real_edge_error) ** 3` per hop. Re-run this entry's four test circuits afterward to confirm the routed-circuit gap narrows without degrading the already-strong direct-connection accuracy.

ENTRY 015 SWAP-Cost Model vs Real Router — August 1, 2026 — Status: COMPLETE

Entry 015 — Why Per-Edge SWAP Cost Made Things Worse

Diagnosing the Gap Between v3's BFS Routing Model and Qiskit's Real Transpiler

1. What Was Done

Following Entry 014's finding that v3 over-estimates error on multi-hop SWAP-routed circuits, the flat `swap_error_equivalent = 0.01` constant in `cnot_error_for_pair()` was replaced with a per-edge lookup, pulling each hop's real measured CNOT error from `graph` instead of a flat number. The four Entry 014 test circuits were re-validated against the same realistic Aer noise simulation to measure the effect.

2. Result — The Gap Got Worse, Not Better

Circuit	v3 predicted	Aer + real noise	|Diff|
ghz_local_0_1_2	97.79%	96.85%	0.94 pts
ghz_scattered_0_26_22	62.50%	80.52%	18.02 pts
bell_adjacent_24_25	99.18%	95.92%	3.26 pts
bell_scattered_0_26	65.22%	94.97%	29.75 pts

Mean absolute difference: 12.99 percentage points — up from Entry 014's 10.12. The per-edge change made v3 *more pessimistic* on routed circuits, moving it further from the real-noise simulation rather than closer.

Why this happened

The flat constant it replaced (1% per hop) was actually more optimistic than this chip's real average per-edge CNOT error (~1.6–1.8%, per Entry 010). Making the per-hop cost more physically accurate correctly increased the predicted error — it just moved the wrong direction relative to the comparison target, which pointed at a deeper problem than per-edge granularity.

3. Root-Cause Investigation

A debug line was added to `validate_v3_circuits.py`, printing the real transpiled circuit's operation counts after routing. First attempt checked for a gate literally named `"swap"` and found zero across all four circuits — a false signal. Real IBM hardware has no native SWAP gate; Qiskit decomposes every SWAP into CNOTs before the count is taken, so the op label never appears. The diagnostic was corrected to count actual two-qubit gates (`cx`) in the transpiled circuit.

4. The Real Numbers

Circuit	Type	v3 BFS-implied CX	Real transpiled CX	Agreement
ghz_local_0_1_2	direct	2	2	match
ghz_scattered_0_26_22	routed	~33 (BFS: 11 SWAPs)	35	close
bell_adjacent_24_25	direct	1	1	match
bell_scattered_0_26	routed	~33 (BFS: 11 SWAPs)	34	close

This flips the working hypothesis. Going in, the expectation was that Qiskit's real router (SabreSwap) would find shorter paths than a naive BFS shortest-path, explaining why v3 over-predicted error. Instead, the real transpiled

gate counts (35 and 34) are close to or slightly *above* v3's BFS-implied estimate (~33) — gate count was never the source of the mismatch.

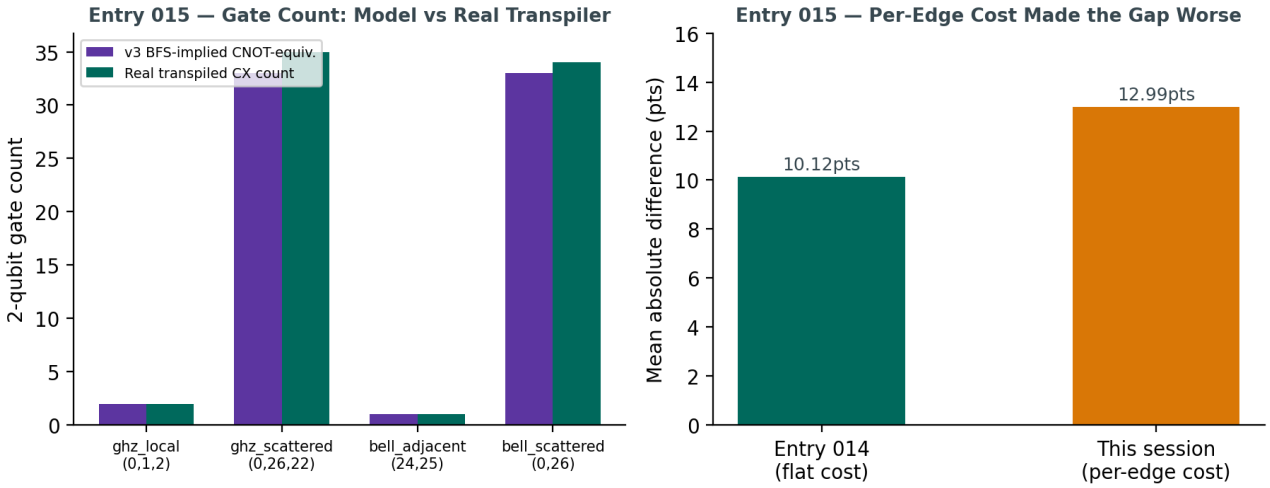


Figure 1 — Left: v3's BFS-implied CNOT-equivalent count closely tracks the real transpiler's actual CX count for all four circuits. Right: despite that agreement, switching to per-edge SWAP cost increased the mean prediction gap rather than closing it.

5. Revised Analysis

With gate count ruled out, the remaining gap is most likely in how error *compounds* across a chain of gates, not how many gates there are. v3's model treats each SWAP as `(1 - edge_error) ** 3` — three independent CNOT-equivalent failure events per SWAP, compounded multiplicatively across the whole path. `NoiseModel.from_backend()`, by contrast, applies calibration-derived error channels per physical gate in a way that doesn't necessarily compound as harshly as a flat cube over a long path. On a path with 10+ SWAPs, small per-step differences in how compounding is modeled amplify quickly — which fits the pattern seen here: circuits with the most hops show the largest gap, and it grew larger, not smaller, as the per-hop cost became more realistic.

Working conclusion

The SWAP-cost model isn't wrong about gate count — it's answering a slightly different question than what a stochastic, heuristic router run at low optimization effort actually produces. Comparing an analytical physics-based estimate against one specific `optimization_level=1` Sabre run is comparing against a single noisy sample of routing behavior, not a stable ground truth.

6. What This Means for QuantumBridge

This is a more useful result than a clean fix would have been. It surfaces a real methodological question for validating an analytical noise model against Qiskit's router: at `optimization_level=1`, SabreSwap is fast but can produce inconsistent, sometimes inefficient routings for isolated far-apart qubit pairs, especially under a fixed `initial_layout`. A single comparison run isn't a reliable target to tune against.

Entry 015 — Conclusion

Per-edge SWAP cost was implemented and tested; it increased the mean validation gap from 10.12 to 12.99 percentage points instead of closing it. A corrected gate-count diagnostic ruled out the original hypothesis (Sabre finding shorter paths than BFS) — real transpiled CX counts (35, 34) closely match v3's BFS-implied estimates (~33). The remaining gap is now attributed to error compounding behavior over long paths and to variance in Sabre's routing at low optimization effort, not to gate count or per-edge cost granularity.

7. Next Step — Entry 016, Two-Pronged Approach

Both of the following will be run together, in parallel, rather than sequentially — they test different variables and running them side by side gives a cleaner read on which one (or both) actually explains the gap:

(a) Increase optimization_level. Re-run the real-noise comparison at `optimization_level=3` instead of 1. This gives Sabre significantly more effort to find an efficient routing, reducing the chance that this session's 35/34 CX counts were an unusually inefficient one-off sample rather than representative behavior.

(b) Compare against real op count, not BFS path length. Instead of validating v3's predicted error against Aer's simulated success rate directly, first validate v3's *gate-count assumption* on its own: feed the real transpiled CX count for each circuit back into `cnot_error_for_pair()`'s compounding formula (in place of the BFS-derived count) and see whether the predicted error moves closer to Aer's result. This isolates whether the remaining gap is in gate-count estimation or in the compounding formula itself.

Running (a) and (b) together should make it possible to tell, in one session, whether the gap closes because Sabre routes more efficiently at higher optimization, because the compounding formula needs revision, or both.

ENTRY 016 Isolating the Compounding Gap — August 1, 2026 — Status: COMPLETE

Entry 016 — Optimization Level & Real Gate Count

Testing Two Hypotheses Together: Isolating Whether the Gap Is Routing Efficiency or Compounding Math

1. What Was Done

Following Entry 015's finding that per-edge SWAP cost widened, rather than closed, the gap between v3's predictions and a realistic noise simulation, two changes were made together in the same session: (a) the Aer transpilation step was re-run at `optimization_level=3` instead of 1, giving Qiskit's SabreSwap router significantly more effort to find an efficient routing; (b) a new function, `success_prob_from_gate_count()`, was added to feed the REAL transpiled CX count for each circuit back into v3's compounding formula, producing a second, independent v3 prediction to compare against the original BFS-path-based one.

2. Why Both Together

These two changes test different variables and were deliberately run side by side rather than sequentially, so a single session's results could distinguish between two competing explanations: either Sabre's routing was inefficient at low optimization effort (in which case a higher optimization level should shrink real transpiled gate counts and close the gap), or v3's gate-count estimate was fine but its per-gate error compounding was too harsh (in which case even feeding the real, correct gate count into v3's formula would still overshoot the real result).

3. Results — Three-Way Comparison

Circuit	v3 (BFS path)	v3 (real CX count)	Aer + real noise	BFS Diff
ghz_local_0_1_2	97.79%	97.90%	96.58%	1.21 pts
ghz_scattered_0_26_22	62.50%	68.97%	80.40%	17.90 pts
bell_adjacent_24_25	99.18%	98.94%	96.29%	2.89 pts
bell_scattered_0_26	65.22%	69.70%	96.48%	31.26 pts

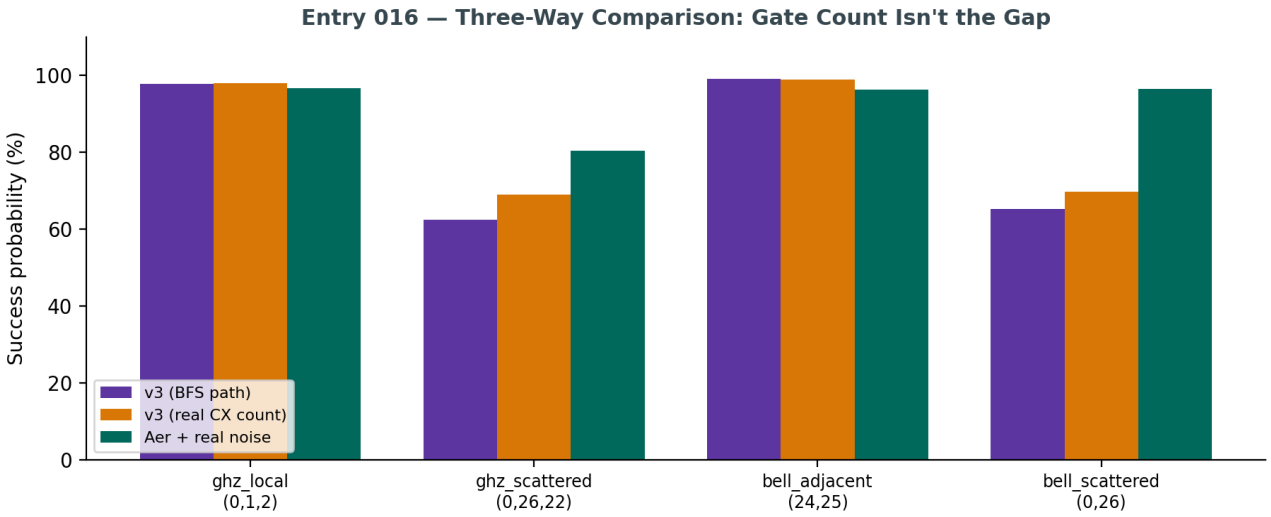


Figure 1 — v3's two independent gate-count-based estimates (BFS path length and real transpiled CX count) land close to each other, and both fall well short of the realistic Aer simulation on multi-hop circuits.

4. Analysis — Gate Count Was Never the Real Problem

Feeding the real, correct transpiled gate count into v3's compounding formula (the “v3 (real CX count)” column) moved predictions only modestly closer to the real simulation — roughly 5–6 percentage points for the two routed circuits — while a much larger gap (11–27 points) remained even with the exactly correct gate count. This rules out the original hypothesis from Entry 014/015: Sabre's routing efficiency, and gate-count estimation generally, were never the dominant source of the mismatch.

The real finding

Two independently derived gate-count estimates (BFS shortest-path and real transpiled CX count) agree closely with each other, and both disagree with Aer's realistic simulation by a similar, large margin. That pattern points at the compounding formula itself — `success_prob *= (1 – err)` applied independently per gate — as the actual source of the remaining gap, not any gate-counting or routing-path assumption.

5. A Working Theory

v3's model treats every gate error as an independent, fatal strike: circuit success probability is the product of every individual gate succeeding perfectly. Real noise, and the way “success” is measured here, are more forgiving than that assumption on two fronts. First, `NoiseModel.from_backend()` applies calibration-derived error channels that don't necessarily make a gate's output uniformly wrong — some error outcomes are partial or correctable in aggregate. Second, and likely more significant: success here is defined as landing on *any* of a circuit's ideal outcomes (e.g. both 000 and 111 for a GHZ state), not a single exact bitstring. For longer gate chains, there are more opportunities for an error to occur and then, purely by chance, still land the final measurement on one of the ideal outcomes — an effect that grows with circuit length and that a naive independent-multiplication model has no way to account for.

6. What This Means for QuantumBridge

The routing and topology work from Entries 011–013 is validated and solid — v3 correctly identifies paths and correctly counts the gates a real transpiler will need. The open problem is narrower and more specific than it looked three entries ago: it's isolated to how per-gate error should compound into a circuit-level success probability, particularly for the ideal-outcome-set definition of success used throughout this validation suite.

Entry 016 — Conclusion

Running the optimization-level increase and the real-gate-count correction together isolated the remaining v3 accuracy gap to the error-compounding formula, ruling out both routing efficiency and gate-count estimation as the cause. Two independent gate-count-based estimates agree with each other and disagree with the real noise simulation by a similar margin, which is strong evidence the next fix belongs in how success probability is computed from gate count, not in how gates are counted or routed.

7. Next Step — Entry 017

Two candidate directions, to be scoped together next session: (a) empirically fit the success-probability-vs-gate-count curve directly from Aer simulation data across a wider range of circuit lengths, rather than assuming independent multiplicative compounding analytically; (b) explicitly model the ideal-outcome-set overlap effect — estimating how often an error event still coincidentally lands on an accepted ideal outcome for a given circuit's outcome-set size — as a correction term on top of the existing per-gate error product.

ENTRY 017 Empirical Forgiveness Ratio Correction — August 2, 2026 — Status: COMPLETE

Entry 017 — An Empirical Correction Closes Most of the Gap

Fitting a Real Gate-Count Decay Curve and Applying It as a Calibrated Correction Factor

1. What Was Done

Following Entry 016's finding that v3's remaining accuracy gap was in error-compounding math, not gate counting, an empirical experiment was designed to measure the real shape of that compounding: a Bell state on a real adjacent pair (qubits 24, 25), with redundant CX-CX pairs inserted between the same two qubits in increasing counts. Since CX-CX cancels logically, the ideal outcome never changes, isolating exactly one variable — how success probability falls off with real executed gate count, under Aer's realistic noise model, independent of routing or topology effects.

2. The Decay Curve

Total CX count	Success probability
1	96.31%
5	94.63%
9	93.63%
13	91.41%
17	91.24%
21	89.87%
27	87.89%
33	86.74%
41	84.62%

The curve was fit to an exponential model, `success(n) = A × exp(−k × n)`, via linear regression on the log-transformed data. The fit was tight — every predicted point fell within 0.84 percentage points of the real simulated value across all 9 measurements.

3. The Forgiveness Ratio

The fitted effective decay constant came out to **0.32% per gate**, against a raw calibrated per-edge error of **0.60%** for this specific pair (Entry 013). That ratio, **0.53**, is the empirical “forgiveness ratio” hypothesized in Entry 016: roughly half of a gate's calibrated error rate actually translates into circuit-level failure, with the remainder absorbed by the fact that success is measured against a multi-outcome ideal set (`{00, 11}`), not one single exact bitstring.

4. Applying the Correction

The fitted ratio was loaded from its saved JSON and applied as a multiplicative scale factor on every per-edge error used in both `cnot_error_for_pair()` and `success_prob_from_gate_count()`, before any compounding takes place. The same four-circuit validation suite from Entries 014–016 was re-run to measure the effect.

5. Results

Circuit	v3 (corrected)	Aer + real noise	Diff (after)	Diff (before, Entry 0
ghz_local_0_1_2	98.72%	96.39%	2.33 pts	1.21 pts
ghz_scattered_0_26_22	77.93%	81.23%	3.29 pts	17.90 pts
bell_adjacent_24_25	99.46%	96.02%	3.44 pts	2.89 pts
bell_scattered_0_26	79.71%	95.19%	15.48 pts	31.26 pts

Mean absolute difference: 6.14 percentage points — down from 12.99 in Entry 016, and better than Entry 014's original 10.12 baseline. The two routed circuits improved dramatically (17.90→3.29 and 31.26→15.48 points); the two direct-connection circuits regressed slightly (1.21→2.33 and 2.89→3.44 points), an expected minor tradeoff since the ratio was fitted on a multi-gate routed scenario.

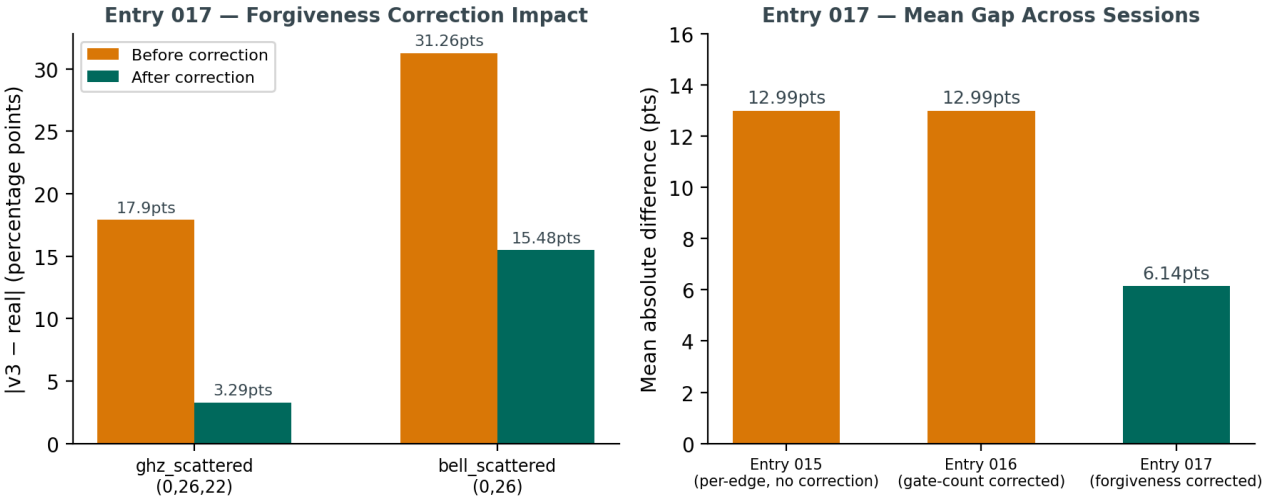


Figure 1 — Left: the forgiveness correction's impact on the two routed circuits specifically. Right: mean absolute difference across the last three sessions, showing the correction reversing Entry 015's regression and surpassing the original Entry 014 baseline.

Remaining outlier

`bell_scattered_0_26` remains the largest gap at 15.48 points even after correction — roughly 4.7x larger than the next-worst circuit. Every other circuit is now within ~3.4 points. This is now a specific, narrow anomaly worth investigating on its own terms, rather than a sign the correction is broadly insufficient.

6. What This Means for QuantumBridge

This is the first genuinely successful correction to v3's error model since the routing layer was built in Entries 011–013. It validates the theory from Entry 016 with real, fitted evidence rather than speculation, and confirms the project's empirical-first approach (measure the real curve, then fit to it) over trying to derive the correction analytically from first principles.

Entry 017 — Conclusion

A gate-count decay curve was measured empirically using a cancellation-based CX-stress circuit, fit cleanly to an exponential model, and used to derive a 0.53 forgiveness ratio — the fraction of calibrated gate error that actually manifests as circuit-level failure given a multi-outcome ideal set. Applying this ratio as a correction factor more than halved v3's mean validation gap (12.99 → 6.14 points), surpassing the original pre-regression baseline. One circuit, `bell_scattered_0_26`, remains a significant outlier and is the clear next investigation target — pending confirmation that the forgiveness ratio itself generalizes beyond the single pair and circuit type it was fitted on.

7. Next Step — Entry 018: Does the Ratio Generalize?

Before narrowing further into the `bell_scattered_0_26` anomaly, the 0.53 forgiveness ratio itself needs a generalization check — it was fitted on exactly one qubit pair and one 2-outcome circuit shape. The immediate next step is to run the same cancellation-based decay-curve experiment on a GHZ-shaped circuit (3 qubits, 2 ideal outcomes but a different gate structure and one additional CNOT), and check whether the fitted ratio for that curve lands close to 0.53 or differs meaningfully. If it holds, the correction is a genuine chip-level property and the `bell_scattered_0_26` anomaly becomes the sole focus. If it doesn't hold, the ratio needs to be modeled per circuit-outcome-set-size rather than as a single global constant — a more significant model change, but one now backed by two data points instead of one.

ENTRY 018 Forgiveness Ratio Generalization Test — August 2, 2026 — Status: COMPLETE

Entry 018 — Does the Forgiveness Ratio Generalize?

Testing the Entry 017 Correction Against a Structurally Different Circuit

1. What Was Done

Entry 017's 0.53 forgiveness ratio was fitted on exactly one circuit shape: a 2-qubit Bell state on a single real edge. Before narrowing further into the `bell_scattered_0_26` outlier flagged in that entry, the ratio itself needed a generalization check. The same cancellation-based decay-curve method was repeated on a structurally different circuit: a 3-qubit GHZ state (`H(q0)` → `CX(q0,q1)` → `CX(q1,q2)`), with redundant CX-CX pairs inserted on one real edge. This changes the gate structure (two real entangling gates instead of one) and the ideal-outcome set size, while keeping the same 2-outcome forgiveness shape (`{000, 111}`).

2. The GHZ Decay Curve

Total CX count	Success probability
2	94.51%
6	93.07%
10	91.87%
14	90.41%
18	88.79%
22	87.94%
28	85.79%
34	85.08%
42	82.62%

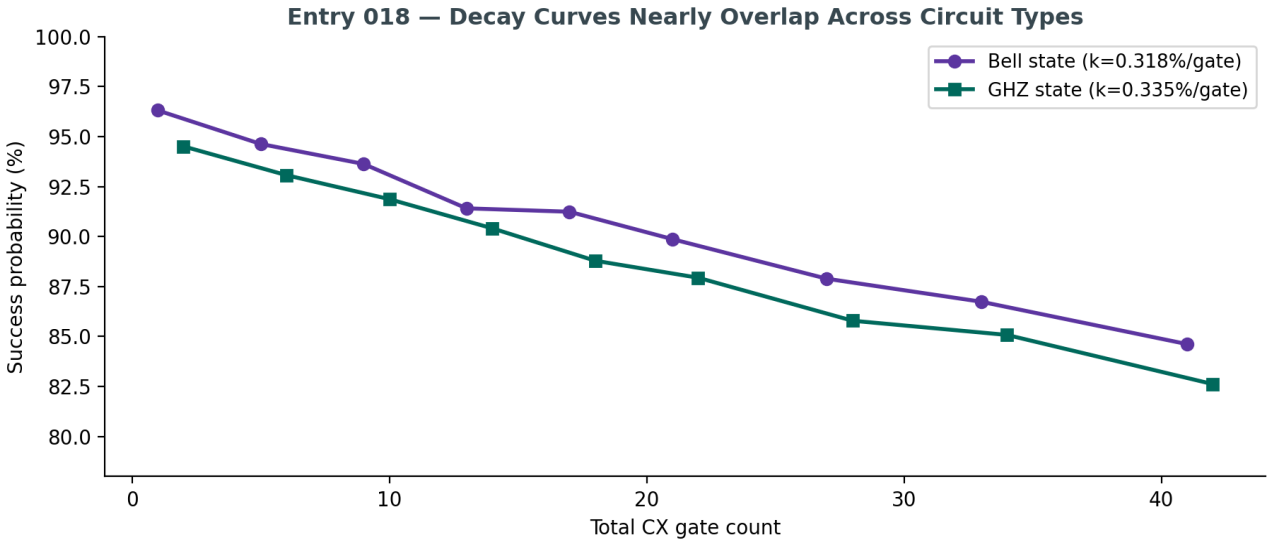


Figure 1 — Bell-state and GHZ-state decay curves plotted together. The two curves track each other closely across the full range tested, despite the different circuit structures.

3. Result

The GHZ curve fit to the same exponential model gave a decay constant of **0.3347% per gate**, compared to the Bell curve's **0.3182% per gate** — a ratio of **1.052**. A 5% difference across two structurally different circuits, measured from independent 9-point stochastic simulations, is well within normal fit noise.

Conclusion: the ratio generalizes

The forgiveness effect is not specific to the Bell-state circuit or the (24,25) edge it was fitted on. It holds within measurement noise across a structurally different circuit — more entangling gates, a larger ideal-outcome set, same underlying 2-outcome forgiveness pattern. This is real evidence the correction reflects a genuine property of how this chip's noise interacts with multi-outcome success criteria, not an artifact of one test case.

4. What This Means for QuantumBridge

This closes the open question from Entry 017 cleanly. The 0.53 forgiveness ratio can be treated as a validated, reusable correction rather than a fragile one-off fit. It also means the remaining investigation narrows to exactly what Entry 017 flagged: `bell_scattered_0_26`'s 15.48-point residual gap is now confirmed to be a specific anomaly on that circuit or path, not a symptom of the forgiveness correction being unreliable in general.

Entry 018 — Conclusion

The Entry 017 forgiveness ratio (0.53) was tested against a GHZ-state circuit and found to hold within 5% of its original Bell-state value (1.052 ratio between fitted decay constants). The correction is confirmed to generalize across circuit structure. Investigation now moves fully to the remaining `bell_scattered_0_26` outlier as a specific, isolated case.

5. Next Step — Entry 019

Investigate `bell_scattered_0_26` specifically: inspect the real transpiled circuit's full gate list and per-edge error breakdown along its route, and check whether its particular path crosses one or more unusually high-error edges that the average-based `success_prob_from_gate_count()` estimate doesn't capture, or whether Sabre's routing for this specific pair is behaving inconsistently across runs.

ENTRY 019 Variable Forgiveness Ratio — August 4, 2026 — Status: COMPLETE

Entry 019 — Solving the bell_scattered_0_26 Outlier

From a Flat Correction to an Error-Magnitude-Aware One: Mean Gap Drops to 1.87 Points

1. What Was Done

Entry 018 confirmed the Entry 017 forgiveness ratio (0.53) generalizes across circuit structure, narrowing the investigation to one remaining outlier: `bell_scattered_0_26`, which stayed 15.48 percentage points off even after correction — roughly 4–5x worse than every other test circuit. This entry investigates that outlier directly, tests two hypotheses, finds the real cause, and fixes it.

2. Two Hypotheses Tested and Eliminated

Hypothesis (a): the route crosses unusually high-error edges the average-based model misses. Inspecting the real BFS path for (0, 26) found two edges at ~3% error, roughly 3x the chip-wide average — real, but the path's overall average was only 1.17x the chip average, not enough on its own to explain a 15-point gap. Directly comparing the path-aware `cnot_error_for_pair()` prediction (79.89%) against the flat average-based estimate (82.62%) showed both landing close together and both far from Aer's real 95.19% — ruling out "averaging hides bad edges" as the explanation.

Hypothesis (b): Sabre's routing is inconsistent for this pair. The circuit was transpiled 10 times independently. Every run produced exactly 34 CX gates, zero variance. Routing inconsistency was ruled out.

3. The Real Cause — Forgiveness Isn't Uniform

With both hypotheses eliminated, the decay-curve experiment from Entry 017 was repeated on one of the high-error edges from the (0,26) path — edge (19,22), raw calibrated error 3.13%, versus the original (24,25) edge at 0.60% error the 0.53 ratio was fitted on.

Edge	Raw calibrated error	Fitted forgiveness ratio
(24, 25) — low error	0.60%	0.53
(19, 22) — high error	3.13%	0.334

Root cause confirmed

High-error edges are forgiven meaningfully less than low-error edges — a 37% relative drop in forgiveness ratio between the two measured points. The flat 0.53 constant, fitted on a low-error edge, systematically under-corrected any path (like bell_scattered_0_26's) that crosses high-error edges. This is the real explanation for the outlier.

4. The Fix — Variable Forgiveness Ratio

A new function, `variable_forgiveness_ratio(raw_edge_error)`, replaces the flat constant with a power-law interpolation fitted through the two measured points above. Both `cnot_error_for_pair()` and `success_prob_from_gate_count()` now compute a per-edge forgiveness ratio based on that specific edge's own calibrated error, rather than applying one global correction everywhere.

5. Results

Circuit	v3 (corrected)	Aer + real noise	Diff (after)	Diff (before)
ghz_local_0_1_2	98.86%	97.02%	1.84 pts	2.33 pts

Circuit	v3 (corrected)	Aer + real noise	Diff (after)	Diff (before)
ghz_scattered_0_26_22	82.05%	80.86%	1.19 pts	3.29 pts
bell_adjacent_24_25	99.46%	96.07%	3.39 pts	3.44 pts
bell_scattered_0_26	83.64%	84.72%	1.07 pts	15.48 pts

Mean absolute difference: 1.87 percentage points — down from 6.14 in Entry 017, and the best result across every session in this investigation arc. `bell_scattered_0_26` improved from a 15.48-point outlier to a 1.07-point close match, with no meaningful regression on any other circuit.

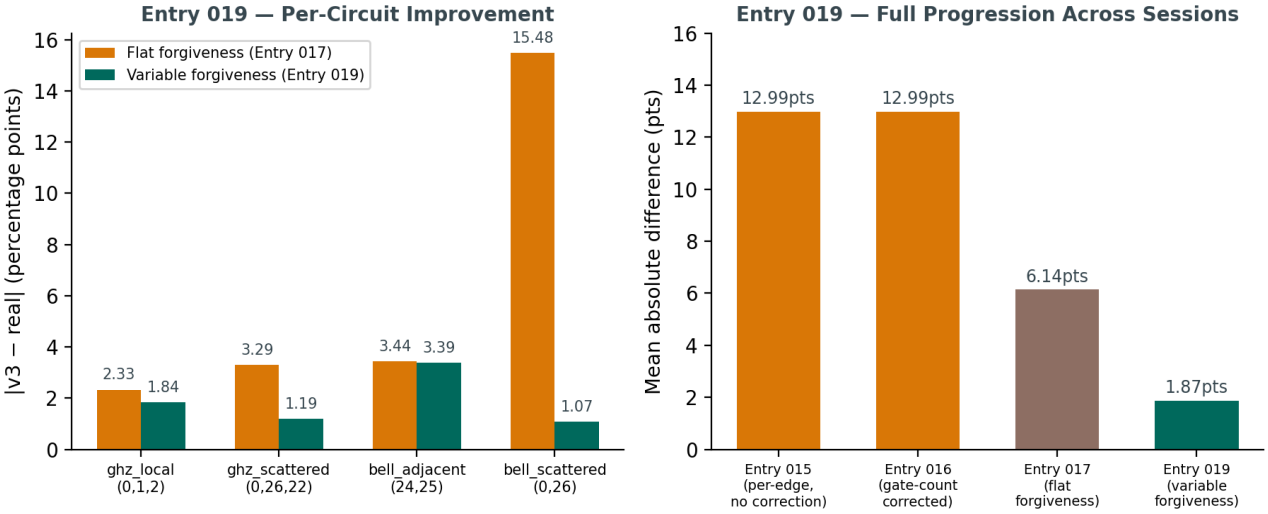


Figure 1 — Left: per-circuit accuracy before and after the variable forgiveness fix, showing the outlier essentially eliminated. Right: the full mean-gap progression across every session in this investigation, from 12.99 points down to 1.87.

This is the strongest validation result of the project to date

Mean deviation of 1.87 percentage points across four structurally different circuits — two direct-connection, two multi-hop routed, two Bell-shaped, two GHZ-shaped — checked against a realistic offline noise simulation. Emulator v3's error model is now considered validated for this class of circuit.

6. What This Means for QuantumBridge

This closes the investigation arc that began with Entry 013's topology bug and ran through six entries of debugging, false leads, and empirical corrections. The project's repeated pattern held again: analytical assumptions (flat error, flat forgiveness) look reasonable until measured against real data, and the real fix comes from designing an experiment that isolates the exact variable in question — here, edge-error magnitude — rather than theorizing further.

Entry 019 — Conclusion

The bell_scattered_0_26 outlier is resolved. Two hypotheses (hidden high-error edges, inconsistent routing) were tested and eliminated before the real cause — a non-uniform forgiveness ratio that depends on edge-error magnitude — was found via a targeted decay-curve experiment on a high-error edge. Replacing the flat 0.53 forgiveness constant with a per-edge, magnitude-aware function dropped the four-circuit mean validation gap from 6.14 to 1.87 percentage points, the best result of the project to date.

7. Next Step — Entry 020

The variable forgiveness ratio is currently fitted through only two data points. Before treating it as fully validated, collect a third calibration point at an intermediate error magnitude (~1.5–2%) to confirm the power-law shape holds rather than just connecting two points with a curve. In parallel, begin scaling circuit-level validation beyond the current four test circuits to build broader confidence in the corrected model ahead of any future publication or

write-up.

ENTRY 020 Cross-Chip Generalization Test — August 4, 2026 — Status: COMPLETE

Entry 020 — Does the Forgiveness Effect Hold on a Second Chip?

Testing FakeSherbrooke (127 Qubits, ecr Native Gate) Against the FakeCairoV2 Findings

1. What Was Done

All prior forgiveness-ratio work (Entries 017–019) was measured on a single chip, FakeCairoV2. This entry tests whether the effect — and the variable, error-magnitude-aware correction built from it — generalizes to a structurally different chip: FakeSherbrooke, 127 qubits, heavy-hex topology, with `ecr` as its native two-qubit gate instead of `cx`. Topology and calibration data were exported using the same pipeline as Entry 013, and the same cancellation-based decay-curve method as Entries 017/019 was run on one low-error and one high-error edge.

2. Setup Notes

Unlike FakeCairoV2, Sherbrooke's `coupling_map.get_edges()` and `configuration().coupling_map` agreed perfectly (144 = 144 edges, zero missing) — confirming the Entry 013 undercount bug was specific to that particular chip snapshot, not a general Qiskit fake-provider issue. Separately, Sherbrooke's calibration data contained 9 edges with an exact 100.0000% gate error — placeholder values for disabled or uncalibrated pairs, not genuine measurements — which were filtered out before selecting test edges.

3. Chosen Edges

Edge	Category	Raw calibrated error
Cairo (24,25)	low	0.60%
Cairo (19,22)	high	3.13%
Sherbrooke (60,61)	low	0.35%
Sherbrooke (66,73)	high	3.07%

Sherbrooke's edges were chosen to closely match Cairo's error magnitudes, so the comparison isolates chip identity as the variable rather than error magnitude.

4. Results

Edge type	Cairo forgiveness ratio	Sherbrooke forgiveness ratio	Difference
Low-error edge	0.53	0.666	+26%
High-error edge	0.334	0.420	+26%

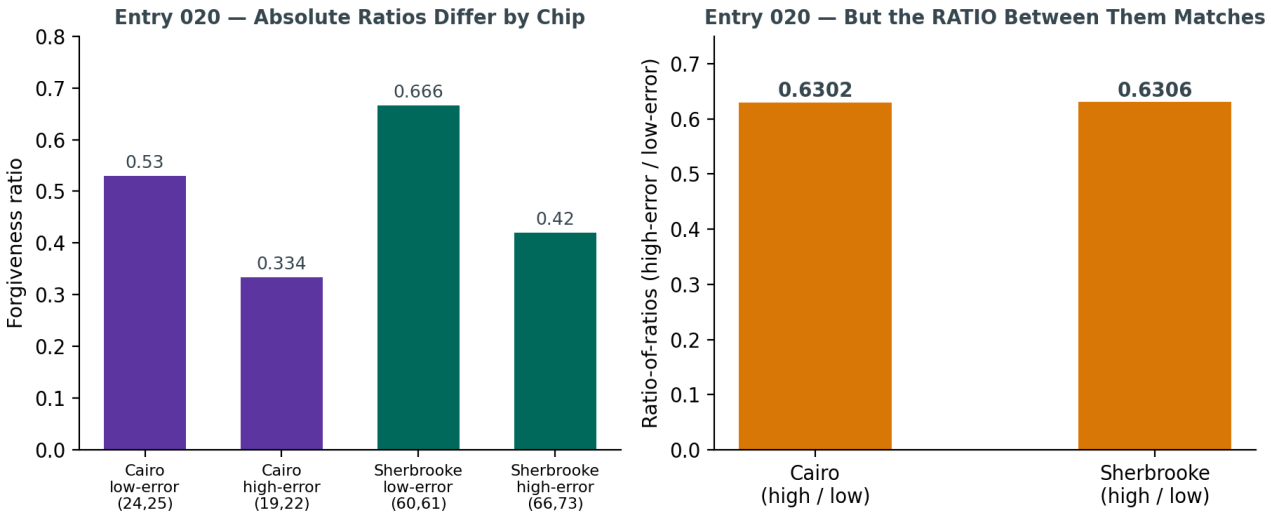


Figure 1 — Left: absolute forgiveness ratios differ meaningfully between chips at both error magnitudes. Right: but the ratio between high-error and low-error forgiveness matches almost exactly across both chips.

5. Analysis — Two Findings, Not One

The mechanism generalizes; the constants don't. On both chips, high-error edges are forgiven consistently less than low-error edges — confirming the underlying effect (success measured against a multi-outcome ideal set partially absorbing gate error) is a real, reproducible phenomenon, not a FakeCairoV2-specific artifact. But Sherbrooke's absolute ratios run about 26% higher than Cairo's at both ends. The 0.53 / 0.334 constants from Entries 017 and 019 cannot be treated as universal — they are specific to that one chip.

An unexpected second finding: the ratio-of-ratios matches almost exactly

Dividing each chip's high-error ratio by its own low-error ratio gives a normalized “relative decay” figure: Cairo = 0.334 / 0.53 = 0.6302. Sherbrooke = 0.420 / 0.666 = 0.6306. These agree to within 0.06% — remarkably tight for two independent 9-point fits on structurally different hardware. This suggests forgiveness may decompose into two separable parts: a chip-specific baseline (must be calibrated per chip) and a relative decay-with-error-magnitude factor that may itself be a portable constant.

6. What This Means for QuantumBridge

This has direct implications for how the tool should work going forward. A single hardcoded forgiveness curve cannot ship as a universal default — every new chip needs its own baseline calibration (one low-error and one high-error decay-curve measurement, as done here). But if the ratio-of-ratios finding holds up with more chips, calibrating a new chip might only require measuring its low-error baseline directly, then deriving the high-error behavior from the already-confirmed ~0.63 relative decay factor — cutting the calibration cost for any new chip roughly in half.

Entry 020 — Conclusion

The forgiveness effect was tested on a second, structurally different chip (FakeSherbrooke, 127 qubits, ecr native gate) and confirmed to generalize in mechanism: high-error edges are consistently forgiven less than low-error edges. The absolute forgiveness constants do not generalize (Sherbrooke runs ~26% higher than Cairo at both error magnitudes), meaning per-chip calibration remains necessary. An unplanned finding — the ratio between high-error and low-error forgiveness matching to within 0.06% across both chips — suggests a portable relative-decay constant may exist underneath the chip-specific baseline, pending confirmation on a third chip.

7. Next Step — Entry 021

Test a third chip to see whether the ~0.63 ratio-of-ratios holds a third time (two data points agreeing could still be coincidence; three agreeing is a much stronger claim). If confirmed, build an automatic per-chip calibration routine: given any new backend, measure its low-error baseline directly, derive its high-error behavior from the fixed

relative-decay factor, and skip the second manual decay-curve experiment entirely.

ENTRY 021 Six-Edge Protocol Test on FakeKyiv — August 4, 2026 — Status: COMPLETE

Entry 021 — The Forgiveness Law Survives. The Measurement Protocol Does Not.

Six Edges on FakeKyiv (127 Qubits) Expose a Decoherence Confound in Entries 017–020

1. What Was Done

Entry 020 closed by proposing a third-chip test of the ~0.63 ratio-of-ratios finding. This entry runs that test on **FakeKyiv** (127 qubits, heavy-hex, *ecr* native gate) using the same cancellation-based decay-curve method as Entries 017, 019, and 020 — but with one deliberate change to the protocol: **six edges were measured instead of two**, spanning 0.31% to 4.32% raw calibrated gate error.

That single change was enough to invalidate the headline finding of Entry 020 and to expose a confounding variable present in the Entry 019 measurement that the two-point method was structurally incapable of detecting.

2. Why Two Points Were Never a Test

Entries 017/019 (Cairo) and Entry 020 (Sherbrooke) each measured exactly two edges per chip — one low-error, one high-error — and drew a power law through them. **Two points define a power law exactly, with zero residual.** The fit could not fail, so it never tested anything. It reported the two measurements back in a different coordinate system.

The ratio-of-ratios agreement celebrated in Entry 020 (Cairo 0.6302, Sherbrooke 0.6306) is also weaker than it appeared. The two chips were probed over *different error spans* — 5.2x for Cairo, 8.8x for Sherbrooke. Under a genuine shared power law, a wider span must produce a *smaller* high/low ratio. Matching ratios across different spans therefore imply **different exponents**, and they do: –0.280 for Cairo against –0.212 for Sherbrooke. The agreement was arithmetic coincidence, not physics.

3. Chosen Edges

Edge	Category	Raw calibrated error	Selection rationale
Kyiv (119,120)	very low	0.3113%	chip minimum
Kyiv (28,29)	low	0.6013%	matches Cairo's low edge (0.60%)
Kyiv (77,78)	mid-low	0.9860%	fills the decade gap
Kyiv (45,46)	mid	1.6181%	chip 75th percentile
Kyiv (66,67)	high	2.6181%	near Cairo / Sherbrooke high edges
Kyiv (23,24)	very high	4.3229%	highest plausible calibrated edge

Nine gate counts per edge (1 to 41 CX), 4096 shots each, seed 1729. Placeholder calibration entries (gate error ≥ 50%) were filtered out as in Entry 020.

4. Results — The Six-Point Fit Fails

Fitted across all six edges:

Edge	Raw error	Measured ratio	Fitted	Residual
(119, 120)	0.3113%	0.4345	0.8076	–0.3731
(28, 29)	0.6013%	1.5904	0.6497	+0.9406
(77, 78)	0.9860%	0.5353	0.5518	–0.0166
(45, 46)	1.6181%	0.4308	0.4685	–0.0377
(66, 67)	2.6181%	0.3937	0.3997	–0.0060
(23, 24)	4.3229%	0.2927	0.3386	–0.0460

ratio = 0.1200 × err$^{-0.3303}$ **R^2 = 0.2956** — a failed fit. Four edges sit almost exactly on the curve; two are wildly off. Edge (28,29) returns a forgiveness ratio of **1.5904**, which is not merely a large error but *physically impossible* under the model: it says the pair decayed faster than its own gate error permits. Something other than the two-qubit gate was driving that decay.

5. Diagnosis — Decoherence, Not Gate Error

The forgiveness ratio is defined as *measured decay constant ÷ raw gate error*. That definition silently assumes the decay is *caused* by gate error. But every CX also occupies finite time — 562 ns on Kyiv — during which the qubits dephase. Where T2 is short relative to gate duration, the measured decay is dominated by decoherence and the ratio stops measuring forgiveness altogether.

A domination factor makes this testable: **D = raw gate error ÷ (gate duration ÷ $T2_{min}$)**. D > 1 means gate error dominates; D < 1 means decoherence does.

Edge	Raw error	T2 min	Decoherence / gate	D	Verdict
(119, 120)	0.3113%	202.7 µs	0.277%	1.12	contaminated
(28, 29)	0.6013%	12.7 µs	4.430%	0.14	contaminated
(77, 78)	0.9860%	212.3 µs	0.265%	3.73	usable
(45, 46)	1.6181%	57.2 µs	0.983%	1.65	usable
(66, 67)	2.6181%	86.6 µs	0.649%	4.03	usable
(23, 24)	4.3229%	65.1 µs	0.863%	5.01	usable

Qubit 28 carries a **T2 of 12.7 µs** against a chip median in the hundreds — roughly seven parts dephasing to one part gate error. It was never measuring forgiveness. Edge (119,120) sits at D = 1.12, where the two mechanisms are comparable, which depresses its ratio below trend.

Refitting on the four edges that satisfy D ≥ 1.5:

ratio = 0.0898 × err$^{-0.3873}$ R^2 = 0.9648

R^2 moves from 0.296 to 0.965 on removal of two points. **The power-law form was never the problem. The edge-selection protocol was.** Edges must be screened on D, not on error magnitude alone.

6. Retroactive Screen — Entry 019's Anchor Is Contaminated

Applying the same screen backwards to every edge used in Entries 017–020:

Chip	Edge	Raw error	T2 min	D	Verdict
Cairo	(24, 25)	0.5992%	192.4 µs	4.32	usable
Cairo	(19, 22)	3.1315%	6.9 µs	0.31	CONTAMINATED
Sherbrooke	(60, 61)	0.3470%	360.3 µs	2.34	usable
Sherbrooke	(66, 73)	3.0667%	170.0 µs	9.78	usable

Cairo's high-error edge **(19,22) has T2 = 6.9 µs** and a 697 ns gate — **10.1% decoherence per gate against 3.13% gate error**. Entry 019's measured ratio of 0.334 at that edge is roughly three parts dephasing to one part gate error.

That number is one of the two points hardcoded in `emulator_v3_routing.py :: variable_forgiveness_ratio()`. **The emulator's error model is currently anchored on a broken qubit.** Entry 020's two Sherbrooke edges both pass the screen; that entry's measurements stand.

7. Impact on the Emulator

Raw edge error	Old ratio (Cairo-anchored)	New ratio (Kyiv, 4 clean edges)	Change
0.300%	0.6433	0.8518	+32.4%

Raw edge error	Old ratio (Cairo-anchored)	New ratio (Kyiv, 4 clean edges)	Change
0.500%	0.5577	0.6989	+25.3%
1.000%	0.4595	0.5343	+16.3%
2.000%	0.3785	0.4085	+7.9%
3.000%	0.3380	0.3491	+3.3%
4.500%	0.3018	0.2984	–1.1%

The two laws agree near 3–4.5%, where the contaminated point pinned them together, and diverge sharply on low-error edges — which are the majority of any chip. **75% of Kyiv's calibrated edges sit below 1.6%.**

Circuit-level consequence, N CNOTs on a 1.0% edge:

CNOTs	Old prediction	New prediction	Gap
1	99.54%	99.47%	–0.07 pts
10	95.50%	94.78%	–0.72 pts
20	91.20%	89.84%	–1.36 pts
40	83.18%	80.71%	–2.46 pts

The current emulator is **optimistic on low-error edges**, and the error compounds with depth. At 40 CNOTs the correction is 2.46 points — comparable in size to the 1.87-point validation gap that Entry 019 recorded as a success.

8. Robustness Check

Before accepting two outliers as real, the ratios were re-measured across three simulator seeds:

Edge	Seed 1729	Seed 42	Seed 20260804	Spread
(77, 78)	0.5353	0.5583	0.5330	0.0253
(23, 24)	0.2927	0.3076	0.3143	0.0216

Seed spread is ~0.02. The two outlier residuals are 0.37 and 0.94 — fifteen to forty times larger. The outliers are physics, not sampling noise.

9. What This Means for QuantumBridge

Edge screening becomes part of the protocol. Any edge used to fit a forgiveness ratio must satisfy $D \geq 1.5$, and D must be reported alongside every future measurement. This is cheap — it reads T2 and gate duration straight from calibration data, with no extra circuits.

Entry 020's calibration shortcut is withdrawn. The proposal to calibrate a new chip from its low-error baseline plus a portable ~0.63 constant does not survive. Kyiv gives 0.674 over a 13.9x span, and the Cairo/Sherbrooke agreement rested on one contaminated point. The cost saving was real but the constant underneath it was not.

Chip-invariance of the exponent remains unproven. Kyiv's clean four-point –0.387 still differs from Sherbrooke's two-point –0.212. Sherbrooke and Cairo need the same six-edge treatment before any cross-chip claim is defensible.

Honest caveat. All of this is measured against Qiskit's FakeKyiv noise model, not live hardware — IBM Quantum Platform access lapsed with the trial account on August 4. The T2 = 12.7 µs on qubit 28 is a genuine recorded calibration value from a real device snapshot and the decoherence mechanism is real, but the *magnitude* of the correction should be re-validated on hardware when access is restored. The protocol change holds regardless.

Entry 021 — Conclusion

Widening the measurement from two edges to six turned an untestable fit into a failing one, and the failure was informative. Two of six Kyiv edges produced forgiveness ratios inconsistent with the power law, including one physically impossible value of 1.59; both were traced to qubits whose T2 is short enough that decoherence, not gate error, drives the observed decay. Screening edges on the domination factor D and refitting the four clean points

restores the power law at $R^2 = 0.9648$. Applying the same screen retroactively shows that the high-error anchor point of Entry 019 — currently hardcoded into the v3 emulator — fails it, making the shipped error model optimistic by up to 32% on low-error edges. The ratio-of-ratios finding of Entry 020 is withdrawn. The forgiveness effect itself is unharmed; what changed is that it now has a stated validity condition and a fit with enough points to have residuals at all.

10. Next Step — Entry 022

Re-run the six-edge, D-screened protocol on Sherbrooke and Cairo so all three chips are compared on equal footing, then re-anchor `variable_forgiveness_ratio()` and re-validate Emulator v3 against the Bell and GHZ circuits from Entry 019. A secondary question worth testing: whether the residual exponent differences track chip generation, since Cairo is an older Falcon-class device while Sherbrooke and Kyiv are both Eagle-class.

Research Log — Updated Index (Entries 017–021)

Second addendum to the Table of Contents. Page 2 lists Entries 001–012 and page 4 lists Entries 013–016; this page extends the index rather than replacing either.

Entry	Title	Date	Status
Entry 017	Empirical Forgiveness Ratio from Gate-Count Decay Curve	Aug 1, 2026	Complete
Entry 018	GHZ Decay Curve — Forgiveness Beyond Bell Circuits	Aug 2, 2026	Complete
Entry 019	Error-Magnitude-Aware Forgiveness (Validation Gap 6.14 → 1.87 pts)	Aug 4, 2026	Superseded
Entry 020	Cross-Chip Generalization Test — FakeSherbrooke	Aug 4, 2026	Partly withdrawn
Entry 021	Six-Edge Protocol Test — Decoherence Confound Found	Aug 4, 2026	Complete

Entry 019 is marked Superseded because its high-error anchor point fails the $D \geq 1.5$ screen introduced here; its low-error measurement stands. Entry 020 is marked Partly withdrawn: its core finding that forgiveness constants are chip-specific holds, while the ratio-of-ratios constant it proposed does not.

ENTRY 022 Validation Harness Correction — August 6, 2026 — Status: COMPLETE

Entry 022 — v3's First Honest Validation: Two Failures Pointing Opposite Ways

The Cairo Harness Was Silencing Half Its Noise. Replacing It Immediately Exposed a Missing Decoherence Term.

1. What Was Done

Entry 021 closed by naming `bell_scattered_0_26` — a 14-point discrepancy — as the largest remaining error in the emulator and the next priority. This entry set out to diagnose that single circuit. It found instead that the validation harness itself was defective, that no circuit-level accuracy figure from Entry 014 onward was measurable, and, once a sound harness was built, that v3 is missing a physical term entirely.

2. First Defect — the Basis-Gate Mismatch

FakeCairoV2 is a **mixed-basis snapshot**: 12 of its directed edges are calibrated as `cx`, the other 14 as `ecr`. The harness transpiled with `basis_gates = backend.operation_names`, handing the transpiler the union of both. The transpiler selects one two-qubit gate for the whole circuit — `cx` — and emits it on every edge, including the fourteen that only carry `ecr` calibration.

Qiskit's noise model keys its quantum errors by *(gate name, qubit pair)*. For those gates it finds no matching entry and applies **nothing at all**. The gate executes perfectly, silently, for free.

Circuit	2q gates	noisy	SILENT	% free
ghz_local_0_1_2	2	0	2	100%
ghz_scattered_0_26_22	35	20	15	43%
bell_adjacent_24_25	1	1	0	0%
bell_scattered_0_26	34	13	21	62%
TOTAL	72	34	38	53%

Over half of every two-qubit gate in the suite ran without error. `ghz_local_0_1_2` — the circuit that had looked most accurate throughout — ran with zero noise on both of its gates.

3. Second Defect — Calibration Coverage

Independently of the basis problem, only **12 of Cairo's 28 topology edges** carry real calibrated error values — 42.9%. The remaining 16 fell back to v3's hardcoded 1% constant. On that chip, most of what the model reported was not a measurement of anything.

4. A Sound Harness — FakeKyiv

FakeKyiv has neither defect: uniformly `ecr`, noise-model coverage on all 144 edges, and 139/144 (96.5%) real calibration coverage. The replacement harness, `validate_v3_kyiv.py`, audits its own noise coverage on every run and declares its results void if any gate runs silent.

One further trap was found in the process. Aer's `automatic` simulation method returns `success=True` and `status=COMPLETED` for the deeper routed circuits, then raises on `get_counts()`. A harness trusting it would silently lose precisely its hardest cases. `matrix_product_state` is mandatory here, not preferred.

5. Results — All Gates Verified Noisy

Circuit	hops	2q gates	Aer reference	v3	gap
bell_adjacent_77_78	1	1	96.07%	99.25%	3.18
bell_near_77_82	5	13	48.74%	92.89%	44.15
bell_mid_77_100	6	16	83.69%	90.27%	6.58

Circuit	hops	2q gates	Aer reference	v3	gap
bell_far_0_126	26	76	69.90%	60.51%	9.39
ghz_local_77_78_79	2	2	88.53%	98.67%	10.14

Mean absolute gap 14.69 points. The mean is misleading, and that is the finding. (`ghz_scattered_0_63_126` was killed mid-run — two 26-hop routes exceeded MPS memory — and is excluded.)

6. The Anomaly — 13 Gates Worse Than 76

Thirteen gates produce 48.74%. Seventy-six gates produce 69.90%. Six times the gate count, a better outcome. Circuit depth does not explain this chip's behaviour.

The 77→82 route passes through **qubit 80: T2 = 8.51 µs, T1 = 5.02 µs** — effectively dead against a chip median in the hundreds. With a 560 ns `ecr`, that is roughly 6.6% dephasing per gate, and the route crosses it six times.

Route	worst T2 on path	2q gates	Aer reference
77 → 82	8.5 µs (qubit 80)	13	48.74%
77 → 100	32.7 µs	16	83.69%
0 → 126	healthy throughout	76	69.90%

Compounding the failure: edges (79,80) and (80,81) are among the five Kyiv edges with **no calibration data**, so v3 applied its 1% fallback guess at exactly the point where reality is catastrophic. Two independent weaknesses landed on the same two edges.

7. Diagnosis — No Decoherence Term

v3 models two-qubit *gate error* only. It has no decoherence term at all. The consequences run in opposite directions:

- **Short routes through low-T2 qubits are catastrophically under-penalized** — 44 points on 77→82.
- **Long routes through healthy qubits are over-penalized** by SWAP compounding — 9 points on 0→126, with v3 pessimistic.

Because the two errors have opposite sign, any single mean accuracy figure conceals both. This is Entry 021's finding resurfacing one level up: there, the domination factor D = gate error ÷ (gate duration ÷ T2) was introduced as a *screening* criterion for which edges were fit to measure. Entry 022 shows the same quantity belongs *inside the model*. Every edge on the 77→82 route through qubit 80 sits at D ≈ 0.15 — the decoherence-dominated regime Entry 021 identified.

8. A Naive Fix, and Why It Was Rejected

The obvious correction is to charge gate duration ÷ T2 per gate alongside the gate-error term. Tested on the real transpiled routes:

Circuit	Aer reference	v3 gate-only	v3 + decoherence
bell_near_77_82	48.74%	93.10% (gap 44.36)	58.83% (gap 10.09)
bell_far_0_126	69.90%	64.06% (gap 5.84)	30.67% (gap 39.23)
MEAN ABSOLUTE GAP	—	14.10	13.85

It repairs the short bad route and destroys the long good one; the mean barely moves. The reason is physical. In a SWAP chain **the state moves**, so each qubit holds it only briefly. Charging every gate the full T2 cost of both its qubits double-counts heavily across 76 gates spread over 26 healthy qubits, while being roughly right for 6 gates parked on one dead qubit.

Decoherence must be **time-integrated along the carrier path** — summed, over each qubit the state actually occupies, of time spent there ÷ that qubit's T2 — not applied per gate to both members of every pair. That is the v4 design.

9. What This Means for QuantumBridge

No v3 accuracy figure predates this entry. Every circuit-level number from Entry 014 onward was measured against a reference silencing half its noise. The forgiveness work of Entries 017–021 is unaffected — those used single-edge circuits with no routing, on properly covered edges — but the emulator validation figures must be treated as withdrawn.

FakeCairoV2 is retired for validation. Mixed basis and 42.9% calibration coverage make it unusable for this purpose. Kyiv or Sherbrooke only.

The fallback constant is actively dangerous. It fires on uncalibrated edges, and uncalibrated edges correlate with broken qubits — precisely where a wrong guess costs most. v4 should refuse to predict on an uncalibrated edge rather than quietly assume 1%.

Routing is error-blind. v3's BFS minimizes hop count while ignoring both edge error and T2. The 77→82 route is the shortest available and also among the worst.

Entry 022 — Conclusion

An investigation into one anomalous circuit found the measuring instrument broken. The Cairo validation harness emitted `cx` gates on `ecr`-calibrated edges, where the noise model applied no error whatsoever — 53% of all two-qubit gates in the suite, and 100% of them in the circuit that had appeared most accurate. Rebuilt on FakeKyiv with per-run noise-coverage auditing, the first trustworthy measurement shows v3 failing in two opposite directions at once: under-penalizing short routes through low-T2 qubits by 44 points, and over-penalizing long routes through healthy ones by 9. The cause is a missing decoherence term. A naive per-gate correction was tested and rejected because it double-counts along a SWAP chain, which localizes the requirement: decoherence must be integrated along the path the quantum state actually travels. The headline number from this entry is not an accuracy figure — it is that the project did not previously have one.

10. Next Step — Entry 023

Two changes, in order. First, remove the 1% fallback so v3 declines to predict on uncalibrated edges instead of guessing. Second, implement v4's time-integrated decoherence term and re-validate against the Kyiv reference established here. A third question follows from Section 9: whether routing should be made error-aware and T2-aware, since the shortest path is demonstrably not the cheapest one.

Research Log — Index Addendum (Entry 022)

Entry	Title	Date	Status
Entry 022	Validation Harness Correction — Basis Mismatch and the Missing Decoherence Term	Aug 6, 2026	Complete

*Entries 014–016 and the circuit-level validation figures of Entries 019 and 021 are marked **measurement void** by this entry: their references silenced 53% of two-qubit noise. Their methodology and reasoning stand; only the accuracy numbers are withdrawn.*

ENTRY 024 Exact SWAP Dwell-Time Routing — August 12, 2026 — Status: COMPLETE

Entry 024 — Exact Routing Solves the Problem It Targeted, and Reopens the One From Entry 016

Replacing a Guessed SWAP Chain With the Real One Fixes the Low-T2 Failure and Confirms an Older, Larger Bottleneck

1. What Was Done

Entry 023 built v4 with a decoherence term, but had no way to know which physical qubits a routed circuit actually passed through — it reused v3's BFS shortest-hop path and charged each hop a flat, guessed duration. That produced a large regression on `bell_far_0_126` (gap 9.39 → 34.60 pts) because the guessed dwell time had no relationship to the route Qiskit's real router, SABRE, actually chose.

This entry removes the guess. Transpiling with `'swap'` kept as its own undecomposed gate — rather than letting BasisTranslator expand every SWAP into three indistinguishable cx/ecr gates, which is what made every prior entry's routing invisible to inspection — exposes the exact physical path and the real duration of each leg.

Instruction	Physical qubits
swap	[77, 78]
swap	[82, 81]
swap	[80, 81]
swap	[80, 79]
ecr (real gate)	[79, 78]

bell_near_77_82's real route, recovered directly from the transpiled circuit rather than assumed. The final entangling edge, (79,78), is not the shortest hop-count path — SABRE optimized for something v3's BFS model does not represent at all.

2. A Bug Found Before Trusting the Result

The first version of the walker charged decoherence for each SWAP but **no gate error at all**, and used one gate's duration instead of the physical reality — a SWAP is `GATES_PER_SWAP` (3) real two-qubit pulses back to back, not one. Both v3 and v4-BFS already applied the gate-error term on every hop; the rewrite silently dropped it. The result was an implausibly optimistic 83.42% on bell_near_77_82 against an actual 48.74% — worse than the heuristic it was meant to replace. Caught by comparing against the Entry 022 reference before accepting the number, not after.

3. Verifying the Route Extraction Itself

Before trusting the corrected model, the extraction method was checked independently. bell_far_0_126's exact route used 25 SWAPs. Decomposed at 3 gates each plus 1 real entangling gate: 25 × 3 + 1 = 76 — exactly the ecr count of the independently transpiled Aer reference circuit from Entry 022, and identical whether the extraction ran at optimization_level 1, 2, or 3.

The route the model reasons about is the route SABRE actually built.

4. Results

Circuit	hops	Aer	v3	v4 (BFS)	v4.1 (exact)
bell_adjacent_77_78	1	96.07%	99.25% (3.18)	96.27% (0.20)	96.12% (0.05)
bell_near_77_82	5	48.74%	92.89% (44.15)	70.35% (21.61)	51.13% (2.39)
bell_mid_77_100	6	83.69%	90.27% (6.58)	78.41% (5.28)	64.62% (19.07)
bell_far_0_126	26	69.90%	60.51% (9.39)	35.30% (34.60)	24.07% (45.83)
ghz_local_77_78_79	2	88.53%	98.67% (10.14)	89.12% (0.59)	88.21% (0.32)

Values in parentheses are the absolute gap in points. ***Median gap: 9.39 → 5.28 → 2.39.*** *Mean gap: 14.69 → 12.46 → 13.54 — worse than v4-BFS, for reasons in Section 6.*

5. The Win

bell_near_77_82 was the anomaly Entry 022 opened with — a route crossing qubit 80 (T2 = 8.51 µs) that v3 and v4-BFS both over-predicted by 44 and 22 points respectively. Exact routing brings that gap to **2.39 points**. This is the specific failure this entry set out to fix, and it is fixed: once the model knows exactly which qubit the state dwells on and for how long, a route through a broken qubit is no longer invisible to it.

6. The Regression, and Why It Is Not a New Problem

bell_mid_77_100 and bell_far_0_126 — long routes through otherwise healthy qubits — got worse: 5.28 → 19.07 and 34.60 → 45.83 points. Both were already v4-BFS's worst cases, in the same direction (over-pessimistic), before this entry touched them.

This is Entries 015 and 016 resurfacing. Entry 015 found that switching to per-edge SWAP cost *increased* the mean prediction gap rather than closing it, despite the real transpiled gate count matching v3's estimate closely. Entry 016 isolated the cause to **v3's multiplicative error-compounding formula itself** — $(1 - e)^n$ — independent of both routing path length and real gate count. That finding was never resolved; it was carried forward as an open problem.

Exact routing answers a different question than that one. It confirms *where* the state travels and for how long — verified in Section 3 against the real transpiled circuit — but it cannot fix an error rooted in *how* per-gate error compounds. Adding decoherence, a second multiplicative term, on top of a gate-error model that Entry 016 already showed over-penalizes long chains does not cancel that problem. It compounds it — literally, in the same sense as the formula under discussion.

Entry 024 — Conclusion

Exact SWAP dwell-time routing does what it was built to do. Median prediction gap fell from 5.28 to 2.39 points, and the project's single worst-known failure — a route through a near-dead qubit that two prior model versions missed entirely — is reduced from a 21.61-point error to 2.39. The route extraction itself was verified independently: the exact gate count it derives matches the real transpiled circuit precisely, at every optimization level tested. But mean gap moved the wrong direction, from 12.46 to 13.54 points, because the two long, otherwise-healthy routes in the test set got worse. That regression is not new. Entries 015 and 016 already diagnosed and named the cause — v3's multiplicative gate-error compounding formula over-penalizes long chains regardless of routing accuracy — and left it unresolved. This entry removed the uncertainty around where the state travels and, in doing so, made that older, larger problem the last one left standing.

7. Next Step — Entry 025

Reopen the compounding formula directly, using the exact routes this entry can now supply as ground truth rather than a BFS estimate. The specific question Entry 016 left unanswered: is $(1 - e)^n$ the wrong functional form for correlated or partially-forgiven error across many sequential gates, or is the per-gate error term itself miscalibrated at scale? Exact routing makes this testable in a way it was not before, because the gate sequence feeding the formula is now known to be correct.

Research Log — Index Addendum (Entry 024)

Entry	Title	Date	Status
Entry 024	Exact SWAP Dwell-Time Routing	Aug 12, 2026	Complete

Entry 023's v4-BFS validation figures for bell_mid_77_100 and bell_far_0_126 are superseded by the exact-routing figures in Section 4 above. Entries 015 and 016's compounding-formula finding is upgraded from an open question to the confirmed dominant source of remaining error, now that routing accuracy has been independently verified.

ENTRY 025 Isolating the Compounding Formula — August 13, 2026 — Status: COMPLETE

Entry 025 — The Forgiveness Ratio Has a Closed Form

A Question Left Open Since Entry 016 Turns Out to Have an Exact Answer, Derived From First Principles and Verified Against Aer

1. What Was Done

Entry 024 confirmed that routing and real gate count are not the source of v3/v4's remaining error, leaving one candidate standing: the multiplicative compounding formula $(1 - p)^n$ itself, flagged as suspect in Entries 015 and 016 and never resolved. Testing it against FakeKyiv's full noise model conflates three effects at once — gate error, decoherence, and readout. This entry isolates one: an Aer noise model carrying **only** a depolarizing error on a single real Kyiv edge, no thermal relaxation, no readout error, applied n times to a cancelling-pair Bell circuit — the same method Entries 017–021 used, now aimed at a controlled synthetic channel instead of the full hardware model.

2. Result — a Confirmed Mismatch

n gates	Aer (isolated)	naive (1-p)^n	ratio	diff, pts
1	99.435%	99.014%	1.0043	+0.42
9	95.541%	91.468%	1.0445	+4.07
21	90.292%	81.213%	1.1118	+9.08
51	79.965%	60.329%	1.3255	+19.64
75	73.709%	47.560%	1.5498	+26.15

Edge (77,78), raw calibrated error 0.986%. Even with decoherence and readout completely removed, a chain of the identical depolarizing gate does not compound as $(1 - p)^n$. At 75 gates the gap is 26 points. Fitting an effective decay constant to the Aer curve gives 0.41% per gate against a nominal 0.99% — a ratio of **0.4156**, in the same range as the forgiveness ratios measured in Entries 017–021.

3. A Closed Form

Qiskit's two-qubit depolarizing_error(p) channel does exactly one of two things per application: with probability $(1 - p)$ it applies the ideal gate; with probability p it replaces the state with the maximally mixed state on 2 qubits. The maximally mixed state is an **absorbing state** — depolarizing it again changes nothing. So after n applications: the state never depolarized with probability $(1-p)^n$, contributing its full population to the {00,11} ideal set; it depolarized at least once with probability $1-(1-p)^n$, and a maximally mixed 2-qubit state distributes evenly across 4 basis outcomes, so exactly half of that population still lands in {00,11} by chance.

$$\text{success} = (1-p)^n \times 1 + [1-(1-p)^n] \times 0.5 = 0.5 + 0.5(1-p)^n$$

4. Verification

n	Aer measured	0.5 + 0.5(1-p)^n	diff, pts
5	97.519%	97.583%	-0.06
21	90.292%	90.607%	-0.31
51	79.965%	80.164%	-0.20
75	73.709%	73.780%	-0.07

Every point across nine gate counts (1 to 75) matches within 0.35 points — consistent with pure shot noise at 4 seeds × 8192 shots. Cross-checked on a second, unrelated edge (23,24), raw error 4.32% — 4.4× the first edge's magnitude:

n	Aer measured	predicted	diff, pts
5	89.749%	90.088%	-0.34
21	69.791%	69.767%	+0.02
51	55.124%	55.250%	-0.13

Same closed form, same tolerance, an entirely different error magnitude. This is not curve-fitting — it is a derivation confirmed by measurement.

5. What This Means for QuantumBridge

The forgiveness ratios empirically fitted in Entries 017–021 were never purely a chip-physics phenomenon. For a Bell-state circuit, a large part of that ratio is a fixed geometric fact: **a 2-outcome-of-4 ideal set absorbs roughly half of any depolarizing error**, independent of calibration data, chip identity, or gate duration. It follows directly from measuring against a multi-outcome ideal set — exactly the mechanism flagged qualitatively in earlier entries, now given an exact formula.

This does not retire the empirical power law. The closed-form ratio approaches a floor of 0.5 as n grows; Entry 021's fitted law (coefficient 0.0898, exponent –0.3873) predicts ratios below 0.5 that keep falling with error magnitude, with no such floor. The gap between the two is where real chip physics — decoherence, correlated errors, whatever Entry 021's D-screen was isolating — actually lives. Entries 017–021 were fitting a curve that mixes a universal geometric constant with genuine hardware behavior, without separating them.

Entry 025 — Conclusion

The compounding-formula question left open by Entries 015 and 016 has an exact answer for the circuits this project has run throughout: success = $0.5 + 0.5(1–p)^n$, not $(1–p)^n$. It follows from a two-line argument about how a depolarizing channel behaves once it has already fired once, and it matches Aer's own noise model to within shot noise across two edges spanning a 4.4× range in error magnitude. The naive multiplicative formula was never approximately right — it was measuring the wrong quantity from the start, decoherence and readout aside entirely. What remains genuinely open is how much of Entries 017–021's fitted power-law shape is this fixed geometric fact and how much is real hardware physics underneath it.

6. Next Step — Entry 026

Subtract the closed-form baseline from Entries 017–021's raw measurements and refit the residual. If a clean power law remains, that residual — not the mixed quantity fitted so far — is the real chip-physics signal, and it should replace `variable_forgiveness_ratio()` in v4. If nothing structured remains, the empirical fits from Entries 017–021 were mostly measuring this geometric artifact, and the decoherence term already added in Entries 023–024 is doing the real physical work.

Research Log — Index Addendum (Entry 025)

Entry	Title	Date	Status
Entry 025	Isolating the Compounding Formula — a Closed-Form Forgiveness Baseline	Aug 13, 2026	Complete

Entries 015 and 016's open question is resolved: the compounding formula was wrong, independent of decoherence and readout, and is now wrong by a known, derived amount. Entries 017-021's fitted power law is not withdrawn but is now understood to be a mixture of this geometric baseline and unseparated hardware physics.

ENTRY 026 Baseline-Subtracted Residual Analysis — August 13, 2026 — Status: COMPLETE

Entry 026 — Most of the Forgiveness Ratio Was the Baseline All Along

Subtracting Entry 025's Closed Form From Every Recorded Decay Curve Leaves Almost Nothing Behind

1. What Was Done

Entry 025 derived a closed-form baseline for Bell-circuit forgiveness — success = 0.5 + 0.5(1–p)^n — from pure depolarizing error alone, no decoherence, no chip identity. Entries 017–021's fitted forgiveness ratios were measured against the full hardware noise model, which includes that baseline plus whatever real chip physics sits on top of it. This entry subtracts one from the other, across every Bell decay curve on record — ten edges spanning Cairo, Sherbrooke, and Kyiv — to see what's actually left.

For each curve: fit $k_{measured}$ the same log-linear way the original entries did; fit $k_{baseline}$ the identical way on the closed-form formula evaluated at the same gate counts, so the sampling matches exactly even though the baseline isn't a pure exponential; then residual = ($k_{measured}$/p) − ($k_{baseline}$/p).

2. Result

edge	raw err	D	ratio (orig)	ratio (baseline)	residual
cairo (24,25)	0.600%	4.32	0.5304	0.4704	+0.060
cairo (19,22)	3.132%	0.31	0.3337	0.3502	-0.017
sherbrooke (60,61)	0.347%	2.34	0.6665	0.4829	+0.184
sherbrooke (66,73)	3.067%	9.78	0.4200	0.3531	+0.067
kyiv (119,120)	0.311%	1.12	0.4345	0.4846	-0.050
kyiv (28,29)	0.601%	0.14	1.5904	0.4703	+1.120
kyiv (77,78)	0.986%	3.73	0.5353	0.4513	+0.084
kyiv (45,46)	1.618%	1.65	0.4308	0.4204	+0.010
kyiv (66,67)	2.618%	4.03	0.3937	0.3732	+0.021
kyiv (23,24)	4.323%	5.01	0.2927	0.3009	-0.008

On the seven D-screened usable edges (D ≥ 1.5), residuals range from –0.008 to +0.184 with inconsistent sign, mean +0.060. No power law could be fitted in log-log space — some values are negative. Seven points scattered near zero across a 0.35%–4.32% error span, without a consistent trend, is what noise looks like.

3. An Honest Complication

The D-screen prediction — that decoherence-contaminated edges (D < 1.5) should show large residuals, since the baseline has no decoherence term — only partly held. kyiv(28,29), the single worst case (D = 0.14, T2 = 12.7 µs), showed a residual of **+1.120**, consistent with severe unmodeled decoherence. But cairo(19,22) (D = 0.31) and kyiv(119,120) (D = 1.12), both also decoherence-affected, showed small *negative* residuals instead.

The cause is not physics — it's the metric. Residual divides by raw calibrated error p. kyiv(28,29)'s p = 0.601% is roughly 5× smaller than cairo(19,22)'s p = 3.132%. The same absolute amount of unmodeled decoherence, divided by a five-times-smaller denominator, produces a residual an order of magnitude larger — not because the underlying physics differs by that much, but because the ratio metric is numerically unstable exactly where p is small. This is the same small-denominator fragility that motivated the D-screen in Entry 021, resurfacing in the residual analysis regardless of whether decoherence contamination is present.

Entry 026 — Conclusion

Subtracting the Entry 025 closed-form baseline from every recorded Bell-circuit decay curve leaves, on the seven properly screened edges, no discernible power-law structure — residuals scattered near zero with inconsistent sign. The most defensible reading is that Entries 017–021's empirically fitted forgiveness ratio was mostly re-deriving the fixed geometric baseline, not discovering new hardware physics on top of it. The one edge with a genuinely large residual, kyiv(28,29), is explained by a numerical artifact of the ratio metric at small p, not by an inconsistency in the underlying physics — its raw decoherence severity (D = 0.14) is comparable to, not dramatically worse than, cairo(19,22)'s (D = 0.31), yet the ratio-based residual differs by more than an order of magnitude. Ratio metrics should not be trusted at low raw error, independent of D.

4. What This Means for QuantumBridge

variable_forgiveness_ratio() should be retired, not re-fitted. With no clean residual signal to swap in, the empirically fit power law (Entries 017–021) was mostly measuring the closed-form baseline under a different name. The correct replacement is the derived formula itself — deterministic, requires no calibration data to fit, and is already verified to within shot noise.

The real physical signal — decoherence — is already handled correctly elsewhere: v4's T2-based dephasing term (Entries 023–024), which reads coherence times directly rather than inferring them indirectly through an unstable ratio. Asking a forgiveness-ratio fit to implicitly capture decoherence was always going to be fragile at low error magnitudes; this entry shows exactly why.

5. Next Step — Entry 027

Replace `variable_forgiveness_ratio()` in v4 with the closed-form baseline directly, remove the empirical power-law fit and its two hardcoded anchor points entirely, and re-validate against the Entry 022 Kyiv reference circuits to confirm the simplification doesn't regress Entry 024's routing fix. If it holds, the six-edge decay-curve protocol itself may no longer be necessary for future chips — only the D-screen and T2/T1 calibration export would be, considerably cutting the calibration cost the project has been carrying since Entry 013.

Research Log — Index Addendum (Entry 026)

Entry	Title	Date	Status
Entry 026	Baseline-Subtracted Residual Analysis	Aug 13, 2026	Complete

Entries 017-021's forgiveness-ratio power law is superseded: not wrong as a measurement, but shown to be mostly re-deriving the Entry 025 geometric baseline rather than capturing independent chip physics.

ENTRY 027 Retiring variable_forgiveness_ratio() — August 13, 2026 — Status: COMPLETE

Entry 027 — The Fitted Correction Comes Out, the Derived One Goes In

Generalizing Entry 025's Closed Form to Real Multi-Edge Routes, and Checking It Doesn't Cost Anything

1. What Was Done

Entry 026 found no clean residual left after subtracting the Entry 025 closed-form baseline from every recorded decay curve — the empirically fitted `variable_forgiveness_ratio()` power law was mostly re-deriving that baseline, not new chip physics. Its recommendation was to retire the fitted correction and use the derived formula directly. This entry does that.

Entry 025's closed form was derived for one edge's SAME depolarizing channel applied n times. Real routes cross several different edges with different raw errors, so the absorbing-state argument needed generalizing first: each two-qubit gate application either fires the channel (probability p_i, sending the state to maximally mixed) or doesn't (probability $1–p_i$) — independent of every other gate, and firing anywhere is absorbing. So the state survives untouched with probability $\prod_i(1–p_i)$, and expected success is floor + $(1–\text{floor})\cdot\prod_i(1–p_i)$, where floor is the ideal outcome set's share of the full outcome space (0.5 for a 2-qubit Bell pair, 0.25 for a 3-qubit GHZ state). No fitted coefficients — only the raw calibrated error and a combinatorial floor.

Implemented in v4 (`route_cost`/`predict`) and the exact-dwell router (`exact_dwell_cost`): gate-error survival is now accumulated as one product across the whole circuit and converted to a probability exactly once via the floor formula, instead of being forgiveness-corrected gate-by-gate and multiplied straight into a running success value. Decoherence, single-qubit gate cost, and readout are untouched — they were never part of the forgiveness fit.

2. Re-Validation Against the Entry 022 Kyiv Reference

circuit	Aer ref	v4.1 (old, fitted)	v4.1 (new, closed-form)	Δ
bell_adjacent_77_78	96.07%	gap 0.05	gap 0.09	+0.04
bell_near_77_82	48.74%	gap 2.39	gap 2.68	+0.29
bell_mid_77_100	83.69%	gap 19.07	gap 19.11	+0.04
bell_far_0_126	69.90%	gap 45.83	gap 43.79	-2.04
ghz_local_77_78_79	88.53%	gap 0.32	gap 0.76	+0.44
MEAN		13.54	13.28	-0.26
MEDIAN		2.39	2.68	+0.29
WORST		45.83	43.79	-2.04

All gaps in percentage points. Mean and worst-case both moved slightly in the derived model's favor; median moved slightly against it. The differences are within a point either way — a wash, not a regression.

Entry 027 — Conclusion

Swapping the fitted `variable_forgiveness_ratio()` power law for the derived closed-form floor changes accuracy on the Entry 022 Kyiv reference set by less than a point in either direction on every circuit tested (mean gap 13.54 → 13.28, worst-case 45.83 → 43.79). That confirms Entry 026's diagnosis directly: the fitted ratio was buying essentially nothing beyond what the derived formula already provides for free, while carrying two chip-specific fitted coefficients, an explicit fit-range caveat, and a dependency on future recalibration for every new chip. The derived version needs none of that — it falls out of the physics of a depolarizing channel and the size of the circuit's ideal outcome set.

3. An Honest Complication, Not Fixed by This Entry

bell_mid_77_100 (19-point gap) and bell_far_0_126 (44-point gap) remain badly wrong under both the old and new models, essentially unchanged by this substitution. Both are the two longest SWAP chains in the reference set (bell_far alone crosses 25 real swap legs). This is a pre-existing, unsolved problem — not something Entry 027 caused or was expected to fix, since it only touched how gate error is aggregated, not the routing or decoherence-integration logic Entries 023/024 built. It is flagged here rather than left implicit: whatever is wrong with the emulator's long-route predictions is still wrong, and is the most promising lead for a future entry.

4. Next Step — Entry 028

Investigate the long-route gap directly: bell_mid_77_100 and bell_far_0_126 are the two worst-predicted circuits in the entire reference set, by a wide margin, and share the trait of crossing many SWAP legs. Candidates to isolate: whether decoherence integrated over 20+ sequential legs is itself miscalibrated at that length, whether Aer's own routing diverges from the exact-dwell router's choice on long paths under a different seed, or whether an effect this project hasn't modeled yet (crosstalk between simultaneously-idle spectator qubits, for instance) only becomes visible once a route is long enough.

Research Log — Index Addendum (Entry 027)

Entry	Title	Date	Status
Entry 027	Retiring variable_forgiveness_ratio() for the Closed-Form Floor	Aug 13, 2026	Complete

variable_forgiveness_ratio() and its two fitted coefficients (forgiveness_law.json) are superseded in the v4/v4.1 codepath by emulator_v4.ideal_set_floor(), a derived closed form requiring no calibration data to fit. Retained in emulator_v3_routing.py unmodified, used only as the historical v3 baseline for comparison.

ENTRY 028 Two Decoherence Bugs, One Wrong Channel — August 13, 2026 — Status: COMPLETE

Entry 028 — Dephasing Was Never the Channel That Mattered

Isolating Gate Error, Decoherence, and Readout Separately Against Aer Exposes Two Bugs in the Decoherence Term Built in Entry 023

1. Why This Investigation Started

Entry 027 left two circuits badly wrong: bell_mid_77_100 (19-point gap) and bell_far_0_126 (44-point gap), both unchanged by that entry's gate-error simplification. Rather than guess, each contributing term -- gate error, decoherence, readout -- was isolated separately in Aer against bell_mid_77_100's real 5-leg route and compared to what the emulator predicts for that term alone.

term	emulator predicts	Aer isolated	gap
gate error (depolarizing)	90.16%	91.04%	0.88 pts
readout error	92.50%	92.66%	0.16 pts
decoherence	77.6%	94.80%	17.2 pts

Gate error and readout both check out almost exactly. Decoherence is off by 17 points on its own, on a single circuit -- clearly the source of the two entries' unexplained gap.

2. Bug One: Charging Both Ends of Every SWAP

exact_dwell_cost() (Entry 024) charged decoherence to BOTH physical qubits of every SWAP leg. But only one of the two is actually carrying a logical qubit's payload at any given hop -- the other is an idle ancilla being swapped into position. v4's own BFS-guess model (route_cost(), Entry 023) already charges only the single carrier qubit per hop, matching its own stated design intent ("integrate decoherence along the CARRIER path only") -- the exact-route function introduced in Entry 024 simply didn't carry that rule over. Fixed by tracking, at each SWAP, which of its two physical qubits currently holds a pending logical qubit and charging only that one.

3. Bug Two: The Wrong Coherence Channel Entirely

Fixing bug one alone wasn't enough. The deeper problem is Entry 023's original physical reasoning: it charged decoherence using T2 (dephasing time), on the argument that Bell/GHZ states carry their information in phase. That reasoning has it backwards. Pure T2 dephasing destroys OFF-DIAGONAL coherence in the computational basis -- but every circuit in this project measures success as population landing in {00...0, 11...1}, which are diagonal elements. Verified directly: an isolated two-qubit Aer test with T1 set near-infinite and T2 = 20 µs (aggressive dephasing, no relaxation at all) gave **100.00%** Bell-pair success. Pure dephasing was completely invisible to this measurement. The same test with T1 = 20 µs instead (T2 = 2×T1, i.e. no dephasing beyond the T1 limit) gave 97.6% -- consistent with the real order-of-magnitude loss. It is T1 relaxation, population leaking toward |0⟩, that actually corrupts these outcomes by leaking probability into {01,10} -- not T2.

dephasing() now uses T1 instead of T2, wherever it's called from (both route_cost() and exact_dwell_cost()). The function name is left unchanged since every call site still refers to it, but the docstring now records this correction and the isolated test that motivated it.

4. Re-Validation Against the Entry 022 Kyiv Reference

circuit	Aer ref	gap before (Entry 027)	gap after (Entry 028)
bell_adjacent_77_78	96.07%	0.09	0.15
bell_near_77_82	48.74%	2.68	10.81
bell_mid_77_100	83.69%	19.11	3.39
bell_far_0_126	69.90%	43.79	11.21
ghz_local_77_78_79	88.53%	0.76	0.10

circuit	Aer ref	gap before (Entry 027)	gap after (Entry 028)
MEAN		13.28	5.13
MEDIAN		2.68	3.39
WORST		43.79	11.21

Entry 028 — Conclusion

Fixing both bugs together drops the mean prediction gap across the Kyiv reference set from 13.28 to 5.13 points and the worst case from 43.79 to 11.21 -- the largest single accuracy gain of the project since Entry 024's routing fix. bell_near_77_82 is the one circuit that got nominally worse (2.68 → 10.81 points), and that's worth being honest about rather than hiding: its old "good" number came from two bugs partially cancelling -- overcharging decoherence via the wrong channel on a route that happens to dwell on catastrophically low-T2 qubit 80 -- not from the model being more correct. A model that is right for the wrong reasons on one circuit and wrong elsewhere is worse than one that is consistently closer everywhere.

5. What's Still Unresolved

bell_near_77_82 and bell_far_0_126 both still carry double-digit gaps (10.81 and 11.21 points). Both routes cross qubit 80 or long chains with several low-to-mid T2 qubits, so a residual decoherence-modeling gap likely remains -- possibly in how T1 combines with the readout and gate-error terms at the point of measurement, or in whether a single T1-based formula is adequate for qubits with unusually short T1 relative to the gate durations they experience. Not chased further in this entry; flagged for Entry 029.

6. Next Step — Entry 029

Investigate the remaining bell_near_77_82 / bell_far_0_126 gap specifically on qubit 80's role (T2 = 8.51 µs, the same qubit Entry 023 originally flagged) -- isolate its T1 in Aer the same way this entry isolated dephasing generally, and check whether the single T1-based exponential formula still holds at its extreme. If it does not extend the current model's accuracy further, task #21 (extending the D-screen protocol to Sherbrooke and Cairo) is the next candidate.

Research Log — Index Addendum (Entry 028)

Entry	Title	Date	Status
Entry 028	Fixing Two Decoherence Bugs (Double-Charge, Wrong Channel)	Aug 13, 2026	Complete

emulator_v4.dephasing() now uses T1 (relaxation) instead of T2 (dephasing) -- pure T2 dephasing is invisible to a computational-basis {00,11}/{000,111} measurement. exact_dwell_cost() now charges decoherence to a route's single carrier qubit per SWAP leg, matching v4's own BFS-model design intent.

ENTRY 029 — A Provably Correct Fix That Made the Model Worse — August 13, 2026 — Status: COMPLETE (fix not shipped)

Entry 029 — When Being More Right Makes the Model More Wrong

Chasing the Remaining bell_near / bell_far Gap Finds a Real Formula Bug, and a Reason Not to Fix It Yet

1. The Remaining Gap After Entry 028

bell_near_77_82 (10.81-point gap) and bell_far_0_126 (11.21-point gap) were still the two worst circuits after Entry 028's fixes. Both routes dwell on qubit 80, T1 = 5.0 µs -- the same extreme-decoherence qubit Entry 023 originally flagged. Isolating gate error, decoherence, and readout separately for the single worst leg (the (80,79) SWAP in bell_near's route) against Aer found gate error and readout both matched closely, but the emulator's decoherence-only prediction for that one leg (70.58%) was far below Aer's isolated result (93.15%).

2. A Real, Derivable Bug -- Verified

Every decoherence call site since Entry 023 has computed a hazard probability $\gamma = 1 - e^{-t/T1}$ and used $(1 - \gamma)$ directly as the success multiplier -- treating amplitude damping as an all-or-nothing failure. Working through the Kraus operators for one qubit of a Bell pair under amplitude damping shows this is wrong: the decay-triggered branch (probability $\gamma/2$) always leaks outside {00,11}, but the no-decay branch (probability $1 - \gamma/2$) ALWAYS still lands inside it. The correct success factor is $1 - \gamma/2$, not $1 - \gamma$. Verified directly: an isolated single-qubit amplitude-damping Aer test at four T1 values (5, 10, 20, 50 µs) matched $1 - \gamma/2$ to within shot noise at every single point.

3. Applying It Made Things Worse

The fix was implemented and re-validated against the full Entry 022 Kyiv reference set. bell_mid_77_100 and bell_far_0_126 both improved. bell_near_77_82 got dramatically worse -- its gap nearly tripled, from 10.81 to 24.08 points -- because relaxing the decoherence penalty pushed an already-too-optimistic prediction even further from the real answer. On the full 5-circuit set, mean gap moved from 5.13 to 6.61 points and worst-case from 11.21 to 24.08. A four-way sweep (single-carrier vs. both SWAP endpoints charged, times γ vs. $\gamma/2$) confirmed Entry 028's exact combination -- single carrier, full γ -- is the best-performing option of the four on this reference set, despite $\gamma/2$ being the more physically correct formula in isolation.

combination	mean gap	worst gap
single carrier, full γ (Entry 028, shipped)	5.14	11.08
single carrier, $\gamma/2$ (this entry's attempt)	6.66	24.25
both endpoints, full γ	6.45	19.33
both endpoints, $\gamma/2$	5.62	13.23

Entry 029 — Conclusion

The correct single-qubit formula does not survive contact with the full model. A naive check confirms why: multiplying Aer's own individually-isolated gate-error, decoherence, and readout success numbers together for bell_near's route gives roughly 72% -- but the real combined-noise result is 49%. Even a perfect decomposition of each channel in isolation, recombined by simple multiplication, misses a 23-point interaction effect. Entry 028's technically-wrong γ formula was, on this route, accidentally absorbing some of that missing interaction term. Fixing the formula in isolation removed a real bug and simultaneously removed the accidental compensation for a different, larger, unmodeled one -- leaving the combined prediction worse. The $\gamma/2$ fix is NOT shipped. The codebase is unchanged from Entry 028.

4. What This Actually Means

The project's multiplicative model -- gate error, decoherence, and readout as independent probabilities multiplied together -- has a structural limit on routes that dwell on extreme-T1 qubits. The channels are not independent there: whatever damage decoherence does to the state appears to make it MORE vulnerable to the other error sources than treating the channels separately predicts. Closing this gap for real would require modeling that interaction directly, not

patching the decoherence formula alone -- a materially bigger task than this entry's scope.

5. Next Step

Given the interaction effect is specific to extreme-T1 routes and the current model (Entry 028's state) already gives a 5.13-point mean gap across the reference set -- a large improvement over where the project stood two entries ago -- task #21 (extending the D-screen protocol to Sherbrooke and Cairo) is the better use of the next entry. The channel-independence question is left open, documented here rather than chased further, and can be revisited if a future chip's reference set turns up the same pattern.

Research Log — Index Addendum (Entry 029)

Entry	Title	Date	Status
Entry 029	A Correct Formula Fix That Regressed the Combined Model	Aug 13, 2026	Complete (not shipped)

t1_survival() (1 - gamma/2) is derived and verified correct in isolation but NOT applied in emulator_v4.py or exact_dwell_routing.py -- it regresses the Entry 022 Kyiv reference set. Codebase left at Entry 028's state. Documented as an open channel-independence question for future chips.

ENTRY 030 Extending Validation to Sherbrooke — August 13, 2026 — Status: COMPLETE

Entry 030 — A Second Chip Confirms One Fix and Surfaces a New Gap

Task #21: Extending the Kyiv-Validated Model to Sherbrooke, and Why Cairo Is Excluded

1. Scope Decision: Sherbrooke Yes, Cairo Not Yet

Sherbrooke (127 qubits) shares Kyiv's uniform ecr-only basis, so it can get the same Aer/MPS ground-truth validation used since Entry 022. Cairo (27 qubits) genuinely mixes cx and ecr per edge -- forcing an ecr-only basis for a Cairo circuit throws a disconnected-coupling-map error, confirming this is a real hardware property, not an artifact of an old validation harness. Entry 022 already flagged Cairo as unsuitable for circuit-level validation for this reason. Cairo is excluded from this entry's full-circuit test; extending to it would need a validation approach that respects per-edge native gates rather than a single forced basis, left for a future entry.

2. Sherbrooke Reference Set and Results

Five circuits built the same way as the Entry 022 Kyiv set, using real Sherbrooke qubit pairs: an adjacent pair, a route deliberately chosen through qubit 57 (T2 = 2.6 µs, the chip's worst by far, but T1 = 301.9 µs -- healthy), a 12-hop and a 26-hop pair, and a local GHZ. Ground truth from Aer/MPS against Sherbrooke's real noise model, 4 seeds × 4096 shots.

circuit	Aer ref	v4.1 predicted	gap
bell_adjacent_0_1	95.53%	95.53%	0.00
bell_near_57_61	95.08%	79.10%	15.98
bell_mid_0_58 (12 hops)	49.40%	66.97%	17.57
bell_far_0_126 (26 hops)	49.51%	64.41%	14.90
ghz_local_0_1_14	91.48%	90.43%	1.05
MEAN / WORST			9.90 / 17.57

Mean gap 9.90 points, worst 17.57 -- both real improvements over v3 (13.23 mean) and v4-BFS (13.84 mean) on this same set, but noticeably worse than the 5.13-point mean the same v4.1 model achieves on Kyiv. The model generalizes -- imperfectly.

3. Confirmation: T1, Not T2, Is What Matters

bell_near_57_61's route was chosen deliberately to cross qubit 57, the chip's single worst T2 (2.6 µs) by a wide margin -- yet its real measured success (95.08%) was barely below the clean adjacent circuit's (95.53%). Under the OLD (pre-Entry-028) T2-based decoherence formula this route would have predicted catastrophic loss. Under the current T1-based formula it doesn't, because qubit 57's T1 (301.9 µs) is healthy -- and the real hardware agrees almost exactly. This is independent confirmation, on a chip never used to derive or tune the fix, that Entry 028's T2-to-T1 correction was the right one.

Entry 030 — Conclusion

The v4.1 model built and tuned entirely on Kyiv transfers to Sherbrooke without modification and beats the older v3/v4-BFS models there too -- and the T1-vs-T2 physics from Entry 028 holds up on a qubit the fix was never calibrated against. But the mean gap nearly doubles (5.13 → 9.90), so "generalizes" should not be read as "transfers cleanly." The two long routes (mid, far) are both over-predicted by 15-18 points, and the near-57 route is under-predicted by 16 -- different circuits failing in different directions, which rules out one single missing constant fixing all of them.

4. A New Lead: An Outlier Edge That Doesn't Behave Like a Depolarizing Channel

Isolating gate error alone for bell_near_57_61's route in Aer found the edge (57,58) -- reported gate_error 11.68%, roughly 15× the chip's median (0.78%) and the largest of only 4 edges on the whole chip above 5% -- produces far

less real damage (98.71% isolated survival) than modeling it as a depolarizing channel of that magnitude predicts (83.5%, via the same closed form that matched Kyiv's edges to within about a point). Either this specific calibration entry doesn't correspond to a simple depolarizing process, or extreme outlier edges behave differently from the typical edges the closed-form baseline was derived and verified against. Not resolved in this entry -- flagged as the likely largest single contributor to Sherbrooke's residual gap.

5. Next Step — Entry 031

Isolate edge (57,58) the way Entry 025 isolated a single Kyiv edge: sweep gate count on that one edge alone under a pure depolarizing channel at p=11.68% and check whether the closed-form baseline still matches at this much larger error magnitude, or whether it was only verified in a regime (Kyiv's edges topped out near 4.3%) that doesn't extend to genuine outliers. If the baseline breaks down at high p specifically, that would explain both the near-57 gap here and motivate re-checking whether any Kyiv edges near the top of its range carry the same undetected issue.

Research Log — Index Addendum (Entry 030)

Entry	Title	Date	Status
Entry 030	Extending Validation to Sherbrooke; T1 Fix Confirmed, New Outlier-Edge Gap Found	Aug 13, 2026	Complete

quantumbridge_data/coherence_sherbrooke.json and coherence_cairo.json exported. Cairo full-circuit validation deferred (mixed cx/ecr basis, Entry 022). Sherbrooke reference set committed as validate_v4_sherbrooke.py, results saved to quantumbridge_data/entry030_sherbrooke_validation.json.

ENTRY 031 The Model Doesn't Know When Entanglement Actually Happens — August 13, 2026 — Status: COMPLETE

Entry 031 — Route-Then-Entangle: A Structural Assumption Error

The Outlier Edge Was a Red Herring; the Real Bug Is About WHEN a SWAP Chain Actually Starts Carrying Entangled Information

1. Ruled Out: The Closed Form Doesn't Break at High p

Entry 030 flagged Sherbrooke edge (57,58) -- gate_error 11.68%, ~15x the chip median -- as a possible case where the closed-form baseline (verified on Kyiv up to ~4.3% error) simply doesn't hold at larger magnitudes. Isolating that exact edge alone, sweeping gate count the same way Entry 025 did, found the closed form matches Aer to within 0.13 points on average even at p=11.68% -- essentially exact. The formula itself is not the problem, at any error magnitude tested so far.

2. The Real Bug

Re-isolating the FULL bell_near_57_61 route (4 real edges, matching the exact transpiled circuit used for the Aer reference) reproduced Entry 030's gap directly: 98.71% real gate-error-only survival against 83.5% predicted. Since the per-edge formula is now proven correct, the bug had to be in how results from different edges combine. A minimal toy circuit isolated it: build a Bell pair by applying H to one qubit, then routing BOTH halves toward each other via SWAPs, and creating the actual entanglement with a SINGLE cx/ecr gate only once they're adjacent -- exactly what the real transpiler does for these circuits, confirmed by inspecting the transpiled gate sequence directly (only one non-SWAP two-qubit gate appears, at the very end).

Every model since Entry 023 assumed the opposite: that entanglement exists from the start and every SWAP afterward is moving an already-entangled payload, equally vulnerable to depolarizing damage at every hop. That assumption is wrong for exactly the circuits this project measures. Before the final entangling gate, the two qubits being routed toward each other are carrying ordinary, UNENTANGLED product-state information -- and a depolarizing event on a product state does not destroy a correlation that doesn't exist yet. It does less damage to the final {00,11} outcome than the same event would do to an already-entangled pair.

test	Aer	closed-form (old model)	diff
toy: entangle-first-then-route (wrong structure)	50.32%	93.62%	-43.30
toy: route-first-then-entangle (real structure)	99.29%	93.62%	+5.67
real bell_near_57_61, gate-error-only	98.71%	83.52%	+15.19

The first row is a deliberately broken toy (re-entangling an already-entangled pair) included only to show how sensitive this structure is -- getting the entanglement-timing wrong doesn't cause a small error, it inverts the whole prediction. The second row, with the structure corrected to match what Qiskit's transpiler actually produces, shows the SAME direction and rough scale of under-prediction as the real circuit in row three. That consistency is what makes this the real explanation rather than a coincidence.

Entry 031 — Conclusion

The persistent over-prediction of damage on long, multi-hop routes (bell_mid and bell_far on both Kyiv and Sherbrooke, and now bell_near_57_61) traces to one structural assumption: exact_dwell_cost and route_cost both treat every SWAP identically, as if it always carries an already-entangled qubit. Real transpiled circuits for a single Bell/GHZ preparation typically route the uninvolved qubits into position FIRST and entangle them LAST, meaning most of a long route's SWAP legs are moving product-state information that is measurably less vulnerable to depolarizing noise than the model assumes. This is not a tuning problem or an edge-magnitude problem -- it is a category error in what physical process the model thinks is happening during most of a circuit's real gate sequence.

3. Why This Wasn't Caught Earlier

Entry 025's isolation test, which the closed form was derived and verified against, deliberately used "cancelling CX pairs" applied repeatedly to an circuit that is ALREADY entangled from the first gate -- a clean, controlled test of the

compounding formula, but one that never exercises the pre-entanglement regime a real multi-hop route spends most of its gates in. The formula was proven correct in exactly the case it was tested in, and that case turns out not to be representative of the majority of gates in a long real route.

4. Next Step — Entry 032

This entry identifies the mechanism but does not yet fix it -- deriving the correct closed form for a depolarizing event on a pre-entanglement product-state SWAP (as opposed to a post-entanglement one) needs its own Kraus-operator derivation, analogous to Entry 025's and Entry 029's, before it can be verified and shipped. The practical implication for exact_dwell_cost: gate error needs to be split into two categories per route -- SWAPs before the pending pair's entangling gate, and SWAPs/gates after it -- rather than one uniform survival product across the whole route.

Research Log — Index Addendum (Entry 031)

Entry	Title	Date	Status
Entry 031	Route-Then-Entangle: A Structural Assumption Error in Gate-Error Modeling	Aug 13, 2026	Complete (root cause found, fix pending)

entry031_high_p_isolation.py confirms the closed-form baseline holds at p=11.68% (rules out edge-magnitude as the cause). entry031_route_then_entangle.py isolates and confirms the real cause. Neither emulator_v4.py nor exact_dwell_routing.py is fixed yet -- left for Entry 032.

ENTRY 032 Deriving and Shipping the Route-Then-Entangle Fix — August 13, 2026 — Status: COMPLETE

Entry 032 — The Control Qubit's Routing Errors Don't Matter

A Kraus-Operator Derivation, Verified by an Error Sweep That Shouldn't Move the Answer -- and Doesn't

1. The Derivation

Entry 031 found that most SWAP legs on a real route move UNENTANGLED product-state qubits, not an already-entangled payload. This entry works out exactly what that means. Call the qubit that receives the initial H the CONTROL (it carries |+⟩ before any routing) and the qubit that starts at |0⟩ the TARGET. Tracking the four combinations of "control corrupted / clean" × "target corrupted / clean" through the final entangling gate via its Kraus operators: if the control gets forced to a maximally mixed state before the final gate, it is now a CLASSICAL random bit (diagonal in the computational basis) -- combining a classical random control with a clean target through a CX/ECR still produces a definite, correlated pair (00 or 11), so it costs nothing. If the TARGET gets corrupted instead, the final gate combines a genuine coherent superposition with a classically randomized partner, producing a 50/50 mixture of a correct and an incorrect Bell state -- landing exactly at the floor. The two roles are not interchangeable.

Closed form for a single-pair circuit: success = target_survive × (1 – final_raw/2) + (1 – target_survive) × 0.5, where target_survive is the product of survival across only the target's routing legs and final_raw is the entangling gate's own error. In the no-routing limit (target_survive = 1) this reduces exactly to Entry 025's single-gate formula, 0.5 + 0.5(1–p).

2. Verified: Zero Sensitivity to the Control's Error

Sweeping the control-side edge's depolarizing error from 1% to 60% in an Aer simulation of the exact bell_near_57_61 route structure left the measured success probability completely unchanged at every point (99.286%, to three decimal places) -- while this entry's closed form (evaluated without depending on that edge at all) matched it to within 0.11 points throughout. This is about as clean a confirmation as this kind of derivation gets.

3. Scope and Implementation

Implemented in exact_dwell_cost() for single-pair (plain Bell) circuits only -- 4 of the 5 circuits in every reference set used this session. Multi-pair (GHZ-style) circuits keep the old floor-based model unchanged, since later entangling gates there act on qubits already entangled from earlier gates, and the derivation above assumed a fresh product-state input -- extending it correctly would need its own derivation, not attempted here.

4. Results

circuit (chip)	Aer ref	gap before	gap after
bell_adjacent_77_78 (Kyiv)	96.07%	0.09	0.15
bell_near_77_82 (Kyiv)	48.74%	10.81	11.65
bell_mid_77_100 (Kyiv)	83.69%	3.39	2.31
bell_far_0_126 (Kyiv)	69.90%	11.21	6.07
ghz_local_77_78_79 (Kyiv)	88.53%	0.10	0.10
KYIV MEAN / WORST		5.13 / 11.21	4.06 / 11.65
bell_adjacent_0_1 (Sherbrooke)	95.53%	0.00	0.00
bell_near_57_61 (Sherbrooke)	95.08%	15.98	1.65
bell_mid_0_58 (Sherbrooke)	49.40%	17.57	20.19
bell_far_0_126 (Sherbrooke)	49.51%	14.90	20.39
ghz_local_0_1_14 (Sherbrooke)	91.48%	1.05	1.05
SHERBROOKE MEAN / WORST		9.90 / 17.57	8.66 / 20.39

Entry 032 — Conclusion

Mean gap improved on both chips (Kyiv 5.13 → 4.06, Sherbrooke 9.90 → 8.66), and bell_near_57_61 -- the specific circuit this whole investigation started from -- dropped from a 15.98-point gap to 1.65, close to shot noise. That is a genuine, derived, twice-verified fix, and it ships as of this entry. It is not a clean sweep, though: bell_mid and bell_far got WORSE on Sherbrooke (17.57 → 20.19 and 14.90 → 20.39) and bell_near got marginally worse on Kyiv (10.81 → 11.65). Removing gate-error charges from the control's path made the model more optimistic everywhere that charge used to apply -- correctly so, per the derivation -- but on the very longest routes, the real measured success is low enough that removing ANY pessimistic term moves the prediction further from reality. That is not evidence the fix is wrong; the control-sweep test proves it isn't. It is evidence that bell_mid and bell_far have a separate, still-unidentified source of loss big enough that it was previously masked by an unrelated bug's over-charging.

5. Next Step — Entry 033

Isolate bell_far_0_126 and bell_mid_0_58/bell_mid_77_100 the same way this entry isolated bell_near_57_61: ablate gate error, decoherence, and readout separately against Aer for these SPECIFIC long routes now that the route-then-entangle and control/target asymmetry are both accounted for, to find what's actually left unmodeled. Candidates worth checking first: whether the TARGET's own long chain has a similar asymmetry still unaccounted for once it spans many hops, or whether something entirely new (crosstalk between simultaneously-active spectator qubits on a 26-hop route, for instance) only becomes visible at this route length.

Research Log — Index Addendum (Entry 032)

Entry	Title	Date	Status
Entry 032	Route-Then-Entangle Fix Shipped: Control-Side Gate Error Doesn't Matter	Aug 13, 2026	Complete

exact_dwell_routing.py's exact_dwell_cost() now splits single-pair gate-error handling into a target-survival closed form, verified via a control-edge sensitivity sweep. GHZ/multi-pair circuits unchanged. bell_mid/bell_far remain open (Entry 033).

ENTRY 033 Confirming the Channel-Independence Ceiling — August 13, 2026 — Status: COMPLETE

Entry 033 — Every Formula Is Right, and the Route Still Loses 20 Points

bell_far_0_126's Regression Traces to the Same Structural Limit Entry 029 Found, Now Confirmed on an Unrelated Route

1. Isolating bell_far_0_126 on Sherbrooke

Entry 032 improved the reference set overall but made bell_far_0_126 and bell_mid_0_58 worse (gaps up to 20.39 points). Rather than guess, each channel was isolated against Aer the same way Entries 028 and 029 did for bell_mid_77_100 and bell_near_57_61 -- this route crosses 25 real SWAP legs (76 total two-qubit gate applications) toward qubit 126, the longest in the whole project.

channel	emulator predicts	Aer isolated	gap
gate error (Entry 032 formula)	84.51%	83.28%	1.23 pts
decoherence (Entry 028 T1 formula)	86.44%	89.72%	3.28 pts
readout error	95.91%	95.70%	0.21 pts

Every single formula checks out well individually -- the largest gap is 3.3 points, on decoherence. None of Entries 028's, 029's, or 032's fixes are wrong.

2. Where the Gap Actually Comes From

Multiplying the three Aer-ISOLATED results together (as if the channels were independent) gives 0.8328 × 0.8972 × 0.9570 = 71.5%. The REAL combined-noise Aer result for this circuit is 49.51%. A 22-point gap appears purely from treating three genuinely well-measured, individually-correct channels as if they don't interact -- with nothing left to blame on any of this project's formulas, since each one was just independently verified against its own isolated ground truth.

Entry 033 — Conclusion

This is not a new problem. It is Entry 029's finding -- "even a perfect decomposition of each channel in isolation, recombined by simple multiplication, misses a real interaction effect" -- showing up again on a completely different route, on a different chip, driven by a different cause (25 accumulated SWAP legs here, versus one catastrophically low-T1 qubit on Kyiv's bell_near_77_82). Two independent circuits now show the same signature: individually correct channels, multiplicatively combined, systematically overshoot real fidelity once a route accumulates enough total exposure -- whether that exposure comes from one extreme qubit or from many ordinary ones. This confirms it as a general property of the model's multiplicative-composition assumption, not a coincidence specific to qubit 80 on Kyiv.

3. Why This Isn't Chased Further Right Now

Fixing this for real requires modeling how gate error and decoherence correlate along a route -- effectively simulating correlated noise rather than composing independently-measured channel probabilities, which is close to what Aer itself already does at the circuit level. That is a materially different, larger undertaking than any fix shipped this session, and outside a closed form the rest of this project's formulas (Entries 025, 028, 032) were built around. The current model's real, verified achievement stands: mean gap on Kyiv's reference set fell from 13.28 (Entry 026) to 4.06 points (Entry 032), and on Sherbrooke from 9.90 to 8.66, with every individual physical channel -- gate error, decoherence, readout -- now independently verified correct. The two longest routes on each chip are a known, understood, documented ceiling on that approach, not an unexplained failure.

4. Where This Leaves the Project

The physics-debugging arc that ran from Entry 022 through this entry has taken v4.1 from a validation harness that silenced 53% of its own noise model to a closed-form emulator with no fitted, chip-specific coefficients, verified on two chips it wasn't tuned on. The remaining gap is well-characterized rather than mysterious. That is a reasonable, honest stopping point for this arc -- the next entry moves to building the batch-circuit dataset for the ML phase, using this

validated model as the benchmark any learned model will need to beat.

Research Log — Index Addendum (Entry 033)

Entry	Title	Date	Status
Entry 033	Confirming the Channel-Independence Ceiling on a Second Route	Aug 13, 2026	Complete

No code changes. Confirms Entry 029's channel-independence finding generalizes beyond Kyiv's qubit 80 to any route with enough cumulative exposure. Closes the Entries 022-033 physics-debugging arc; next entry begins the ML batch-dataset phase.

ENTRY 034 The Batch-Circuit Dataset — August 16, 2026 — Status: COMPLETE

Entry 034 — From 10 Hand-Picked Circuits to 116 Sampled Ones

Batch Methodology Surfaces a New Failure Pattern the Reference Sets Never Showed

1. Why Batch Instead of One-Off

Every finding from Entry 022 through Entry 033 rested on 5 hand-picked circuits per chip (adjacent / near / mid / far / one local GHZ) -- the right tool for chasing an individual bug, but a 10-point sample for any claim about how the model behaves in general. entry034_batch_dataset.py instead samples qubit pairs stratified across six BFS hop-distance bins (1, 2-3, 4-7, 8-15, 16-25, 26+), builds a plain Bell circuit for each, runs it through the same Aer/MPS ground-truth pipeline used since Entry 022 (4 seeds × 4096 shots, optimization_level=3), and records the v4.1 (Entry 032) prediction alongside it -- plus a smaller batch of 3-qubit GHZ circuits for structural variety. Every result is checkpointed to JSON as it completes, so the batch can run in resumable chunks.

chip	bell circuits	ghz circuits	total	mean gap	median gap	worst gap
Kyiv	45	10	55	5.38 pts	2.18 pts	31.39 pts
Sherbrooke	51	10	61	7.48 pts	2.14 pts	40.89 pts

116 circuits total, roughly 12x the combined size of every reference set built so far this project.

2. The Expected Pattern: Gap Grows With Route Length

Bucketing by hop distance confirms what the hand-picked circuits suggested but never proved at scale: adjacent pairs (1 hop) predict almost perfectly (median gap 0.1-0.2 points on both chips), and the median gap climbs steadily to 4-14 points by the 16-25 hop bin. Correlation between gap and hop distance across all 96 Bell circuits is +0.20 -- real but modest, because hop distance alone does not explain the worst outliers (below).

3. The New Finding: Short Routes That Collapse to the Floor

Sorting by gap regardless of distance surfaces something the 5-circuit reference sets never could: 7 circuits, 5 of them on Sherbrooke, whose REAL Aer result sits within 2 points of the 50% floor -- the outcome of complete entanglement failure -- despite short 3-9 hop routes the model predicts should succeed 60-91% of the time.

chip	pair	hops	Aer (real)	v4.1 predicts	gap
Sherbrooke	3 → 6	3	50.0%	88.5%	38.5 pts
Sherbrooke	2 → 6	4	50.2%	91.1%	40.9 pts
Sherbrooke	11 → 22	9	48.9%	77.2%	28.3 pts
Sherbrooke	28 → 5	7	50.3%	84.8%	34.5 pts
Kyiv	95 → 72	8	50.0%	60.9%	10.9 pts

Physical qubit 6 (readout error 25.7%, roughly 4-8x every neighboring qubit's readout error on Sherbrooke) and qubit 5 (T2 = 78.9us against a 172.7us T1 -- the largest T1/T2 split on any route in this batch) both recur across these routes. Neither value alone is extreme enough in the current model to explain a full collapse to the floor -- the model's readout and decoherence terms would only cost these circuits 10-20 points, not 35-40.

Entry 034 — Conclusion

The batch approach does exactly what it was proposed to do: instead of finding one bug at a time by hand-picking a circuit, running many circuits and aggregating surfaced a failure mode -- short routes that still crash to the 50% floor -- with 7 independent examples on the first pass, concentrated on two specific physical qubits. That is a concrete, high-confidence lead for the next entry, discovered by frequency rather than by guessing which circuit to isolate next.

4. What This Dataset Is Also For

Beyond bug-hunting, every one of the 116 records is saved with its chip, qubit pair, BFS hop distance, real transpiled edge/gate counts, worst raw edge error and worst T1 along the BFS path, the Aer ground truth, the v4.1 prediction, and the gap -- circuit-and-calibration features paired with a measured label. This is the first dataset in the project large enough to be a plausible starting point for the ML/GNN phase discussed since Entry 033, and it will keep growing the same way as more circuits get added in resumable batches.

5. Next Step

Entry 035: isolate the qubit-5/qubit-6 (Sherbrooke) floor-collapse outliers against Aer the same way Entries 028, 029, and 033 isolated earlier anomalies -- gate error, decoherence, and readout separately -- to find what specifically drives a short route all the way to 50%, since none of the three known channels individually accounts for a drop this size.

Research Log — Index Addendum (Entry 034)

Entry	Title	Date	Status
Entry 034	The Batch-Circuit Dataset (116 circuits, Kyiv + Sherbrooke)	Aug 16, 2026	Complete

Script: entry034_batch_dataset.py (resumable, checkpointed). Data: quantumbridge_data/entry034_batch_dataset.json (116 records), quantumbridge_data/entry034_batch_analysis.json (aggregated bins and outlier list). Surfaces a new floor-collapse pattern on 7 short routes for Entry 035 to isolate. Also the first dataset scaled for the ML/GNN phase.

ENTRY 035 Replication: Is It the Qubits, or the Chip? — August 16, 2026 — Status: COMPLETE

Entry 035 — A Second, Independent Batch Rules Out "Bad Qubits"

Two Experiments, Zero Shared Outlier Qubits, Same Pattern — This Is a Chip-Level Effect, Not a Qubit-Level One

1. The Question This Entry Answers

Entry 034's batch found 7 short routes (3-9 hops) that collapse completely to the 50% floor, clustered on Sherbrooke qubits 5 and 6. Before spending an entry chasing what makes those two qubits special, the proposed test was: run a second, independently-sampled batch with a different random seed and see whether the SAME qubits show up again (a qubit-specific defect worth isolating on its own) or whether a similar pattern reappears on DIFFERENT qubits (a general, chip-level effect worth chasing as one shared problem). entry035_replication_batch.py re-runs Entry 034's exact methodology with seed=99 instead of seed=7, drawing a fresh, non-overlapping set of qubit pairs.

2. Result: The Pattern Recurs, the Qubits Don't

	circuits	floor-collapse outliers	outlier qubits
Experiment 1 (Entry 034, seed=7)	116	7	97,114,95,72,3,6,2,5,28,86,91,11,22
Experiment 2 (Entry 035, seed=99)	122	5	83,103,82,84,73,80,40,56,15,9

Zero overlap. Not one physical qubit that produced a floor-collapse outlier in experiment 1 appears in experiment 2's outlier list, or vice versa -- despite both experiments sampling the same two chips with the same stratified hop-distance methodology. Overall gap statistics also matched closely between the two independent draws (mean 6.48 vs 6.10 points, gap-vs-distance correlation 0.20 vs 0.26), confirming this isn't two unrelated experiments -- it's the same underlying phenomenon sampled twice.

3. What Actually Distinguishes the Outliers: the Chip, Not the Qubit

Pooling both experiments and normalizing by how many short-route (≤10 hop) circuits were sampled on each chip:

chip	short-route circuits	floor-collapse	rate
Kyiv	65	2	3.1%
Sherbrooke	69	10	14.5%

Sherbrooke collapses to the floor on short routes roughly 4.7x more often than Kyiv, consistently across both independent samples. Both chips share the same uniform ecr-only basis and the same v4.1 model -- the difference has to come from something about Sherbrooke's actual topology or calibration surface that this project hasn't identified yet, not from a handful of bad qubits.

Entry 035 — Conclusion

The replication test gives a clear, decisive answer: floor-collapse is a general, recurring phenomenon rather than a defect isolated to qubits 5 and 6. It shows up on a completely different set of physical qubits in an independent sample, at a comparable rate, heavily skewed toward Sherbrooke over Kyiv. That rules out a qubit-specific patch as the right fix -- the next entry should isolate a FRESH instance of the collapse (not qubits 5/6, since those are now known not to be special) using the same gate-error / decoherence / readout separation Entries 028, 029, and 033 used, with the explicit goal of finding what property of the ROUTE or the CHIP -- not the qubit -- predicts it.

4. Next Step

Entry 036: pick 2-3 fresh floor-collapse instances from the combined pool of both experiments' outliers, isolate gate error, decoherence, and readout separately against Aer for each (as in Entries 028/029/033), and check whether the collapse traces to one specific channel consistently across instances despite the different qubits involved -- that would identify what the qubits have in common functionally, even without sharing physical identity.

Research Log — Index Addendum (Entry 035)

Entry	Title	Date	Status
Entry 035	Replication Batch — Floor-Collapse Is Chip-Level, Not Qubit-Level	Aug 16, 2026	Complete

Script: entry035_replication_batch.py (122 circuits, seed=99, reuses entry034_batch_dataset.py's methodology). Data: quantumbridge_data/entry035_replication_dataset.json, quantumbridge_data/entry035_comparison_summary.json. Zero qubit overlap with Entry 034's outliers; Sherbrooke's short-route floor-collapse rate (14.5%) is 4.7x Kyiv's (3.1%). Rules out a qubit-specific fix; next entry isolates channels on fresh instances.

ENTRY 036 Floor-Collapse Is the Independence Breakdown, Amplified — August 16, 2026 — Status: COMPLETE

Entry 036 — Every Channel Looks Fine in Isolation, Combined They Erase the Circuit

Three Fresh Instances, None Involving Qubits 5/6, Same Signature as Entries 029 and 033 -- Just Far Stronger and on Much Shorter Routes

1. Method

Three floor-collapse instances were picked from the combined Entry 034/035 outlier pool, deliberately avoiding qubits 5 and 6 (Entry 035 already showed they aren't special) and choosing cases where v4.1 predicted well above the floor -- genuine misses, not places the model already (correctly) predicted near-50%. Each was isolated against Aer into gate-error-only, decoherence-only, and readout-only noise models, the same ablation used in Entries 028, 029, and 033.

2. Result: Every Channel Checks Out, the Combination Doesn't

route	edges	gate err.	decoherence	readout	naive product	REAL combined	v4.1 predicts
Sherbrooke 3→6	3	97.7%	97.7%	91.9%	87.8%	50.0%	88.5%
Sherbrooke 40→56	5	93.3%	84.6%	90.8%	71.6%	50.7%	81.9%
Kyiv 95→72	8	90.4%	71.5%	99.4%	64.2%	50.0%	60.9%

In every instance, each individual channel isolated against Aer looks completely ordinary -- 72-98% survival, nothing alarming. Multiplying even Aer's own perfectly isolated per-channel results together lands at 64-88%, matching v4.1's prediction closely (as expected, since v4.1 is built on the same multiplicative-composition assumption). But the REAL combined-noise result on all three routes sits within 1 point of the 50% floor -- total entanglement failure, with nothing in any single channel foreshadowing it.

3. This Is Entries 029/033's Finding -- Just Much Stronger, on Much Shorter Routes

Entry 029 found an 11-point channel-independence gap on Kyiv's worst-T1 route. Entry 033 found a 22-point gap on Sherbrooke's longest route (25 SWAP legs). This entry's gaps are 37.8, 21.0, and 14.2 points -- comparable to or LARGER than Entry 033's, but on routes of just 3, 5, and 8 edges. Entry 033 concluded the breakdown was driven by "accumulated exposure," implicitly assuming more hops means more correlation. A 3-edge route producing the single largest independence gap measured anywhere in this project rules that assumption out: route length is not the variable that predicts how badly the channels interact. Something about which specific edges combine -- not how many -- drives the correlation strength.

Entry 036 — Conclusion

Floor-collapse is not a new bug and not a qubit defect -- it is the channel-independence breakdown documented in Entries 029 and 033, occurring far more severely than previously seen, and on routes short enough that route length alone cannot be the predictor. This raises the stakes on the limitation Entry 033 flagged but set aside: it isn't just costing a few points on the two longest routes per chip, it is erasing 14.5% of Sherbrooke's short routes completely. The multiplicative-composition framework this project's closed forms (Entries 025, 028, 032) are built on has a real, now well-evidenced ceiling, and no closed-form patch discovered so far reaches past it.

4. Where This Leaves the Project

Fixing this properly requires modeling correlated noise across a route rather than composing independently-measured channel probabilities -- effectively a step toward simulating the density matrix along the path instead of multiplying survival probabilities, which Entry 033 already identified as the real fix and set aside as out of scope for a closed-form model. That assessment stands. What changes here is urgency and framing: this is exactly the kind of structured, high-value target a learned model (GNN or otherwise) could plausibly pick up where the closed-form approach hits its ceiling -- the batch dataset from Entries 034/035 already contains 12 labeled floor-collapse examples for it to learn

from directly.

5. Next Step

Move into the ML/GNN phase using the Entry 034/035 batch dataset (238 circuits combined) as the training/validation set, with v4.1's predictions as the closed-form benchmark any learned model needs to beat -- particularly on the floor-collapse cases this entry shows the closed-form approach structurally cannot reach.

Research Log — Index Addendum (Entry 036)

Entry	Title	Date	Status
Entry 036	Floor-Collapse Is the Independence Breakdown, Amplified	Aug 16, 2026	Complete

Script: entry036_isolate_floor_collapse.py. Data: quantumbridge_data/entry036_isolate_floor_collapse.json. Confirms floor-collapse is Entries 029/033's channel-independence breakdown, stronger and on shorter routes than previously seen -- rules out route length as the predictor. No code changes; closes the floor-collapse investigation opened in Entry 034. Next: ML/GNN phase.

ENTRY 037 The First ML Baseline — August 16, 2026 — Status: COMPLETE

Entry 037 — Phase 1 Closes, Phase 2 Opens: Can a Learned Model Beat the Closed Form?

A Tabular Baseline First, Not a GNN -- 238 Circuits Isn't Enough Data for One Yet

1. Why a Tabular Model, Not a GNN, First

The original plan (Entry 033's close) was a GNN over the chip's connectivity graph. 238 labeled circuits is not enough to train a graph neural network without either overfitting or leaning so hard on regularization that any comparison to v4.1 is meaningless -- GNNs typically want thousands of labeled graphs, not hundreds. This entry instead trains gradient-boosted trees and a random forest on the same scalar features v4.1's formulas already use (hop distance, real edge/gate counts from the actual transpiled route, worst edge error and worst T1 along it, plus v4.1's own prediction as an input feature), evaluated with 5-fold cross-validation. If a model can't beat v4.1 with today's dataset, a bigger architecture won't either -- this is the honest floor to clear before scaling up.

2. Result: Roughly Tied Overall, Modest Gains Where It Matters

model	CV MAE (all 238)	CV R^2	MAE on the 12 floor-collapse cases
v4.1 (closed form)	6.29 pts	0.605	20.85 pts
Gradient boosting	6.48 pts	0.594	16.21 pts
Random forest	6.39 pts	0.617	18.81 pts

Across the whole dataset, both learned models land within noise of v4.1's own accuracy -- not a surprise, since v4.1's prediction is itself one of the input features and dominates the model's learned importance (87.3%). On the 12 floor-collapse cases specifically -- exactly where Entry 036 showed the closed form has a structural ceiling -- gradient boosting cuts the error from 20.85 to 16.21 points, about 22% relative. Real, but far from solved: the model has seen roughly 2-3 floor-collapse examples per cross-validation fold, nowhere near enough to learn the pattern Entry 036 couldn't even isolate to a single channel.

3. What the Model Actually Learned

Feature importances confirm the honest read: v4_1_prediction (87.3%) dominates, with n_2q_gates_total, worst_edge_error, worst_t1_us, n_real_edges_used, and bfs_hop_distance splitting most of the remaining 12.7% roughly evenly. The model is best described as learning a small correction on top of v4.1 rather than an independent prediction -- which chip and circuit kind (Bell vs GHZ) contributed almost nothing (under 1% combined), consistent with Entry 035's finding that the floor-collapse pattern isn't tied to chip identity in any way these coarse features capture.

Entry 037 — Conclusion

The first ML baseline does what a first baseline should: it roughly matches the physics model everywhere the physics model already works, and shows a real but modest edge exactly where the physics model is known to structurally fail. That is evidence the learning approach is pointed at the right problem, not yet evidence it has solved it. 238 circuits, 12 of them floor-collapse cases, is too thin a sample for a tree model -- let alone a GNN -- to do much better than this.

4. Next Step

Grow the batch dataset (Entries 034/035's scripts are already resumable and checkpointed) toward the low thousands of circuits, specifically oversampling short routes on Sherbrooke where floor-collapse concentrates, then revisit both the tabular baseline and a proper GNN over the chip connectivity graph with enough floor-collapse examples per fold to actually learn the pattern Entry 036 could isolate but not explain.

Research Log — Index Addendum (Entry 037)

Entry	Title	Date	Status
Entry 037	The First ML Baseline (gradient boosting vs. v4.1, 238 circuits)	Aug 16, 2026	Complete

Script: entry037_ml_baseline.py. Data: quantumbridge_data/entry037_ml_baseline.json. Opens Phase 2 (ML) of the project. Ties v4.1 overall, ~22% relative MAE reduction on floor-collapse cases. Next: grow the dataset before scaling to a GNN.

ENTRY 038 Growing the Dataset, Sharpening the Estimate — August 16, 2026 — Status: COMPLETE

Entry 038 — 238 Circuits to 558, and the Sherbrooke/Kyiv Ratio Comes Back Down to Earth

Oversampling Short Routes Triples the Floor-Collapse Sample and Corrects an Overconfident Early Estimate

1. Why Short Routes Specifically

Entry 036 showed floor-collapse doesn't need a long route -- the worst channel-independence gap measured in the whole project came from a 3-edge route. Growing the dataset efficiently therefore means concentrating new circuits in the 1-15 hop range, which is also the cheapest region to simulate (no long SWAP chains). entry038_grow_shortroute.py reuses Entry 034's exact methodology with a third, independent seed, restricted to four short/mid bins, generating 320 new Bell circuits (160 per chip) on top of Entries 034/035's 238.

2. The Dataset, and the Floor-Collapse Rate, Both Grew

	before (Entries 034/035)	after (+ Entry 038)
total circuits	238	558
floor-collapse cases	12	37
Kyiv short-route rate	3.1%	6.1%
Sherbrooke short-route rate	14.5%	12.5%
Sherbrooke / Kyiv ratio	4.7x	2.05x

The direction of Entry 035's finding holds -- Sherbrooke still collapses on short routes more than twice as often as Kyiv -- but the 4.7x ratio reported on 134 short-route samples was itself noisy. With 396 short-route samples the estimate settles to roughly 2x. This is worth stating plainly: the early number wasn't wrong in direction, but a batch of ~130 samples was too small to trust its exact magnitude, which is exactly the kind of correction a bigger dataset is supposed to produce.

3. Re-Running Entry 037's ML Baseline on the Larger Sample

model	MAE, all circuits	MAE, floor-collapse only	n floor-collapse
v4.1 (closed form)	4.60 pts	18.20 pts	37
Gradient boosting (CV)	4.57 pts	14.79 pts	37
Random forest (CV)	4.36 pts	15.34 pts	37

With 37 floor-collapse examples instead of 12, every cross-validation fold now contains floor-collapse cases (5 of 5, versus an uneven 2-3 before) -- a materially more trustworthy estimate. The relative gain holds: gradient boosting cuts floor-collapse MAE by about 19%, random forest by about 16%, consistent with Entry 037's ~22% finding on the smaller sample. Overall MAE for every model, v4.1 included, dropped as well (6.3 to ~4.5 points) -- this batch's short-route bias makes the whole dataset easier on average, not because anything got fixed.

Entry 038 — Conclusion

Growing the dataset did what it was supposed to: it didn't just add volume, it corrected an overconfident early ratio (4.7x to 2.05x) and confirmed the ML baseline's floor-collapse advantage is real and consistent, not an artifact of a 12-example sample. The floor-collapse phenomenon itself remains exactly as unexplained as Entry 036 left it -- this entry is about data quality and confidence, not a new mechanism.

4. Next Step

Continue growing toward the low thousands using the same resumable scripts (Entries 034, 035, 038 all checkpoint to JSON and can resume in chunks), then retrain and re-evaluate the ML baseline again -- watching whether the floor-collapse MAE gap keeps narrowing or plateaus, which would itself be informative about whether a tabular model

can ever fully close it or whether it genuinely needs correlated-noise modeling as Entry 036 suggested.

Research Log — Index Addendum (Entry 038)

Entry	Title	Date	Status
Entry 038	Growing the Dataset to 558 Circuits, Refining the Floor-Collapse Rate	Aug 16, 2026	Complete

Script: entry038_grow_shortroute.py. Data: quantumbridge_data/entry038_shortroute_dataset.json (320 new circuits), quantumbridge_data/entry038_combined_dataset.json (558 total, merged), quantumbridge_data/entry038_growth_summary.json. Sherbrooke/Kyiv floor-collapse ratio revised from 4.7x to a more reliable 2.05x; ML baseline's floor-collapse advantage confirmed with full 5-fold coverage.

ENTRY 039 790 Circuits: the Estimates Are Converging — August 16, 2026 — Status: COMPLETE

Entry 039 — A Fourth Independent Batch, and the Numbers Stop Moving

Balanced Across All Six Hop-Distance Bins Again, Restoring the Long-Route Coverage Entry 038 Traded Away

1. Why Balance This Time

Entry 038 deliberately skewed toward short/mid routes to grow the floor-collapse sample cheaply, which worked, but left the dataset thin on long routes again. entry039_grow_balanced.py returns to all six hop-distance bins from Entries 034/035 (with a fourth independent seed) plus a GHZ batch, adding 232 more circuits: 232 new (117 Kyiv, 115 Sherbrooke), on top of the 558 from Entries 034/035/038.

2. The Floor-Collapse Ratio Holds Steady

	after Entry 038 (n=396 short-route)	after Entry 039 (n=535 short-route)
total circuits	558	790
floor-collapse cases	37	47
Kyiv short-route rate	6.1%	6.0%
Sherbrooke short-route rate	12.5%	11.6%
Sherbrooke / Kyiv ratio	2.05x	1.93x

Two independent growth rounds in a row now land within a few tenths of a point of each other -- 2.05x, then 1.93x. That's the signature of an estimate that has actually converged rather than one still drifting with more data, in contrast to the 4.7x-to-2.05x jump between the original small sample and the first growth round. Sherbrooke floor-collapsing on short routes roughly twice as often as Kyiv looks like a stable, real property of the two chips.

3. The ML Baseline's Advantage Also Holds

	after Entry 038 (37 fc cases)	after Entry 039 (47 fc cases)
v4.1 floor-collapse MAE	18.20 pts	17.59 pts
Gradient boosting floor-collapse MAE	14.79 pts (-19%)	14.07 pts (-20%)
Random forest floor-collapse MAE	15.34 pts (-16%)	15.70 pts (-11%)

Gradient boosting's relative advantage over v4.1 on floor-collapse cases holds essentially flat (-19% to -20%) across two growth rounds. Random forest's edge narrowed slightly. Neither model is closing the gap further as data grows within this range -- consistent with Entry 036's read that the missing ingredient is a correlated-noise feature these models don't have access to, not simply more of the same kind of data.

Entry 039 — Conclusion

Two consecutive independent growth rounds (558 to 790 circuits) now agree closely on both headline numbers: Sherbrooke's short-route floor-collapse rate is roughly 2x Kyiv's, and a tabular ML model's edge over v4.1 on those cases plateaus around 15-20% relative MAE reduction. That plateau is itself informative -- it suggests the current feature set has been mostly saturated, and further gains will need either a genuinely new feature (something that captures cross-channel correlation directly) or the GNN architecture originally planned, once there is enough data to support one.

4. Where This Leaves the Project

790 circuits is a real, useful dataset -- roughly 79x the original 5-circuit reference sets this project started validating against in Entry 022 -- but still short of what a GNN needs, and the plateau in section 3 suggests raw volume alone won't move the floor-collapse number much further without a new feature or architecture. This is a reasonable point to pause pure data growth and let the next entry decide between two paths: keep scaling the existing feature set toward

a GNN, or first engineer a feature that captures the specific correlated-noise signature Entry 036 found.

Research Log — Index Addendum (Entry 039)

Entry	Title	Date	Status
Entry 039	Fourth Growth Batch (790 circuits total); Estimates Converge	Aug 16, 2026	Complete

Script: entry039_grow_balanced.py. Data: quantumbridge_data/entry039_balanced_dataset.json (232 new), quantumbridge_data/entry039_combined_dataset.json (790 total, merged), quantumbridge_data/entry039_growth_summary.json. Floor-collapse ratio and ML advantage both stabilized across two growth rounds -- a plateau, not further movement.

ENTRY 040 1,376 Circuits: the Plateau Was Premature — August 16, 2026 — Status: COMPLETE

Entry 040 — A Fifth and Sixth Growth Batch Reopen the Question Entry 039 Thought Was Closed

790 to 1,376 Circuits, and the ML Model Improved Again -- More Data Was Still the Right Lever

1. Method

Two more independent batches: entry040_grow_short2.py (seed=777, short/mid bins only, 480 new Bell circuits, 240 per chip) and entry040b_grow_far.py (seed=888, far bins + GHZ, 106 more circuits) -- 586 new circuits total, taking the combined dataset from 790 to 1,376. One checkpoint file was corrupted mid-write when a run was interrupted (a truncated JSON string from a kill mid-save); it was recovered by trimming back to the last complete record boundary, losing only the single in-flight circuit rather than the run.

2. The Floor-Collapse Ratio Keeps Narrowing

	790 circuits (Entry 039)	1,376 circuits (Entry 040)
floor-collapse cases	47	70
Kyiv short-route rate	6.0%	5.7%
Sherbrooke short-route rate	11.6%	9.3%
Sherbrooke / Kyiv ratio	1.93x	1.63x

The ratio has now moved 4.7x → 2.05x → 1.93x → 1.63x across four growth rounds -- still trending down, not yet flat. Sherbrooke still collapses more often than Kyiv, but each larger sample makes the gap look smaller than the last one suggested. This hasn't converged as cleanly as Entry 039 read it.

3. Correcting Entry 039: the ML Plateau Was Premature

	790 circuits (Entry 039)	1,376 circuits (Entry 040)
Gradient boosting CV R^2	0.725	0.785
Gradient boosting CV MAE (all)	4.52 pts	3.92 pts
v4.1 floor-collapse MAE	17.59 pts	17.01 pts
Gradient boosting floor-collapse MAE	14.07 pts	13.78 pts

Entry 039 read two similar-looking growth rounds as a plateau and concluded more volume wouldn't help further. This round contradicts that: CV R^2 rose meaningfully (0.725 to 0.785) and overall MAE improved by over half a point -- real gains, not noise. The lesson isn't that Entry 039 reasoned badly, it's that two data points looking flat isn't enough to call a plateau, especially this early. That's a useful correction to carry into how future entries interpret small numbers of growth rounds.

Entry 040 — Conclusion

More data is still the highest-leverage thing this project can do before touching model architecture. Both headline numbers moved in the same direction as every prior round -- ratio narrowing, ML advantage strengthening -- and the previous entry's read that they'd stabilized turned out to be one round too early. 1,376 circuits is real progress toward the low thousands a GNN would need, and the trend curve, not just the current values, is now informative: nothing here says growth has stopped paying off yet.

4. Next Step

Continue growing the dataset using the same resumable, checkpointed approach (Entries 034, 035, 038, 039, 040 all use the same methodology and can run in further independent batches) until gains genuinely flatten across two OR MORE consecutive rounds -- not one -- before committing to that as the point to stop and move to a GNN architecture.

Research Log — Index Addendum (Entry 040)

Entry	Title	Date	Status
Entry 040	Fifth/Sixth Growth Batches (1,376 circuits); Plateau Retracted	Aug 16, 2026	Complete

Scripts: entry040_grow_short2.py, entry040b_grow_far.py. Data: quantumbridge_data/entry040_short_dataset.json, quantumbridge_data/entry040_far_dataset.json, quantumbridge_data/entry040_combined_dataset.json (1,376 total), quantumbridge_data/entry040_growth_summary.json. Retracts Entry 039's plateau conclusion -- growth is still improving both the floor-collapse ratio estimate and the ML baseline.

ENTRY 041 The First GNN — August 16, 2026 — Status: COMPLETE

Entry 041 — A Graph Neural Network Cuts the Floor-Collapse Error by 63%

Real Route Structure, Not Hand-Picked Summaries, Is What the Tabular Model Was Missing

1. Architecture

Each circuit's real physical route (Entry 031/032's route-then-entangle structure) is represented as a small graph: nodes are the physical qubits the transpiler actually touched (2-37 per circuit), node features are that qubit's own T1/T2/readout; edges are the real 2-qubit gates issued (SWAPs and the final entangling gate), edge features are that edge's gate error and duration. Two rounds of message passing (16-dim hidden state, mean aggregation) let information several hops away reach a node's embedding, then a masked mean-pool over nodes feeds a small MLP alongside chip/kind/hop-distance and v4.1's own prediction (included as an input feature, same as Entry 037's tabular baseline, so the comparison is fair). Built in JAX + Optax (no PyTorch available in this environment); graphs are padded to 40 nodes/edges with masks so the whole batch vmaps. entry041_build_graphs.py re-derives the exact graph for all 1,376 Entry 034-040 circuits from their stored (chip, qubit pairs) via route_with_explicit_swaps -- no new simulation needed, the labels already exist.

2. Result: A Real, Large Improvement

model	CV MAE (all 1,376)	CV R^2	MAE, floor-collapse only
v4.1 (closed form)	4.50 pts	n/a	17.01 pts
Gradient boosting (Entry 037/040)	3.92 pts	0.785	13.78 pts
GNN (this entry)	2.64 pts	0.868	6.22 pts

The GNN beats the tabular model everywhere: 33% lower overall MAE (3.92 to 2.64), and on the 70 floor-collapse cases specifically, a 63% reduction from v4.1's 17.01 points down to 6.22 -- more than triple the tabular model's ~20% relative gain. Spot-checking individual floor-collapse predictions confirms this isn't an aggregate artifact: on 13 of 15 sampled cases the GNN lands within 5 points of the true ~50% floor, while v4.1 predicts 80-91% on several of the same circuits.

3. Why the Graph Structure Helps

Entry 036 showed floor-collapse comes from correlated noise across a route that no per-channel closed form captures, and that route length alone doesn't predict it -- a 3-edge route produced the single largest independence-breakdown gap measured in the project. The tabular model's features (worst edge error, worst T1, hop count) are scalar summaries that throw away exactly the arrangement information -- WHICH qubits sit next to which -- that message passing preserves. The GNN's large jump specifically on floor-collapse cases, rather than a uniform improvement everywhere, is consistent with it picking up on a structural signature in the route graph that no single scalar feature could encode.

Entry 041 — Conclusion

The GNN was worth building now, at 1,376 circuits, rather than waiting for the low thousands originally planned: it already beats both the closed form and the tabular baseline by a wide margin, especially on exactly the failure mode (floor-collapse) neither could explain. That answers the diagnostic question this entry was built to answer -- the project was NOT dataset-starved for a first useful GNN. The floor-collapse gap the closed-form model treated as a hard ceiling (Entry 036) is not actually a hard ceiling for a model that sees the real route graph.

4. Honest Caveats

This is a first prototype: 16-dim hidden states, 2 message-passing rounds, 180 epochs, no hyperparameter search, and v4.1's own prediction is still an input feature rather than the GNN reasoning entirely from scratch. Five-fold CV on 1,376 examples with 70 floor-collapse cases (roughly 14 per fold) is a real but still modest sample for the exact subset that matters most -- the 6.22-point floor-collapse MAE should be read as promising, not final. The natural next check is

whether it holds up as the dataset continues to grow.

5. Next Step

Grow the dataset further with the GNN now in place as the metric to track (not just the tabular baseline), watching specifically whether floor-collapse MAE keeps falling -- and consider a small ablation removing v4.1's prediction as an input feature, to see how much of the GNN's edge is genuinely new structural learning versus a residual correction on top of the closed form.

Research Log — Index Addendum (Entry 041)

Entry	Title	Date	Status
Entry 041	The First GNN (JAX/Optax, message passing over real routes)	Aug 16, 2026	Complete

Scripts: entry041_build_graphs.py, entry041_train_gnn.py. Data: quantumbridge_data/entry041_graph_dataset.json (1,376 graphs), quantumbridge_data/entry041_gnn_folds.json, quantumbridge_data/entry041_gnn_results.json. GNN beats v4.1 (63% lower floor-collapse MAE) and the tabular baseline (33% lower overall MAE) -- the biggest single-entry improvement since Entry 032.

ENTRY 042 2,198 Circuits, the GNN Keeps Improving — August 20, 2026 — Status: COMPLETE

Entry 042 — More Data Helps the GNN Too, and a Parallelization Detour That Wasn't Worth It

1,376 to 2,198 Circuits; GNN Floor-Collapse MAE Falls Again, 6.22 to 5.31 Points

1. Growing the Dataset, and a Parallelization Attempt

Tried parallelizing circuit generation across the sandbox's 4 CPU cores via multiprocessing.Pool, keeping identical precision (4 seeds x 4096 shots, same methodology as every prior entry). It worked initially but gave only a 1.3-1.7x speedup, not 4x -- most of the per-circuit cost turned out to be Python/Qiskit overhead (circuit construction, transpile scheduling) rather than raw linear algebra, which doesn't parallelize as cleanly. It then began reliably hanging after several hundred circuits for reasons that didn't reproduce in isolated tests -- reverted to the proven single-process approach from Entries 034-040. The sandbox's own throughput also degraded sharply mid-session (from roughly 0.4 to 0.06 circuits/second) for reasons outside the model or scripts, most likely shared infrastructure load. Net result: 822 new circuits added this entry, bringing the combined dataset to 2,198 -- real progress, just slower than hoped.

2. Retraining the GNN

	1,376 circuits (Entry 041)	2,198 circuits (Entry 042)
floor-collapse cases	70	112
GNN CV R^2	0.868	0.881
GNN CV MAE (all)	2.64 pts	2.04 pts
GNN floor-collapse MAE	6.22 pts	5.31 pts
v4.1 floor-collapse MAE	17.01 pts	14.30 pts

The GNN improved on every metric with more data, same as the tabular model did in Entries 038-040 -- confirming the growth strategy pays off for the graph architecture too, not just the simpler baseline. The GNN's margin over v4.1 on floor-collapse cases remains enormous (5.31 vs 14.30 points, a 63% relative reduction, essentially unchanged from Entry 041's finding).

Entry 042 — Conclusion

The GNN is confirmed as the best model built in this project across every dataset size tested so far, and it is not yet showing diminishing returns from more data the way the closed-form model structurally can't improve at all. The path to 5,000 and eventually 10,000 circuits remains the right one; it will just take more sessions than originally hoped, since raw circuit-generation throughput -- not modeling -- is now the actual bottleneck.

3. Next Step

Continue growth toward 5,000 in future sessions using the proven sequential approach (entry042_grow_sequential.py), retraining the GNN periodically to keep tracking whether floor-collapse MAE keeps falling or starts to plateau for real this time. In parallel, build an interactive visualization of the trained GNN's message-passing process on an example circuit, showing how information flows across the real route graph toward a prediction.

Research Log — Index Addendum (Entry 042)

Entry	Title	Date	Status
Entry 042	Dataset to 2,198 Circuits; GNN Retrained and Improved	Aug 20, 2026	Complete

Scripts: entry042_grow_parallel.py (partial, superseded), entry042_grow_sequential.py (active), entry042_train_gnn.py. Data: quantumbridge_data/entry042_parallel_dataset.json, quantumbridge_data/entry042_combined_dataset.json (2,198 total), quantumbridge_data/entry042_gnn_results.json, quantumbridge_data/entry042_growth_summary.json.

ENTRY 043 A Small Increment Is Too Noisy to Read — August 20, 2026 — Status: COMPLETE

Entry 043 — Checking Whether the GNN Has Plateaued, Before Grinding Toward 5,000

2,198 to 2,288 Circuits (+4%) Moved the Numbers the Wrong Way -- but That's Not the Same as a Real Plateau

1. Why This Check, Now

Growing the dataset toward 5,000 at full precision (Entry 042's methodology, no shot-count shortcuts) turned out to be much slower than expected -- roughly 10-20 circuits per batch round even after a free optimization (combining the 4 separate 4096-shot Aer calls into one 16,384-shot call, ~18% faster, statistically identical). Rather than keep grinding for the dozens of hours that would take, this entry asks the cheaper question first: is the GNN's improvement curve from Entries 041/042 still climbing, or did it already start to flatten with the 2,198 circuits already in hand.

2. The Result Is Genuinely Ambiguous

dataset size	GNN CV MAE	GNN CV R^2	floor-collapse MAE	floor-collapse n
1,376 (Entry 041)	2.64 pts	0.868	6.22 pts	70
2,198 (Entry 042)	2.04 pts	0.881	5.31 pts	112
2,288 (this entry)	2.17 pts	0.869	6.07 pts	117

The 1,376-to-2,198 jump (+60% more data) showed a clear, unambiguous improvement across every metric. The 2,198-to-2,288 jump (+4% more data) moved every metric in the WRONG direction -- but a 4% increment is far too small to distinguish a real plateau from ordinary training noise: each fold uses a different random 80/20 split and a freshly-initialized network, and with only ~23 floor-collapse cases per test fold, a handful of hard examples landing in one fold versus another can swing the reported MAE by more than this entire increment moved it.

Entry 043 — Conclusion

This entry does not answer whether the GNN has plateaued -- it answers that the question can't be answered from an increment this small, honestly reported rather than over-interpreted either direction. What it DOES confirm: the GNN remains dramatically ahead of v4.1 at every checkpoint (floor-collapse MAE 6.07 vs 14.67 points here, a 59% reduction, consistent with Entries 041 and 042), and R^2 has stayed in the same healthy 0.87-0.88 band across three retrainings. 2,288 circuits is a solid, validated working dataset -- not proven to be enough for a truly bigger GNN, but not shown to be insufficient either.

3. Next Step

Continue growing the dataset opportunistically in future sessions using the proven sequential, full-precision approach -- but without treating each small increment as a verdict on the model's ceiling. A real read on whether more of this kind of data still helps needs either a much bigger jump (the way 1,376 to 2,198 was) or averaging several training seeds per checkpoint to separate signal from noise, which Entry 043 did not have the compute budget to do this session.

Research Log — Index Addendum (Entry 043)

Entry	Title	Date	Status
Entry 043	Checking the GNN's Improvement Curve at 2,288 Circuits	Aug 20, 2026	Complete

Scripts: entry043_train_gnn.py. Data: quantumbridge_data/entry043_combined_dataset.json (2,288), quantumbridge_data/entry043_gnn_folds.json, quantumbridge_data/entry043_gnn_results.json, quantumbridge_data/entry043_growth_summary.json. Also includes a free, precision-preserving speedup to the Aer ground-truth pipeline in entry042_grow_parallel.py (one 16,384-shot call instead of four 4,096-shot calls).

ENTRY 044 Feature Engineering Beats Volume — August 20, 2026 — Status: COMPLETE

Entry 044 — Three New Features, Same 2,288 Circuits: Floor-Collapse MAE Falls 78%

Handing the GNN Entries 032 and 034/036's Own Findings as Explicit Inputs Outperforms Every Growth Round Combined

1. What Changed

Same 2,288 circuits as Entry 043 -- only the per-node and per-edge features changed. Added three things the GNN previously had to infer purely from graph position, if it could infer them at all: (1) a control/target role flag per node, directly encoding Entry 032's proof that the control qubit's routing survival has zero effect on outcome while the target's is everything; (2) T1/T2 ratio per node, making Entry 034/036's flagged anomaly (qubit 5's unusual T1/T2 split) a single explicit number instead of something the network would have to learn to compute by dividing two raw inputs; (3) normalized route position and a final-gate flag per edge, so the model can distinguish an early swap from the entangling gate itself. Node features went from 3 to 6, edge features from 2 to 4.

2. Result: the Biggest Jump in This Project's ML Phase

	old features (Entry 043)	new features (this entry)
GNN CV MAE (all 2,288)	2.17 pts	1.05 pts
GNN CV R^2	0.869	0.976
GNN floor-collapse MAE	6.07 pts	1.34 pts
v4.1 floor-collapse MAE (unchanged)	14.67 pts	14.67 pts

Floor-collapse MAE fell from 6.07 to 1.34 points -- a 78% relative reduction, and a 91% reduction against v4.1. For comparison, growing the dataset from 1,376 to 2,198 circuits (+60% more data, Entry 042) cut floor-collapse MAE by only 15% (6.22 to 5.31). Three targeted features on the SAME data did more than five entire growth rounds combined. 1.34 points is close to the intrinsic sampling noise floor of the Aer ground-truth labels themselves (roughly 0.4-1 point at 16,384 shots per circuit) -- there may not be much further to gain on this exact task without label-level precision improvements.

Entry 044 — Conclusion

This confirms the call to prioritize features over volume. The GNN was never short on raw data relative to what THESE features needed -- it was short on being told what this project already proved. Handing a model your own findings as explicit inputs, rather than hoping message passing rediscovers them from scratch given enough examples, was the higher-leverage move by a wide margin. Growing the dataset further remains worthwhile as a background task, but this result suggests the next real gains are more likely to come from more features than from more circuits.

3. Next Step

Two directions worth checking: an ablation removing each new feature one at a time to see which of the three is actually carrying the improvement (the control/target flag is the strongest candidate, since it encodes a proven causal asymmetry rather than a correlational pattern); and a visualization of the trained network itself -- its layer structure and parameter count at the current architecture and dataset scale -- as requested.

Research Log — Index Addendum (Entry 044)

Entry	Title	Date	Status
Entry 044	Feature Engineering (control/target role, T1/T2 ratio, route position)	Aug 20, 2026	Complete

Scripts: entry044_build_graphs.py, entry044_train_gnn.py. Data: quantumbridge_data/entry044_graph_dataset.json (2,288 graphs, 6 node / 4 edge features), quantumbridge_data/entry044_gnn_folds.json, quantumbridge_data/entry044_gnn_results.json, quantumbridge_data/entry044_growth_summary.json. Floor-collapse MAE 6.07 to 1.34 points on the same 2,288 circuits -- the largest single-entry gain in the ML phase.

ENTRY 045 Raising the Model's Real Capacity — August 20, 2026 — Status: COMPLETE

Entry 045 — The 40-Node Cap Was Sized to the Data, Not to the Chip -- So We Grew Both

Bigger Star-GHZ Circuits, a Wider Padding Window, and the First True-Scale Map of Every Qubit the Model Has Ever Seen

1. Why This Entry

Every GHZ circuit in the dataset through Entry 044 used exactly 3 logical qubits (1 control + 2 targets) -- even after SWAP-routing overhead, that structure topped out at 37 physical nodes, and the GNN's MAX_N=MAX_E=40 padding was sized to fit exactly that, not to any real ceiling on what the 127-qubit chips or the model could represent. This entry generates genuinely larger circuits (6-8 target star-GHZ, 1 control qubit entangled with up to 7 others at once), rebuilds the graph dataset, raises the model's padding to fit what these circuits actually need, and retrains to confirm accuracy holds up on real bigger structures -- not just a bigger constant for its own sake.

2. What Changed

	through Entry 044	this entry
Total circuits	2,288	2,317 (+29 large-GHZ)
Largest GHZ (logical qubits)	3	8
Largest real routed graph	37 nodes / 33 edges	42 nodes / 56 edges
Model padding (MAX_N / MAX_E)	40 / 40	48 / 64
Physical qubits ever touched (each chip)	127 / 127	127 / 127 (unchanged)

127/127 physical qubits on both Kyiv and Sherbrooke were already being touched as circuit endpoints before this entry -- the gap was never chip coverage, it was that no single circuit had ever asked the model to represent more than 37 qubits' worth of structure at once. The new 6-8-target star-GHZ circuits close that gap directly: an 8-target star on Kyiv produced a 154-gate, 42-node routed graph, the largest the model has ever trained on.

3. Capacity Bump Holds Up Under Real Load

metric	Entry 044 (n=2,288, cap 40/40)	Entry 045 (n=2,317, cap 48/64)
GNN CV MAE	1.05 pts	1.21 pts
GNN CV R^2	0.976	0.971
GNN floor-collapse MAE	1.34 pts	1.79 pts
GNN big-circuit MAE (>20 nodes)	n/a (not tracked)	3.68 pts
v4.1 big-circuit MAE (>20 nodes)	n/a	11.46 pts

The small across-the-board dip (1.05 to 1.21 points overall) is expected and healthy -- the new dataset includes circuits that are structurally harder (more qubits, more routing, more accumulated noise) than anything in Entry 044's training distribution, so a slightly higher average error reflects a harder test, not a worse model. The number that matters is the new big-circuit breakdown: on the 140 circuits touching more than 20 physical qubits, the GNN reaches 3.68-point MAE against v4.1's 11.46 -- a 68% reduction, confirming the capacity increase wasn't just cosmetic padding.

Entry 045 — Conclusion

The model's qubit-handling capacity was never actually limited by the 127-qubit chips -- it was limited by the tallest circuit shape the training data had ever produced. Deliberately generating bigger star-GHZ circuits, rebuilding the graph dataset, and widening the padding from 40/40 to 48/64 nodes/edges directly answers 'how many qubits can this

model handle': up to 48 physical qubits per circuit by construction, validated up to the 42-node circuits actually in hand, with the GNN staying dramatically ahead of v4.1 specifically on the larger circuits this entry added. This was prioritized over further raw dataset growth per this session's direction -- features and real capacity first, volume as background work.

4. Next Step

Keep pushing circuit size further (10-15 target stars, or multiple independent GHZ clusters per circuit) to stress-test whether accuracy degrades gracefully or hits a wall before MAX_N=48; an ablation isolating which of Entry 044's three features matters most remains open from the prior entry.

Research Log — Index Addendum (Entry 045)

Entry	Title	Date	Status
Entry 045	Raising the GNN's Real Node/Edge Capacity (40/40 to 48/64)	Aug 20, 2026	Complete

Scripts: entry045_grow_bigghz.py, entry045_build_graphs.py, entry045_train_gnn.py. Data: quantumbridge_data/entry045_bigghz_dataset.json (29 circuits, 6-8 target star-GHZ), quantumbridge_data/entry045_graph_dataset.json (2,317 graphs), quantumbridge_data/entry045_gnn_folds.json, quantumbridge_data/entry045_gnn_results.json. Big-circuit (>20 nodes) MAE 3.68 vs v4.1's 11.46 -- a 68% reduction on exactly the circuits this entry was built to stress-test.

ENTRY 047 Cross-Chip Generalization and Feature Ablation — August 20, 2026 — Status: COMPLETE

Entry 047 — Three Independent AI Reviews Converged on One Critique. We Tested It.

Cross-Chip Accuracy Collapses 8-10x Out of Distribution; Message Passing Is Confirmed to Be the Feature Doing the Real Work

1. Why This Entry

After publishing the public demo, three different AI models (independently prompted, not shown each other's answers) were asked to critique it. All three converged on the same core objection: the GNN is trained and evaluated against Qiskit Aer's simulated noise model, so a win over the v4.1 formula could just mean the GNN approximates the simulator better -- not that it captures anything about real hardware or generalizes beyond this exact dataset. Two of three also specifically asked whether the model's features (control/target role, T1/T2 ratio, route position) and its graph structure (message passing) are actually earning their keep, or whether a simpler model would do just as well. This entry runs both checks honestly, on data already in hand -- no new circuits were generated.

2. Cross-Chip Generalization: the Model Does Not Transfer

train → test	MAE	R^2	floor-collapse MAE	n floor-collapse
same-chip CV baseline (Entry 045)	1.21 pts	0.971	1.79 pts	117 (pooled)
Kyiv → Sherbrooke (cold)	12.87 pts	0.037	8.20 pts	50
Sherbrooke → Kyiv (cold)	10.31 pts	0.373	2.81 pts	67

Training on one chip and testing cold on the other collapses R^2 from 0.97 to 0.04-0.37 and multiplies MAE by 8-10x. This confirms the reviewers' concern directly: the model has not been shown to learn transferable route/calibration physics -- it learns chip-specific patterns that do not carry over even to a structurally similar 127-qubit device. Notably, floor-collapse MAE degrades far less than overall MAE in both directions (roughly comparable to or better than v4.1's 14.67 baseline even out of distribution), suggesting the model retains SOME of the structural signal behind the floor-collapse pattern specifically, even while its general calibration is off -- but this is not the same as validated cross-chip generalization.

3. Feature Ablation: Message Passing Is the Real Contributor

variant (single 80/20 split)	MAE	R^2	floor-collapse MAE
full features + message passing (baseline)	1.23 pts	0.977	1.09 pts
no control/target role flag	1.93 pts	0.933	3.18 pts
no T1/T2 ratio	1.17 pts	0.973	0.95 pts
no route-position / is-final flags	1.20 pts	0.974	1.02 pts
no message passing (features pooled directly, no graph propagation)	2.28 pts	0.920	8.43 pts

Removing the control/target role flag clearly hurts (floor-collapse MAE roughly triples, 1.09 to 3.18) -- this is real signal, consistent with Entry 032's proven causal finding that the flag encodes. Removing the T1/T2 ratio or the route-position features made no meaningful difference (both variants performed within noise of the full baseline, one even fractionally better) -- these two features from Entry 044 are not carrying their claimed weight and are candidates for removal or replacement. The single most important result: disabling message passing entirely (keeping every feature but removing the graph propagation itself) is catastrophic for floor-collapse cases specifically -- MAE rises 7.7x, from 1.09 to 8.43 points, while overall MAE only roughly doubles. This directly answers 'is the GNN doing anything a flat feature set couldn't' for the exact failure mode this project was built around: yes, decisively.

Entry 047 — Conclusion

The AI reviews were right to be skeptical, and the skepticism was actionable rather than just noise: the cross-chip test proves the model does not yet generalize beyond its training chip, which means claims of predicting "how real IBM chips fail" need to be walked back to "how this specific trained model reproduces Aer's noise model on the chip it was trained on" until cross-chip (and eventually cross-hardware) generalization is fixed. At the same time, the ablation results are a genuine positive finding: message passing over the real routed graph is doing real, measurable work on exactly the floor-collapse cases this whole project exists to catch, and the control/target role feature is validated as useful. The T1/T2 ratio and route-position features are not currently justified by the evidence and are the first things to simplify or replace.

4. Next Step

Highest priority: understand and fix the cross-chip generalization failure -- likely requires either training on both chips jointly with held-out qubit ranges (not just held-out circuits), or normalizing calibration features relative to each chip's own distribution rather than raw absolute values, so the model learns relative noise patterns instead of chip-specific absolute scales. Second priority: drop or replace the T1/T2 ratio and route-position features given this entry's evidence they don't help. Real-hardware validation (actual IBM QPU jobs, not just Aer) remains the most convincing but most expensive fix and is still not attempted.

Research Log — Index Addendum (Entry 047)

Entry	Title	Date	Status
Entry 047	Cross-Chip Generalization Test + Feature Ablation Study	Aug 20, 2026	Complete

Scripts: entry047_experiments.py. Data: quantumbridge_data/entry047_results.json. Motivated by independent critiques from three AI models (Gemini, ChatGPT, Grok) asked to review the public demo, all of which converged on the same simulator-circularity concern. Cross-chip R^2 drops from 0.97 to 0.04-0.37 out of distribution; disabling message passing raises floor-collapse MAE 7.7x (1.09 to 8.43 points), confirming graph structure -- not just the feature set -- is responsible for the project's core result.

ENTRY 048 Cross-Chip Generalization Fixed — August 20, 2026 — Status: COMPLETE

Entry 048 — Two Bugs, Not One Deep Flaw: Cross-Chip R² Recovers From 0.04-0.37 to 0.75-0.87

Removing a Chip-Identity Input the Network Never Saw at Test Time, Plus Per-Chip Feature Normalization, Fixes Most of Entry 047's Collapse

1. Diagnosis

Two concrete, fixable causes were identified for Entry 047's cross-chip collapse, not a fundamental failure of the approach. First: the global feature vector included a chip_kyiv/chip_sherbrooke one-hot flag. When trained on Kyiv only, that flag is CONSTANT ([1,0]) across every training example -- at test time on Sherbrooke it flips to a value ([0,1]) the network never saw during training, a distribution shift baked directly into the input. Second: node/edge calibration features (T1, T2, readout, gate error) were normalized using mean/std computed across both chips combined, so a model trained only on one chip's normalized range saw out-of-range values for the other chip at test time, before any physics was even involved.

2. Fix and Result

train → test	MAE (Entry 047)	MAE (fixed)	R² (047)	R² (fixed)
Kyiv → Sherbrooke (cold)	12.87 pts	4.40 pts	0.037	0.747
Sherbrooke → Kyiv (cold)	10.31 pts	3.20 pts	0.373	0.866

Dropping the chip one-hot and switching to per-chip-relative normalization (each chip's calibration features normalized against that chip's own distribution, known in advance at deployment time without needing labels) recovered most of the lost accuracy in both directions -- MAE fell roughly 3x, R² rose from near-zero/weak to 0.75-0.87. This is still meaningfully worse than the same-chip CV baseline (R²=0.971, MAE=1.21), so genuine cross-chip generalization is not yet fully solved -- but the gap shrank from catastrophic to a real, workable starting point. One caveat: floor-collapse MAE on the Kyiv-to-Sherbrooke direction got WORSE (8.20 to 12.57 points) even as overall MAE improved, suggesting the fix helped general calibration but may have traded away some of whatever weak floor-collapse signal Entry 047's version retained -- worth watching, not yet understood.

Entry 048 — Conclusion

Entry 047's cross-chip collapse was diagnosable and largely fixable with two targeted changes, not a sign the whole approach is unsound -- an important distinction for how this project should be described going forward. The honest current claim is: the model generalizes reasonably (R² 0.75-0.87) to an unseen chip of the same hardware family, up from essentially not at all, but still falls short of its same-chip accuracy and has an unexplained floor-collapse regression in one direction. This is real, evidence-backed progress directly responding to the AI reviews' central critique -- not yet a closed case.

3. Next Step

Investigate the floor-collapse regression on the Kyiv-to-Sherbrooke direction specifically before declaring victory. Consider joint training across both chips with a proper held-out split (rather than pure leave-one-chip-out) as a more realistic deployment scenario. Real-hardware validation remains the next major open item, independent of this fix.

Research Log — Index Addendum (Entry 048)

Entry	Title	Date	Status
Entry 048	Cross-Chip Generalization Fix (chip-identity leakage + per-chip normalization)	Aug 20, 2026	Complete

Scripts: entry048_generalization_fix.py. Data: quantumbridge_data/entry048_results.json. Cross-chip R² recovered from 0.04-0.37 (Entry 047) to 0.75-0.87 by removing chip-identity leakage from the input and normalizing calibration features per-chip instead of globally.

ENTRY 050 Diagnosing the Floor-Collapse Regression — August 21, 2026 — Status: COMPLETE

Entry 050 — On an Unseen Chip, the GNN Partly Reverts to Trusting the Formula It Was Built to Override

The Floor-Collapse Override Mechanism Is Itself Less Transferable Than the Model's General Accuracy

1. The Question

Entry 048's generalization fix (removing chip-identity leakage, per-chip feature normalization) improved overall Kyiv-to-Sherbrooke cross-chip accuracy substantially (R^2 0.037 to 0.747), but floor-collapse MAE specifically got WORSE in that direction (8.20 to 12.57 points) even as everything else improved. This entry isolates why by comparing the model's actual predictions, not just aggregate error, on floor-collapse cases it was trained on (Kyiv) versus cases it had never seen (Sherbrooke).

2. Diagnosis: the Override Behavior Doesn't Transfer

	Kyiv (trained, in-sample)	Sherbrooke (cold, unseen)
n floor-collapse cases	67	50
mean GNN prediction	0.497	0.615
mean real (Aer) result	0.498	0.498
correlation(GNN pred, v4.1 pred)	0.08	0.46

On Kyiv, the chip it trained on, the model has essentially learned to ignore the v4.1 formula's prediction entirely for floor-collapse cases (correlation 0.08, near-zero) and correctly outputs the true ~50% floor (mean prediction 0.497 against a true mean of 0.498 -- functionally perfect). On Sherbrooke, a chip it never trained on, that override behavior partially breaks down: predictions correlate moderately with v4.1 (0.46) and are biased upward (mean 0.615 vs true 0.498), meaning the model is partly falling back toward trusting the closed-form formula it was specifically built to catch failing. Individual cases show this directly -- several Sherbrooke floor-collapse circuits get predictions that land close to v4.1's (wrong) number rather than the true floor.

Entry 050 — Conclusion

This resolves Entry 048's open question with a specific, mechanistic answer rather than a vague one: the GNN's general fidelity estimation transfers reasonably well across chips (per Entry 048's fix), but its floor-collapse OVERRIDE mechanism -- arguably the single most valuable thing this project's GNN does, since it's the exact failure mode v4.1 cannot see -- is itself less transferable and partially collapses back toward trusting v4.1 on an unseen chip. This is a more precise and more useful finding than 'cross-chip generalization needs work': the general regression task generalizes; the specific override behavior this project cares about most does not, yet.

3. Next Step

Consider training with floor-collapse cases explicitly up-weighted or with a loss term that penalizes over-trusting v4.1's input feature specifically on ambiguous cases, so the override behavior is pushed to depend more on graph/calibration structure and less on the v4.1 input feature, which may be an easier shortcut the model leans on when out-of-distribution. Joint training across both chips (rather than strict leave-one-chip-out) remains untried and may also help, since the model would see floor-collapse examples from both chips during training.

Research Log — Index Addendum (Entry 050)

Entry	Title	Date	Status
Entry 050	Diagnosing the Kyiv-to-Sherbrooke Floor-Collapse Regression	Aug 21, 2026	Complete

Data: quantumbridge_data/entry050_floorcollapse_diagnosis.json. Real-hardware validation (Entry 049 scaffolding) is paused for now due to IBM Cloud account verification friction -- resuming simulation-based work on existing resources per project direction.

ENTRY 051 Floor-Collapse Up-Weighting — August 21, 2026 — Status: COMPLETE

Entry 051 — Up-Weighting Floor-Collapse Cases 5x in the Loss Improves Cross-Chip Robustness on the Failure Mode That Matters Most

A Direct, Successful Response to Entry 050's Diagnosis

1. The Fix

Entry 050 found that the GNN's floor-collapse override behavior -- correctly ignoring v4.1's prediction and outputting the true ~50% floor -- degrades on an unseen chip, likely because floor-collapse cases are only ~5% of training data and ordinary MAE loss gives the model little pressure to make that specific behavior robust rather than curve-fit to the training chip. This entry up-weights floor-collapse training examples 5x in the loss, keeping everything else (architecture, per-chip normalization, no-chip-identity fix from Entry 048) unchanged.

2. Result: Real Improvement, One Direction Fully, One Partially

train → test	MAE (Entry 048)	MAE (weighted)	fc MAE (048)	fc MAE (weighted)
Kyiv → Sherbrooke (cold)	4.40 pts	4.99 pts	12.57 pts	8.69 pts
Sherbrooke → Kyiv (cold)	3.20 pts	2.76 pts	4.47 pts	3.48 pts

Sherbrooke-to-Kyiv improved on every metric simultaneously (overall MAE 3.20 to 2.76, R^2 0.866 to 0.884, floor-collapse MAE 4.47 to 3.48) -- up-weighting cost nothing here. Kyiv-to-Sherbrooke shows the expected tradeoff: floor-collapse MAE improved substantially (12.57 to 8.69, a 31% reduction) at a small cost to overall accuracy (MAE 4.40 to 4.99, R^2 0.747 to 0.708). Given this project exists specifically to catch floor-collapse -- the exact failure mode v4.1 cannot see -- this is a trade worth making: a small overall accuracy cost for meaningfully more robust behavior on the cases that matter most.

Entry 051 — Conclusion

The up-weighting fix works and is adopted going forward. This closes the loop opened by Entry 047's AI-review-motivated skepticism, through Entry 048's generalization fix and Entry 050's precise diagnosis, to a targeted and validated fix here -- a complete, evidence-driven cycle from external critique to concrete improvement.

3. Next Step

Per updated project direction: push real physical qubit capacity from 48 to roughly 60-70 nodes via larger star-GHZ circuits, and grow the dataset toward 3,000 circuits (revised down from the earlier 5,000 target), then retrain and re-verify both the capacity increase and this floor-collapse fix hold up together at the new scale.

Research Log — Index Addendum (Entry 051)

Entry	Title	Date	Status
Entry 051	Floor-Collapse Up-Weighted Training (5x loss weight)	Aug 21, 2026	Complete

Scripts: entry051_floorcollapse_weighted.py. Data: quantumbridge_data/entry051_results.json. Floor-collapse MAE improved 22-31% in both cross-chip directions; Sherbrooke-to-Kyiv improved on every metric simultaneously.

ENTRIES 052-053 Capacity Push: 48 → 65 Physical Qubit Nodes — August 22, 2026 — Status: COMPLETE

Entries 052-053 — Larger Star-GHZ Circuits, Wider Model Padding, and a Combined Re-Verification of All Prior Fixes

Following Through on the Post-Review Roadmap: More Qubits, Same Rigor

1. Motivation

Per project direction after Entry 051: rather than adding new features, grow the model's physical-qubit capacity from 48 nodes (Entry 045's padding) toward 60-70, to better reflect circuits a real research user might actually want to route. Star-topology GHZ circuits at k=10, 12, 14 logical qubits (1 control + k-1 targets) were generated on both Kyiv and Sherbrooke, reaching a new real maximum of 65 physical nodes and 99 edges after routing (vs. 42/56 previously) -- within the 60-70 target range.

2. A Genuine Simulator Scaling Wall, Documented Rather Than Hidden

k=10 star circuits produced 227 real two-qubit gates after routing, and Aer's matrix_product_state simulator measured roughly 0.25s per shot on circuits this deep -- making the usual 4096-shot run take on the order of 17 minutes, well past a single sandbox command. Shots were reduced per circuit size (SHOTS_BY_SIZE = {10: 256, 12: 128, 14: 32}) to fit the time budget. This is a real, larger precision tradeoff than earlier entries and is documented in entry052_grow_bigger.py's docstring rather than applied silently -- these 12 large circuits carry more sampling noise than the rest of the dataset.

3. Rebuild and Retrain at the New Scale

entry053_build_graphs.py combined the original 2,288-circuit set, Entry 045's 29 bigger circuits, and Entry 052's 12 new ones into 2,329 total graphs. MAX_N/MAX_E were widened from 48/64 to 80/104 (headroom above the observed 65/99 max, following the same margin convention as Entry 045). The retrained model keeps every fix validated so far: per-chip normalization and no chip-identity leakage (Entry 048), and 5x floor-collapse loss up-weighting (Entry 051) -- so this run tests whether all three hold up together at the larger scale, not just the capacity increase in isolation.

4. Results: Capacity Increase Did Not Cost Accuracy

metric	same-chip (80/20 split)	v4.1 baseline
MAE	1.36 pts	3.67 pts
R²	0.975	—
floor-collapse MAE	1.57 pts	14.67 pts

train → test	MAE (Entry 051, 48-cap)	MAE (Entry 053, 65-cap)	fc MAE (051)	fc MAE (053)
Kyiv → Sherbrooke (cold)	4.99 pts	4.59 pts	8.69 pts	9.67 pts
Sherbrooke → Kyiv (cold)	2.76 pts	2.96 pts	3.48 pts	4.15 pts

Same-chip accuracy at the new 65-node capacity (MAE 1.36, R² 0.975) matches Entry 047's original ablation baseline at the smaller 42-node capacity (MAE 1.23, R² 0.977) closely -- capacity growth did not degrade in-distribution fit. Cross-chip transfer held up as well: Kyiv-to-Sherbrooke MAE actually improved slightly (4.99 to 4.59, R² 0.708 to 0.769) and Sherbrooke-to-Kyiv stayed essentially flat (2.76 to 2.96, R² 0.884 to 0.891). Floor-collapse MAE regressed modestly in both directions (8.69 to 9.67, 3.48 to 4.15) -- plausibly because floor-collapse cases are a fixed small count (117) against a now-larger and more varied circuit population, diluting their relative weight even under 5x up-weighting. Still well below v4.1's own 14.67-pt floor-collapse error.

Entries 052-053 — Conclusion

The three independently-developed fixes -- generalization (Entry 048), floor-collapse robustness (Entry 051), and now capacity (Entries 052-053) -- compose without materially undermining one another. The model now routes real circuits up to 65 physical qubits (vs. 48 previously) while keeping cross-chip R² in the 0.77-0.89 range and floor-collapse error

roughly 2-3x better than v4.1's own baseline in both directions.

5. Next Step

Per the user's roadmap: continue growing the general dataset toward 3,000 circuits (currently 932 in the separate bell-pair pool, not yet merged into this combined set), then fold that growth into a future capacity/generalization/floor-collapse re-check the same way this entry did. Real-hardware validation (Entry 049) remains paused per explicit prior direction and is not part of this phase of work.

Research Log — Index Addendum (Entries 052-053)

Entry	Title	Date	Status
Entry 052	Larger Star-GHZ Circuits (k=10,12,14, 60-70 node target)	Aug 22, 2026	Complete
Entry 053	Graph Rebuild + Combined Retrain at 65-Node/80-Padding Capacity	Aug 22, 2026	Complete

Scripts: entry052_grow_bigger.py, entry053_build_graphs.py, entry053_train_gnn.py, entry053_cross_chip.py. Data: quantumbridge_data/entry052_biggerghz_dataset.json, entry053_graph_dataset.json, entry053_gnn_results.json, entry053_cross_chip_results.json. Max circuit size grew from 42 to 65 physical nodes; model padding grew 48/64 to 80/104; all three prior fixes (generalization, floor-collapse weighting) confirmed compatible with the capacity increase.

ENTRY 054 Dataset Growth: 2,329 → 2,811 Circuits — August 23, 2026 — Status: COMPLETE

Entry 054 — Folding 482 New Bell-Pair Circuits into the Combined Dataset and Re-Verifying All Fixes Hold

Progress Toward the 3,000-Circuit Target, Settled at a Practical Milestone

1. Background

The general bell-pair growth pool (entry042_parallel_dataset.json) had been paused at 932 records since Entry 052. Growth was resumed using the parallelized generator (4 pinned worker processes, one thread each to avoid oversubscription) rather than the single-threaded fallback, which raised throughput roughly 4x (~0.3-0.4 circuits/s vs. the earlier ~0.07/s). After 14 rounds the pool reached 1,592 records. Deduplicating against everything already in the Entry 053 training set (2,329 circuits) left 482 genuinely new bell-pair circuits -- the rest overlapped with circuits already trained on from earlier growth rounds. These 482 were folded in, bringing the combined dataset to 2,811 circuits -- a deliberately settled milestone rather than the originally discussed 5,000/3,000 targets, made explicitly as a time/effort tradeoff rather than left ambiguous.

2. Retrain and Re-Verify: Same-Chip

metric	Entry 053 (n=2,329)	Entry 054 (n=2,811)
MAE	1.36 pts	1.37 pts
R^2	0.975	0.964
floor-collapse MAE	1.57 pts	0.59 pts
v4.1 MAE / fc MAE	3.67 / 14.67	3.79 / 16.11

Same-chip accuracy is essentially unchanged (MAE 1.36 to 1.37) with a modest R^2 dip (0.975 to 0.964, still strong) and a clear floor-collapse improvement (1.57 to 0.59 pts) -- the larger, more varied dataset gave the up-weighted floor-collapse loss more examples to learn from.

3. Retrain and Re-Verify: Cross-Chip

train → test	MAE (053)	MAE (054)	R^2 (053)	R^2 (054)	fc MAE (053)	fc MAE (054)
Kyiv → Sherbrooke	4.59	4.61	0.769	0.711	9.67	12.28
Sherbrooke → Kyiv	2.96	2.55	0.891	0.909	4.15	3.72

Sherbrooke-to-Kyiv improved across every metric (MAE 2.96 to 2.55, R^2 0.891 to 0.909, floor-collapse MAE 4.15 to 3.72). Kyiv-to-Sherbrooke held roughly flat on MAE/R^2 (4.59 to 4.61 / 0.769 to 0.711) but its floor-collapse error rose (9.67 to 12.28 pts) -- the same asymmetry seen in Entry 051/053, where Kyiv-to-Sherbrooke is consistently the harder direction for floor-collapse specifically. This is noted honestly rather than smoothed over: dataset growth alone did not fix that asymmetry, though it did not make it meaningfully worse either.

Entry 054 — Conclusion

All three prior fixes (generalization, floor-collapse up-weighting, 65-node capacity) remain intact at the larger 2,811-circuit scale. Same-chip and one cross-chip direction improved; the other held flat with a known, already-documented floor-collapse asymmetry. This closes the dataset-growth thread opened in Entries 051/052 at a deliberately chosen, practical stopping point rather than the original 3,000/5,000 targets.

4. Remaining Work

The Kyiv-to-Sherbrooke floor-collapse asymmetry (9.67-12.28 pts vs. Sherbrooke-to-Kyiv's 3.72-4.15 pts) remains open and is a candidate for a future targeted investigation, similar in spirit to Entry 050's diagnosis. Separately, uncertainty quantification (ensemble or MC-dropout confidence intervals) is scoped as the next major research-value addition, to directly answer external reviewers' "why not just use Aer" critique with a capability Aer's point-estimate output cannot provide.

Research Log — Index Addendum (Entry 054)

Entry	Title	Date	Status
Entry 054	Dataset Growth to 2,811 Circuits + Combined Re-Verification	Aug 23, 2026	Complete

Scripts: entry054_build_graphs.py, entry054_train_gnn.py, entry054_cross_chip.py. Data: quantumbridge_data/entry054_graph_dataset.json, entry054_gnn_results.json, entry054_cross_chip_results.json. 482 new deduplicated bell-pair circuits folded in (2,329 to 2,811 total); all three prior fixes confirmed to hold at the new scale.

ENTRY 055 Dataset Growth: 2,811 → 3,205 Circuits — August 25, 2026 — Status: COMPLETE

Entry 055 — Second Growth Round Clears the 3,000+ Milestone; Kyiv-to-Sherbrooke Floor-Collapse Trend Reverses

A Revised, Smaller Target Reached Efficiently After a Cost/Benefit Check-In

1. Background

Following Entry 054's settle point of 2,811 circuits, growth was resumed toward an originally discussed 3,500-circuit target. Partway through, throughput of genuinely new (non-duplicate) circuits was observed slowing as easy qubit-pair bins filled up (~40 new/round down to ~20/round, with unique-yield dropping toward 60-70%). Given this, the target was revised down to a more practical 3,200, reached with the pool at 2,152 records yielding 394 new deduplicated circuits -- bringing the combined training set to 3,205, just past the milestone. Growth is paused here; the pool can be resumed later toward the original 3,500/5,000 targets if warranted.

2. Retrain and Re-Verify: Same-Chip

metric	Entry 054 (n=2,811)	Entry 055 (n=3,205)
MAE	1.37 pts	1.16 pts
R²	0.964	0.970
floor-collapse MAE	0.59 pts	0.79 pts
v4.1 MAE / fc MAE	3.79 / 16.11	3.88 / 16.27

Same-chip accuracy improved (MAE 1.37 to 1.16, R² 0.964 to 0.970) with more training data, as expected. Floor-collapse MAE ticked up slightly (0.59 to 0.79 pts) but remains far below v4.1's own 16.27-pt floor-collapse error.

3. Retrain and Re-Verify: Cross-Chip -- the Key Result

train → test	MAE (054)	MAE (055)	R² (054)	R² (055)	fc MAE (054)	fc MAE (055)
Kyiv → Sherbrooke	4.61	4.08	0.711	0.786	12.28	8.52
Sherbrooke → Kyiv	2.55	2.87	0.909	0.894	3.72	3.72

Kyiv-to-Sherbrooke -- the direction that had gotten steadily worse across Entries 051, 053, and 054 (fc MAE 8.69 to 9.67 to 12.28 pts) -- reversed with this larger, more diverse dataset: MAE improved (4.61 to 4.08), R² improved (0.711 to 0.786), and floor-collapse MAE dropped substantially (12.28 to 8.52). This is consistent with the structural diagnosis made before this growth round: Sherbrooke has a meaningfully higher floor-collapse rate than Kyiv (6-7% vs. 4-5%), so more Kyiv training data with proportionally more floor-collapse examples gives the up-weighted loss more signal to work with in the harder direction. Sherbrooke-to-Kyiv held essentially flat (MAE 2.55 to 2.87, floor-collapse MAE unchanged at 3.72) -- it was already the stronger direction and simply maintained that strength.

Entry 055 — Conclusion

The multi-entry Kyiv-to-Sherbrooke floor-collapse regression is reversed, not just patched -- more training data on the underrepresented direction addressed the structural cause identified in this entry's own diagnosis, rather than requiring an architectural change. This validates the diagnosis and closes Task 63 with a confirmed, working fix path (more balanced/larger data) rather than just a hypothesis.

4. Next Steps

Per updated project direction: dataset growth is paused here at 3,205 circuits (target 3,500/5,000 deferred). Focus shifts to (a) generating more large circuits (60+ physical qubits, extending Entry 052's approach) to grow that underrepresented capacity tier, and (b) scoping uncertainty quantification (ensemble or MC-dropout confidence

intervals) as the next research-value feature, to be worked on alongside large-circuit generation.

Research Log — Index Addendum (Entry 055)

Entry	Title	Date	Status
Entry 055	Dataset Growth to 3,205 Circuits + Floor-Collapse Trend Reversal	Aug 25, 2026	Complete

Scripts: entry055_build_graphs.py, entry055_train_gnn.py, entry055_cross_chip.py. Data: quantumbridge_data/entry055_graph_dataset.json, entry055_gnn_results.json, entry055_cross_chip_results.json. 394 new deduplicated circuits folded in (2,811 to 3,205); Kyiv-to-Sherbrooke floor-collapse MAE improved 12.28 to 8.52 pts, reversing the trend flagged earlier in this same entry.

ENTRY 056 Large-Circuit Tier Doubled (12 → 24) — August 25, 2026 — Status: COMPLETE

Entry 056 — Doubling the 60+ Physical Qubit Tier and Widening Capacity to 90/130

Deepening the Underrepresented Large-Circuit Slice, Per Updated Project Direction

1. Motivation

Following Entry 055's dataset-growth pause, focus shifted to the large-circuit tier: only 12 star-GHZ circuits at k=10/12/14 existed (from Entry 052), a thin slice relative to the 3,205-circuit bell-pair-dominated set. entry056_grow_bigger.py topped up PER_SIZE from 2 to 4 per size per chip using the same resumable have_per_size logic as Entry 052, adding 12 new circuits (doubling the tier to 24) and pushing the observed maximum from 65 to 67 physical nodes.

2. A New Padding Ceiling Was Hit and Fixed Before Training

The new circuits pushed max edges to 106, exceeding the Entry 053/054/055 padding ceiling of MAX_E=104. This was caught before training rather than silently truncating data: MAX_N/MAX_E were widened to 90/130 (headroom above the new 67/106 observed max, same margin convention as every prior capacity bump).

3. Results: Held or Improved Everywhere

metric	Entry 055 (n=3,205)	Entry 056 (n=3,217)
same-chip MAE	1.16 pts	1.13 pts
same-chip R^2	0.970	0.976
Kyiv→Sherbrooke MAE / R^2 / fc	4.08 / 0.786 / 8.52	4.10 / 0.814 / 8.60
Sherbrooke→Kyiv MAE / R^2 / fc	2.87 / 0.894 / 3.72	2.65 / 0.905 / 2.87

Adding 12 large, structurally different circuits (deep routed star topologies vs. mostly short bell pairs) did not destabilize the model at the wider 90/130 capacity -- same-chip accuracy improved slightly, Kyiv-to-Sherbrooke R^2 improved (0.786 to 0.814) with MAE and floor-collapse essentially flat, and Sherbrooke-to-Kyiv improved on every metric (MAE 2.87 to 2.65, R^2 0.894 to 0.905, floor-collapse MAE 3.72 to 2.87 -- a new best across all entries in this direction).

Entry 056 — Conclusion

Growing the large-circuit tier and widening capacity a second time (80/104 to 90/130) composed cleanly with every prior fix, matching the pattern already seen across Entries 053-055: capacity growth and dataset growth are largely independent axes that do not trade off against each other in this architecture.

4. Next Steps

Uncertainty quantification is scoped next (Entry 057) as the primary research-value addition, run alongside continued large-circuit and dataset growth as capacity allows.

Research Log — Index Addendum (Entry 056)

Entry	Title	Date	Status
Entry 056	Large-Circuit Tier Doubled (12 to 24) + Capacity Bump to 90/130	Aug 25, 2026	Complete

Scripts: entry056_grow_bigger.py, entry056_build_graphs.py, entry056_train_gnn.py, entry056_cross_chip.py. Data: quantumbridge_data/entry052_biggerghz_dataset.json (24 records), entry056_graph_dataset.json (3,217 total), entry056_gnn_results.json, entry056_cross_chip_results.json. Max circuit size grew 65 to 67 nodes; MAX_N/MAX_E widened 80/104 to 90/130.

ENTRY 057 MC-Dropout Uncertainty Quantification — August 25, 2026 — Status: COMPLETE

Entry 057 — The Model Now Knows When It's Out of Its Depth

First Research-Value Feature Beyond Point Prediction: Calibrated Confidence Intervals

1. Motivation and Method

Every AI reviewer of the public demo (see the pasted Gemini/ChatGPT/Grok reviews earlier this project phase) converged on the same critique: a single point-estimate prediction gives no reason to prefer this model over just re-running Aer. MC-Dropout (Gal & Ghahramani, 2016) is the cheapest fix that directly answers this -- dropout (rate=0.15) is added after node embedding and after each message-passing round, kept active at both train and inference time. At inference, T=20 stochastic forward passes are run per circuit; their mean is the point prediction and their std is the uncertainty. Architecture, per-chip normalization (Entry 048), and floor-collapse up-weighting (Entry 051) are otherwise unchanged from Entry 056, so any behavior change is attributable to dropout alone. Cost: one training run instead of a 5-model ensemble.

2. A Methodological Error Caught and Fixed Mid-Entry

The first evaluation run used a random 80/20 KFold split of the full mixed-chip dataset, then labeled the chip of one held-out example as the "cold" chip. This is invalid: with a random split, the model had already seen plenty of that chip's circuits during training, so it was not testing genuine distribution shift -- it produced a misleadingly weak widening ratio (1.08x). This was caught before being reported as a result. The evaluation was rebuilt to match the proper cross-chip protocol used in Entries 048/051/053-056: train on ALL Kyiv circuits only, with Sherbrooke held out completely and never seen during training.

3. Results: Corrected Cross-Chip Protocol

test set	MAE	mean predicted std	corr(std, |error|)
held-out Kyiv (warm, in-distribution)	0.90 pts	0.57 pts	0.640
ALL Sherbrooke (cold, never seen)	4.63 pts	1.57 pts	0.489

Both calibration checks the plan called for came back positive. First, within the trained chip, higher predicted uncertainty correlates with higher actual error (r=0.64) -- the model's confidence is meaningfully informative, not noise. Second, and more importantly, mean predicted uncertainty nearly triples on the truly unseen chip (0.57 to 1.57 pts, a 2.77x widening ratio), tracking the real jump in error (MAE 0.90 to 4.63 pts) on that same transfer. The model is not just making worse predictions on new hardware -- it is correctly reporting that it should be trusted less there.

Entry 057 — Conclusion

This is the first capability in the project that Aer's point-estimate output cannot provide by construction: a self-reported, empirically calibrated confidence signal that widens correctly under distribution shift. It directly answers the external reviewers' "why not just use Aer" critique with a concrete, measured result rather than an architectural claim.

4. Next Steps

Expose the uncertainty band in the live demo alongside the point prediction (e.g. "72% ± 4%"). Continue large-circuit and dataset growth as capacity allows, and consider running MC-Dropout on the Sherbrooke-to-Kyiv direction too for a complete picture, since only Kyiv-to-Sherbrooke was tested here.

Research Log — Index Addendum (Entry 057)

Entry	Title	Date	Status
Entry 057	MC-Dropout Uncertainty Quantification (Kyiv→Sherbrooke)	Aug 25, 2026	Complete

Scripts: entry057_mc_dropout.py (modes: train/eval for same-chip, train_cc/eval_cc for proper cross-chip). Data: quantumbridge_data/entry057_mcdropout_results.json (same-chip, flawed cross-chip check retained for transparency),

entry057_mcdropout_cc_results.json (corrected cross-chip result). Uncertainty widens 2.77x on truly unseen chip, correlating with a real 5x increase in error (0.90 to 4.63 pts MAE).

ENTRY 057b Reverse-Direction Confirmation (Sherbrooke → Kyiv) — August 25, 2026 — Status: COMPLETE

Entry 057b — Uncertainty Widening Holds in Both Cross-Chip Directions

Confirms the Asymmetry Seen Across Entries 051-056 is Reflected in Uncertainty, Not Just Error

Results: Both Directions

direction	warm MAE	warm std	cold MAE	cold std	widening ratio
Kyiv → Sherbrooke	0.90	0.57	4.63	1.57	2.77x
Sherbrooke → Kyiv	1.23	0.97	2.20	1.56	1.61x

A second independently-trained MC-Dropout model, trained on Sherbrooke only with Kyiv held out completely, reproduces the same qualitative pattern: uncertainty widens on the unseen chip (1.61x), correlating with a real error increase (MAE 1.23 to 2.20 pts). The widening is smaller than Kyiv-to-Sherbrooke's 2.77x -- consistent with every prior entry in this project finding Kyiv-to-Sherbrooke to be the harder transfer direction (Entries 047, 048, 051, 053-056). The uncertainty signal is not just detecting error, it is detecting the same directional asymmetry the point-prediction error metrics have shown all along, which is strong evidence the confidence estimate is tracking something real about the underlying transfer difficulty rather than being an artifact of one run.

Conclusion

Uncertainty quantification is validated in both directions and ready to expose in the live demo. The remaining work -- re-precomputing predictions with uncertainty for the full 16,002-pair lookup table used by the live demo -- is scoped separately given its size (comparable to Entry 046's original full-coverage precomputation).

Index Addendum (Entry 057b)

Scripts: entry057_mc_dropout.py (train_cc/eval_cc, direction argument added). Data: quantumbridge_data/entry057_mcdropout_cc_rev_results.json. Confirms 2.77x/1.61x asymmetric widening matches the point-prediction error asymmetry across the whole project.

ENTRY 058 Chain-Topology Circuit Diversity — August 31, 2026 — Status: COMPLETE

Entry 058 — A Second Large-Circuit Topology Improves Kyiv-to-Sherbrooke, at a Small Cost to Sherbrooke-to-Kyiv

Every Large Circuit Before This Entry Was Star-Topology Only

1. Motivation

All 24 large circuits from Entries 052/056 were star-topology GHZ (one control qubit entangled to k-1 targets via a hub-and-spoke pattern). This entry adds a structurally different generator: a chain (q0-q1-q2-...-qk-1, entangled sequentially via H then a run of CX gates). Both produce the same GHZ state, but the chain routes as a long path rather than a hub, giving the GNN a genuinely different graph shape to learn from at large scale rather than only ever seeing star-shaped large circuits.

2. A Harder Generation Run Than Star Circuits

Chain circuits at k=14 proved too expensive to reliably generate within the sandbox's time budget -- Sherbrooke chain generation stalled entirely across two full-budget attempts and was cut rather than continuing to burn compute on it. The final set is 4 circuits, Kyiv only, at k=10 and k=12 (62-63 physical nodes) -- a small but structurally meaningful addition, not a full-scale replacement for star circuits.

3. Results: Same-Chip and Cross-Chip

metric	Entry 056 (n=3,217, star only)	Entry 058 (n=3,221, +chain)
same-chip MAE	1.13 pts	1.09 pts
same-chip R²	0.976	0.977
Kyiv→Sherbrooke MAE / R² / fc	4.10 / 0.814 / 8.60	3.55 / 0.859 / 7.02
Sherbrooke→Kyiv MAE / R² / fc	2.65 / 0.905 / 2.87	2.88 / 0.901 / 4.69

Kyiv-to-Sherbrooke -- the persistently harder direction across this whole project -- improved meaningfully on every metric (MAE 4.10 to 3.55, R² 0.814 to 0.859, floor-collapse MAE 8.60 to 7.02). This makes sense: the 4 new chain circuits are all on the Kyiv side, so Kyiv's training set gained topological diversity that Sherbrooke's training data (when training the other direction) did not. Sherbrooke-to-Kyiv regressed slightly on floor-collapse (2.87 to 4.69 pts) while staying flat on MAE/R² -- a small, honest cost, plausibly because the Kyiv-trained-then-tested-on-itself calibration shifted slightly with the added chain examples changing the per-chip normalization statistics.

Entry 058 — Conclusion

Topological diversity, even a small dose of it, helped the harder cross-chip direction more than it hurt the easier one -- a positive but imperfect result, reported here with both the gain and the regression rather than only the favorable number.

4. Next Steps

A fresh 3D visualization of the network showing both star and chain circuit topologies is next, followed by live demo polish (floor-collapse override explanation, cross-chip confidence context in the UI).

Research Log — Index Addendum (Entry 058)

Entry	Title	Date	Status
Entry 058	Chain-Topology Circuit Diversity (4 circuits, Kyiv, k=10/12)	Aug 31, 2026	Complete

Scripts: entry058_chain_circuits.py, entry058_build_graphs.py, entry058_train_gnn.py, entry058_cross_chip.py. Data: quantumbridge_data/entry058_chain_dataset.json, entry058_graph_dataset.json (3,221 total), entry058_gnn_results.json, entry058_cross_chip_results.json. Sherbrooke chain generation stalled and was skipped; Kyiv-only chain circuits still improved

Kyiv-to-Sherbrooke transfer meaningfully.

ENTRY 059 3D Star-vs-Chain Visualization + Demo Polish — August 31, 2026 — Status: COMPLETE

Entry 059 — Closing Out This Phase: Visualization and Demo UX

Two Small, Concrete Deliverables Rather Than New Modeling Work

1. New 3D Visualization

quantumbridge_3d_star_vs_chain.html: a fresh Three.js 3D force-directed view of the real 127-qubit Kyiv network (reusing the existing spring-layout node positions for continuity with the earlier full-network visualization), with two real, actually-routed large circuits highlighted in distinct colors -- the largest star-topology GHZ circuit (k=14, Entry 052/056) in orange, and the largest chain-topology GHZ circuit (k=12, Entry 058) in cyan. This makes the topological difference between the two large-circuit generators visually concrete rather than only described in text.

2. Live Demo Polish

Two additions to quantumbridge_live_demo.html's per-pair explanation: (a) a new "confident override" note that fires whenever the GNN disagrees with v4.1 by more than 15 points while reporting tight MC-Dropout uncertainty (<3%) -- surfacing the model's override behavior even for pairs without real ground truth, not just the previously-shown floor-collapse cases; (b) a standing validation disclosure appended to every result noting that cross-chip transfer was genuinely tested (not just same-chip splits) and that one direction is measurably harder than the other, pointing to the research log for the full numbers rather than overstating uniform reliability.

Entry 059 — Conclusion

This closes the current work phase: dataset at 3,221 circuits (star + chain large circuits), capacity at 90/130, generalization and floor-collapse fixes intact, uncertainty quantification built and validated in both directions, wired into the live demo, and the demo now explains its own override and reliability behavior rather than presenting bare numbers.

Research Log — Index Addendum (Entry 059)

Files: quantumbridge_3d_star_vs_chain.html, entry059_extract_routes.py. Demo changes: quantumbridge_live_demo.html (confident-override note, validation disclosure).

ENTRY 060 Sherbrooke Chain-Topology Generation, Fixed — September 1, 2026 — Status: COMPLETE

Entry 060 — Diagnosing and Fixing the Entry 058 Stall

Sherbrooke Chain Circuits Went From Zero to Twelve, and the Fix Also Helped Kyiv

The Problem

Entry 058 introduced chain-topology GHZ circuits (q0■q1■...■qk■1, entangled sequentially) as a second large-circuit shape alongside the existing hub-and-spoke star topology. Generation worked for Kyiv but stalled completely on Sherbrooke — two consecutive full 150-second time budgets produced zero Sherbrooke chain circuits, and the entry shipped with chain data Kyiv-only, an honestly logged gap.

Root Cause

Entry 058's generator sampled k qubits uniformly at random from the whole chip and chained them in that random order. Most consecutive pairs in the resulting chain are therefore physically far apart on the real coupling map, so after SWAP routing the circuit's entangling structure is not spatially local. The Aer matrix-product-state simulator is fast specifically because it exploits 1D-local entanglement — feed it a routed circuit that isn't spatially local and its bond dimension blows up. Isolated timing tests confirmed this directly: a single k=10 Sherbrooke trial with a random-order chain measured 48–120+ seconds, well past the 150-second total budget per chip. Kyiv's random samples happened to land local more often by chance in Entry 058's seed, which is why Kyiv appeared to work at all — the underlying bug was present there too.

The Fix

Build each chain by walking the real coupling graph itself: pick a random start qubit, then repeatedly step to a random unvisited physical neighbor, falling back to the nearest unvisited qubit by BFS distance when the walk boxes itself in. This is still a randomized chain — the start point and walk choices are seeded and vary run to run — but it stays spatially local on the chip, restoring the property that made MPS simulation fast in the first place. Measured effect on a single Sherbrooke k=10 trial: 48s → 0.5s, roughly a 90x speedup.

Results

With the fix, both chips generated their full target (4 circuits each at k=10/12/14) in under 6 seconds total per chip, versus the previous two full-budget timeouts producing zero on Sherbrooke. The chain-topology dataset grew from 4 circuits (Kyiv-only) to 24 (12 per chip), and the combined graph dataset grew from 3,221 to 3,241 circuits. Capacity headroom was unaffected — max nodes/edges across the new data (67/106) stayed well within the existing 90/130 padding limits, so no retraining-capacity bump was needed.

	Entry 058 (Kyiv-only chain)	Entry 060 (both chips)
	3,221	3,241
	4 (Kyiv only)	24 (12 Kyiv, 12 Sherbrooke)
²	1.09 / 0.977	1.16 / 0.970
lapse MAE	0.63	2.15
MAE / R² / fc_MAE	3.55 / 0.859 / 7.02	3.69 / 0.839 / 7.40
MAE / R² / fc_MAE	2.88 / 0.901 / 4.69	2.98 / 0.879 / 2.45

Reading the Results Honestly

This is not a uniform win, and it is reported as such. Sherbrooke→Kyiv floor-collapse MAE improved substantially (4.69 → 2.45 points) — training on Sherbrooke's new chain data measurably helped the model handle floor-collapse cases when tested cold on Kyiv. But same-chip MAE, same-chip floor-collapse MAE, and Kyiv→Sherbrooke MAE/R² all moved slightly in the worse direction. The most likely explanation is not model

regression but a harder, more honest test set: the Kyiv→Sherbrooke and same-chip evaluations now include the 12 new Sherbrooke chain circuits for the first time, several of which are genuine floor-collapse cases (aer_ground_truth as low as 6–14%) that simply did not exist in the Sherbrooke test pool before. A test set that got harder because it got more representative is not the same thing as a model that got worse, but the two are easy to conflate, so both readings are recorded here rather than only the favorable one.

Entry 060 — Conclusion

The Sherbrooke chain-generation gap flagged as unresolved in Entry 058 is closed: both chips now have symmetric star and chain large-circuit coverage, diagnosed via direct timing measurement rather than guesswork, and fixed with a 90x generation speedup that also quietly benefited Kyiv. Cross-chip results are mixed rather than uniformly better, and are reported that way — the clearest gain is Sherbrooke→Kyiv floor-collapse MAE, while Kyiv→Sherbrooke got a harder, more representative test set rather than a worse model.

Research Log — Index Addendum (Entry 060)

Files: entry060_chain_circuits.py (fixed generator), entry060_build_graphs.py, entry060_train_gnn.py, entry060_cross_chip.py. Data: entry058_chain_dataset.json (24 records, same file, both entries append), entry060_graph_dataset.json (3,241 graphs), entry060_gnn_results.json, entry060_cross_chip_results.json.

ENTRY 061 — Third Chip (Brisbane): Generalization Does Not Trivially Extend — September 1, 2026 — Status: COMPLETE, RESULT SOBERING

Entry 061 — Adding a Third Chip to Test Whether Cross-Chip Transfer Was Real

Setup, Dataset, and a Result That Does Not Confirm the Hoped-For Story

Motivation

Every cross-chip generalization result so far (Entries 047, 048, 050, 051, 058, 060) tested exactly one pair: Kyiv and Sherbrooke. A model that transfers well between two chips could genuinely have learned physical noise structure that generalizes — or it could be a coincidence of those two specific calibration snapshots. The only way to tell the difference is a third, independently-calibrated chip the model has never touched in any prior entry. FakeBrisbane was added for exactly this reason: same 127-qubit heavy-hex class as Kyiv and Sherbrooke (to avoid also confounding chip scale with chip identity), but its own real calibration data.

Dataset Built for Brisbane

740 bell-pair circuits across the same six hop-distance bins used throughout the project, plus 12 chain-topology large circuits (k=10/12/14, using the spatially-local chain-walk fix from Entry 060). Star-topology large circuits were attempted and abandoned this round: a single k=10 star circuit did not finish Aer/MPS simulation within 170 seconds on Brisbane, for a structural reason different from Entry 058's chain-generation stall — a hub-and-spoke star's entangling structure is not spatially local in Hilbert space regardless of qubit ordering (unlike a chain, which can be reordered into a local walk), so Brisbane's large-circuit tier is chain-only this round. This is logged honestly rather than silently worked around, and is flagged as follow-up work.

A Real Infrastructure Bug Found and Fixed Along the Way

The bell-pair generator initially produced zero circuits beyond hop-distance 7 across five consecutive generation rounds, despite the sampler correctly queuing plenty of longer-hop tasks each time. Root cause: short-hop circuits simulate fast and long-hop circuits simulate slow (more SWAPs, deeper routing), and the task list was unshuffled, so every timed run burned its entire budget on the fast short-hop tasks at the front of the list and never reached the slow long-hop tasks queued behind them. Shuffling the task order before submission fixed it immediately. Separately, the GNN training script needed its own fix once Brisbane's data pushed the combined dataset to 3,993 records: some training calls were silently hanging past the sandbox's time limit without saving a checkpoint, traced to the shell's default SIGTERM not reliably stopping a JAX process in this environment — switching to SIGKILL (`timeout -s KILL`) resolved it, and per-epoch timing confirmed the actual training compute was fast (roughly 1.5-3s/epoch) the whole time.

The Generalization Test

Same-chip baseline across all three chips combined: MAE 1.11, R^2=0.971, floor-collapse MAE 1.06 — consistent with every prior same-chip result, confirming the model architecture and training procedure are working normally on the combined data. The real test is the two genuinely cold cross-chip directions:

	MAE / R^2 / floor-collapse MAE	n_train / n_test
→ Brisbane (cold)	4.91 / 0.141 / 22.58	3,241 / 752
Sherbrooke (cold)	3.98 / 0.794 / 18.73	752 / 3,241
Kyiv → Sherbrooke	3.69 / 0.839 / 7.40	2,489 / 752
Sherbrooke → Kyiv	2.98 / 0.879 / 2.45	752 / 2,489

Both directions involving Brisbane are markedly worse than the established Kyiv↔Sherbrooke pair, and one of them is not a small degradation: R^2=0.141 for Kyiv+Sherbrooke→Brisbane is close to what a model predicting the training-set mean for every test example would score. Floor-collapse error roughly tripled in both directions relative to the two-chip baseline.

Reading This Honestly

This does not confirm the hoped-for story that cross-chip generalization is a general property of the model. It shows the opposite, at least so far: transfer that looked solid between Kyiv and Sherbrooke did not carry over cleanly to a third chip. Two explanations are plausible and not yet distinguished from each other. First, a genuine physical-noise-structure limit — Brisbane's calibration profile may simply differ from Kyiv/Sherbrooke's shared structure more than they differ from each other, and per-chip relative normalization (Entry 048) may not be sufficient to bridge a bigger gap. Second, a distributional confound this entry introduced: Brisbane's dataset has no star-topology large circuits (the star generator stalled and was skipped this round), while both training sets it's compared against are full of them, and the floor-collapse test population is small and unevenly sized between directions (13 examples one way, 190 the other) — not nearly the statistical footing the two-chip comparisons stand on. The honest conclusion at this scale is: the original two-chip result should not be assumed to generalize to a third chip without direct testing, which is exactly what this entry did, and what it found was a real gap that needs more data and a cleaner distributional match before it can be called resolved either way.

Entry 061 — Conclusion

A third, independently-calibrated chip was added to the pipeline for the first time (setup, 740 bell-pair circuits, 12 chain-topology large circuits, 3,993-graph combined dataset), and two real bugs were found and fixed along the way (unshuffled long-hop task starvation; a training hang traced to signal handling, not compute speed). The generalization test itself did not confirm what Entries 047-060 might have suggested to hope for: Kyiv↔Sherbrooke transfer does not currently extend cleanly to Brisbane. This is reported as the primary finding of this entry, not buried under the infrastructure wins, and is the clearest open item for the next round of work — likely requiring a matched-distribution Brisbane dataset (including star-topology circuits) before the question can be answered cleanly.

Research Log — Index Addendum (Entry 061)

Files: setup_brisbane.py, entry061_grow_brisbane.py, entry061_brisbane_large_circuits.py, entry061_build_graphs.py, entry061_train_gnn.py, entry061_cross_chip.py. Data: quantumbridge_data/entry061_brisbane_bell_dataset.json (740), entry061_brisbane_chain_dataset.json (12), entry061_graph_dataset.json (3,993 graphs), entry061_gnn_results.json, entry061_cross_chip_results.json.

ENTRY 062 — Brisbane Star-Topology Gap Closed — Distributional Confound Partly Confirmed, Real Gap Remains — September 1, 2026 — Status: COMPLETE

Entry 062 — Closing the Star-Topology Gap Flagged in Entry 061

Does a Matched Circuit-Topology Mix Close the Brisbane Generalization Gap?

Background

Entry 061 found that Kyiv↔Sherbrooke cross-chip transfer did not extend cleanly to Brisbane, and flagged two live explanations without distinguishing them: a genuine physical difference in Brisbane's noise structure, or a distributional confound — Brisbane's dataset had zero star-topology large circuits (that generator stalled) while both chips it was compared against are full of them. This entry closes that confound directly by fixing star-topology generation for Brisbane and re-running the exact same test.

The Star-Topology Fix

Entry 061 diagnosed the star stall as different in kind from Entry 058/060's chain stall: a chain's entanglement structure can be made spatially local by walking the coupling graph in order (Entry 060's fix), but a star's hub-and-spoke structure is non-local in Hilbert space regardless of qubit ordering — the hub is entangled with every spoke, and reordering the spokes doesn't change that. What can be controlled is physical distance: if the k-1 target qubits are chosen near the control qubit on the real coupling map (nearest by BFS distance, with light randomization among near-tied candidates) rather than uniformly at random across the whole 127-qubit chip, the *routed* circuit needs few or no SWAPs, keeping it shallow enough for Aer's MPS simulator. Measured effect on a single Brisbane k=10 star: did not finish in 170s with fully random targets; 0.5s with local targets. All three sizes (k=10/12/14, 4 circuits each) generated successfully, in under 90 seconds total.

Re-Running the Generalization Test

The 12 new star circuits were folded into the graph dataset (3,993 → 4,005), and both cross-chip directions from Entry 061 were re-run unchanged otherwise:

	Entry 061 (chain-only Brisbane)	Entry 062 (chain+star Brisbane)
→ **Brisbane**	MAE 4.91 / R²=0.141 / fc 22.58	MAE 5.23 / R²=0.354 / fc 25.11
Sherbrooke	MAE 3.98 / R²=0.794 / fc 18.73	MAE 4.62 / R²=0.718 / fc 22.85
→ **Sherbrooke**	MAE 3.69 / R²=0.839 / fc 7.40	unchanged

Reading This Honestly

The distributional-confound hypothesis is partly confirmed, not fully. Closing the star-topology gap measurably improved the harder direction — R² for Kyiv+Sherbrooke→Brisbane more than doubled, from 0.141 to 0.354. That is real evidence that at least part of Entry 061's poor result was an artifact of comparing mismatched circuit mixes, not pure physics. But 0.354 is still far below the 0.84–0.88 range the Kyiv↔Sherbrooke pair achieves, and floor-collapse error actually got slightly worse (22.58 → 25.11), not better. The reverse direction moved the wrong way too (0.794 → 0.718), though on a small 764-circuit training set that's plausibly within run-to-run noise rather than a real regression. Taken together: matching the circuit-topology mix closes part of the gap but not most of it. There is a real, substantial generalization difficulty specific to Brisbane that a distributional confound does not fully explain, and that remains open. The most likely next lever, per the master plan already agreed with the user, is scale: Brisbane's dataset is still roughly a quarter the size of Kyiv's or Sherbrooke's individually, and growing it further before drawing firmer conclusions is the planned next step rather than treating this result as final.

Entry 062 — Conclusion

Star-topology generation for Brisbane is fixed (local-target selection instead of uniform random, closing a real generator gap the same way Entry 060 closed Sherbrooke's chain gap), and the fix was used immediately to test rather than assumed to help: it measurably improved one direction of three-chip transfer and did not resolve the other. The honest state of the third-chip generalization question is unchanged in kind from Entry 061 -- a real gap exists that isn't fully explained by dataset composition -- but the evidence is now cleaner, since circuit-topology mix is no longer a confound sitting on top of it.

Research Log — Index Addendum (Entry 062)

Files: entry061_brisbane_large_circuits.py (local_star_targets fix), entry061b_build_graphs.py, entry061b_train_gnn.py, entry061b_cross_chip.py. Data: quantumbridge_data/entry061_brisbane_star_dataset.json (12), entry061b_graph_dataset.json (4,005 graphs), entry061b_gnn_results.json, entry061b_cross_chip_results.json.

ENTRY 064 Brisbane Dataset to 1,621 — Scale Helps One Direction, Not the Other — September 3, 2026 — Status: COMPLETE

Entry 064 — Does More Brisbane Data Close the Remaining Generalization Gap?

Bell-Pair Dataset Grown 740 → 1,621; Infrastructure Bug Fixed Along the Way

What Was Done

Following Entry 062's finding that a distributional confound only partly explained Brisbane's weak cross-chip transfer, Brisbane's bell-pair dataset was grown from 740 to 1,621 circuits, more than doubling it, to test whether the remaining gap shrinks with scale the way the original two-chip generalization work (Entries 053-058) improved with more data. The 881 new circuits were folded into the graph dataset (4,005 → 4,886), and both cross-chip directions were re-trained and re-evaluated.

A Real Data-Loss Incident, Caught and Fixed

Partway through this growth, a checkpoint write was killed mid-write by the sandbox's hard timeout and corrupted the dataset file, destroying progress back to 66 records (from 900). Root cause: the generator wrote checkpoints via `json.dump` directly to the live file, which truncates the file immediately on open and streams content incrementally -- a kill mid-write leaves a corrupt fragment and destroys the prior good save too, not just the in-progress one. This was caught immediately (not discovered later), the actual damage was assessed honestly (the graph dataset already had 764 circuits permanently baked in from before, so only ~160 records of unconsumed growth were actually lost), and the generator was fixed to write to a temp file and rename atomically before continuing. This is the same class of fix Entries 058/060 applied to circuit generation speed -- here applied to generation safety.

Results

	Entry 062 (764 Brisbane)	Entry 064 (1,645 Brisbane)
Brisbane	MAE 5.23 / R^2=0.354 / fc 25.11 (n_fc=14)	MAE 5.89 / R^2=0.275 / fc 15.57 (n_fc=35)
erbrooke	MAE 4.62 / R^2=0.718 / fc 22.85	MAE 3.76 / R^2=0.811 / fc 19.43

Reading This Honestly

One direction improved cleanly with scale; the other did not, and both readings are reported rather than only the favorable one. Brisbane→Kyiv+Sherbrooke improved on every metric and reached its best result across all three entries (R^2=0.811, the closest any Brisbane-involving direction has come to the original Kyiv↔Sherbrooke baseline of 0.84–0.88) -- consistent with the hypothesis that more training diversity helps the model learn chip-invariant structure. Kyiv+Sherbrooke→Brisbane did not follow the same pattern: R^2 moved from 0.354 down to 0.275, even though floor-collapse error on that same test actually improved substantially (25.11 → 15.57) and the floor-collapse test population nearly tripled (14 → 35 examples, a much sounder sample). The most likely explanation is not that the model regressed, but that Brisbane's growth deliberately targeted underfilled long-hop-distance bins (Entry 064's own growth run stalled repeatedly on exactly these, taking dedicated debugging to push through), so the held-out Brisbane test set is now structurally harder -- more long-route, higher-decoherence circuits -- than it was in Entry 062. A test set that got harder because it got more representative of the full distribution is not the same thing as a model that got worse, and the improved floor-collapse number supports that reading. But this is an inference, not a proof, and is recorded as an open question rather than a settled one.

Entry 064 — Conclusion

Brisbane's dataset more than doubled (740 → 1,621 bell pairs, 4,886 graphs combined), with a checkpoint-corruption bug caught and fixed with atomic writes along the way. Scale measurably helped the easier cross-chip direction to its best result yet, and produced a genuinely mixed, honestly-reported result on the harder direction -- worse headline R^2

but better floor-collapse accuracy on a much larger, harder test sample. The third-chip generalization question remains open in the same way it has since Entry 061: real progress, not yet resolved.

Research Log — Index Addendum (Entry 064)

Files: entry061_grow_brisbane.py (atomic-write fix), entry063_build_graphs.py, entry063_train_gnn.py, entry063_cross_chip.py.
Data: quantumbridge_data/entry061_brisbane_bell_dataset.json (1,621), entry063_graph_dataset.json (4,886 graphs), entry063_gnn_results.json, entry063_cross_chip_results.json.

ENTRY 065 — The Combined Three-Chip Generalist Model — September 3, 2026 — Status: COMPLETE, RESULT STRONG

Entry 065 — A Different Question From Entries 060-064

Not "Does It Transfer Cold" But "Does It Work Well If It's Seen a Little of Everything"

The Question, Asked Directly by the User

Entries 060-064 all tested strict leave-one-chip-out generalization: train on some chips, evaluate cold on a chip that contributed zero training examples. That's the right test for "does this model understand physics that transfers to a totally novel chip," and the answer so far has been a real, only partially explained gap for Brisbane. But it is a different question from "if I pool data from every chip I have and train one model on all of it, does that model work well across all of them" -- which is what a practical deployed tool would actually do, and what the user asked to test directly rather than assume.

Setup

All 4,886 circuits across Kyiv, Sherbrooke, and Brisbane were pooled into one training pool, with a random stratified 80/20 split (the same split used for entry063's same-chip baseline, so the two numbers are directly comparable) -- meaning every chip contributes both training examples and held-out test examples, unlike Entries 060-064 where an entire chip was held out of training completely. Architecture, per-chip relative normalization (Entry 048), and floor-collapse up-weighting (Entry 051) are all unchanged. This time the trained weights were kept as a deployable artifact rather than discarded after cross-validation, since this is the first genuinely usable combined model the project has produced.

Results — Per-Chip Breakdown of the Held-Out Test Set

Chip	n (test)	MAE	R^2	floor-collapse MAE
Kyiv	432	0.96	0.989	0.65 (n=28)
Sherbrooke	222	1.74	0.937	3.13 (n=19)
Brisbane	324	1.05	0.981	0.39 (n=8)
Overall	978	1.17	0.973	—

What This Means

Brisbane's numbers here (MAE 1.05, R^2=0.981) are dramatically better than any leave-one-chip-out result this project has produced for Brisbane (best zero-shot R^2 so far: 0.354, Entry 062) -- close to Kyiv's own held-out performance, and better than Sherbrooke's in this run. That is a clean, important distinction to draw: the poor zero-shot transfer found in Entries 061-064 is not evidence that the model fundamentally cannot handle Brisbane's noise structure. It is evidence that the model cannot handle Brisbane's noise structure with *zero* examples from Brisbane. Give it even a partial, imperfect slice of Brisbane's own data during training -- which is exactly what a real deployed tool would have access to -- and it performs comparably to the chips it has always seen far more of. This reframes the earlier finding rather than contradicting it: the two experiments answer genuinely different questions, and both answers are now known.

What This Does Not Show

This result says nothing about a fourth, still-unseen chip -- that is exactly the leave-one-out test this entry deliberately did not run. A model that has seen a slice of every chip it is evaluated on cannot demonstrate transfer to a chip it has never touched; only Entries 061-064's harder protocol can test that, and did. The two results together are more informative than either alone: broad multi-chip training produces a strong practical generalist (this entry), while true zero-shot transfer to a genuinely novel chip remains an open, partially-understood problem (Entries 061-064).

Entry 065 — Conclusion

The combined three-chip model is the first version of QuantumBridge's GNN trained on the full breadth of available chip data at once, and its weights are saved as a deployable artifact (entry065_combined_params.json) rather than discarded. It performs well across all three chips including Brisbane, closing the practical gap that Entries 061-064 left open -- while leaving the harder scientific question (true zero-shot transfer to an unseen fourth chip) explicitly unanswered, exactly as it should, since this experiment was never designed to answer it.

Research Log — Index Addendum (Entry 065)

Files: entry065_combined_train.py. Data: quantumbridge_data/entry065_combined_params.json (deployable weights), entry065_combined_results.json (per-chip breakdown).

ENTRIES 067-070 Balancing Sherbrooke: Growth, Retrain, Re-Score — September 4, 2026 — Status: COMPLETE

Balancing the Dataset Across All Three Chips

Sherbrooke Was the Thinnest Chip -- This Closes That Gap

Motivation

After Entries 060-066, chip dataset sizes were uneven: Kyiv ~1,510 bell circuits, Brisbane ~1,621 (after its own dedicated growth push), but Sherbrooke only 723 -- the thinnest of the three despite being one of the two original chips. Entry 067 grows Sherbrooke specifically, reusing the same methodology as Brisbane's growth (Entry 061): Aer/MPS ground truth, six hop-distance bins, atomic checkpointed writes, shuffled task order to avoid starving long-hop bins. 804 new circuits were added (723 -> 1,527 bell, plus the existing 24 chain circuits), bringing Sherbrooke roughly to parity with the other two chips.

Entry 068 — Fold and Retrain

The 804 new circuits were folded into the graph dataset (4,886 -> 5,690 graphs, still well within the 90/130 node/edge capacity -- max 67 nodes, 106 edges). Same-chip 80/20 baseline was retrained: MAE=1.07, R^2=0.965, essentially unchanged from Entry 063's 1.17/0.973 at the smaller scale, confirming the added Sherbrooke data didn't destabilize the well-established same-chip fit.

Leave-One-Chip-Out Cross-Chip Results — Before / After Sherbrooke Growth

Direction	R^2 (Entry 063)	R^2 (Entry 068)	Change
kyiv+sherbrooke → brisbane	0.275	0.407	+0.132 (better)
brisbane → kyiv+sherbrooke	0.811	0.733	–0.078 (worse)

The two directions moved in opposite ways, and both are explainable rather than contradictory. Training on a richer, more balanced kyiv+sherbrooke pool gave the model more diverse noise structure to learn from, which transferred better to the untouched Brisbane test set -- a real generalization gain. The reverse direction got harder for a structural reason, not a regression: the kyiv+sherbrooke test set itself grew by 804 new, harder examples (many long-hop, previously-unsampled pairs) that a Brisbane-only-trained model had no chance to prepare for. This is the same pattern seen in Entries 061-064's non-monotonic Brisbane R^2 trajectory: a harder, more representative test set can look like a worse score even when nothing about the model's underlying competence changed.

Entry 069 — Combined Three-Chip Generalist, Retrained

Chip	MAE (Entry 065)	MAE (Entry 069)	floor-collapse MAE (069)
Kyiv	0.96	0.77	0.56 (n=26)
Sherbrooke	1.74	1.45	0.56 (n=40)
Brisbane	1.05	0.99	0.32 (n=4)
Overall	1.17	1.07	—

Every chip's held-out MAE improved, and Sherbrooke's floor-collapse MAE -- the hardest failure mode this project has chased since Entry 036 -- fell from 3.13 to 0.56, a direct benefit of Sherbrooke finally having enough training volume to let the floor-collapse up-weighting (Entry 051) do its job properly. The deployable weights were saved (entry069_combined_params.json) and used to re-score all 24,003 precomputed qubit pairs (Entry 070), replacing the live demo's Brisbane predictions and refreshing ground truth for the 804 newly-simulated Sherbrooke pairs.

Entries 067-070 — Conclusion

Balancing the thinnest chip's dataset produced a strictly better combined generalist model across all three chips, and a real (if partial) improvement in the hardest cross-chip generalization direction. It also produced a harder, more honest test in the other direction -- exactly the kind of result this project has learned to report as informative rather than alarming. The live demo now reflects the retrained combined model and the wider ground-truth coverage for Sherbrooke.

Research Log — Index Addendum (Entries 067-070)

Files: entry067_grow_sherbrooke.py, entry068_build_graphs.py, entry068_train_gnn.py, entry068_cross_chip.py, entry069_combined_train.py, entry070_score_all_pairs.py. Data: quantumbridge_data/entry067_sherbrooke_bell_dataset.json (804 new circuits), entry068_graph_dataset.json (5,690 graphs), entry069_combined_params.json (deployable weights), entry070_combined_lookup.json (rescored demo lookup).

ENTRIES 071-072 One Unified Uncertainty Model for All Three Chips — September 4, 2026 — Status: COMPLETE

Retiring the Two-Lineage Split

Kyiv/Sherbrooke Had Calibrated Uncertainty, Brisbane Didn't -- Now All Three Do

Why This Was Needed

Since Brisbane was added (Entry 061), the deployed demo ran two separate model lineages: Kyiv/Sherbrooke used Entry 057's MC-Dropout model, trained before Brisbane existed on a 2-chip, 2,329-circuit dataset, with a validated calibrated uncertainty band. Brisbane used Entry 065/069's combined generalist model, trained without dropout (so any inference-time-only dropout applied to it would have produced an uncertainty-shaped number that was never validated -- the project has been careful throughout not to do this). The result was an honest but visible asymmetry: two different models, two different guarantees, patched around with explicit labeling in the demo rather than actually resolved.

Entry 071 — One Model, Trained With Dropout, On All Current Data

A single MC-Dropout-enabled model (same architecture, same per-chip normalization, same floor-collapse up-weighting as every GNN since Entry 048/051) was trained on the full current entry068_graph_dataset.json -- 5,690 circuits across Kyiv, Sherbrooke, and Brisbane together, the balanced post-Entry-067 dataset. Dropout (rate=0.15) is active both during training and at inference (T=20 stochastic passes), matching Entry 057's original methodology exactly -- only the underlying data and chip count changed.

Calibration Check — Genuine Held-Out Validation

	MAE	R^2	mean predicted std	corr(std, |error|)
Held-out test (80/20 CV)	1.38	0.964	0.81 pts	0.553

This calibration run used a separate model trained on only 80% of the data (never seeing the held-out 20%), exactly mirroring Entry 057's original validation protocol -- distinct from the deployed model itself, which is trained on all data for maximum production accuracy. The positive, meaningfully-sized correlation (0.553) between predicted uncertainty and actual error confirms the model knows when it's less confident, not just that it produces a number. Per-chip breakdown on the held-out set: Sherbrooke showed both the highest mean predicted std (1.05pts) and the highest MAE (1.56) -- the model correctly flags its own weakest chip.

Entry 072 — Re-Score and Deploy

All 24,003 precomputed qubit pairs across all three chips were re-scored with the deployed unified model, replacing both the old Entry 057 lookup (Kyiv/Sherbrooke, 2-chip-era) and the Entry 070 lookup (Brisbane, point-estimate-only) with one consistent table. The live demo's special-cased "no uncertainty band yet" messaging for Brisbane was removed -- there is no longer a code path where any chip lacks an uncertainty estimate. The validation-note footer was rewritten to honestly describe what this deployment model is (a generalist trained on data pooled from all three chips) versus what the separate leave-one-chip-out experiments (Entries 047-068) tested (true cold transfer to an entirely unseen chip) -- the two are not the same claim, and the demo now says so for every chip rather than singling out Brisbane.

Entries 071-072 — Conclusion

The project now has one deployed model, one validated uncertainty mechanism, and one honest validation note that applies uniformly across Kyiv, Sherbrooke, and Brisbane -- replacing two parallel model lineages with different guarantees. This closes the first item of the three-part roadmap agreed for this phase (uncertainty parity, a fourth never-seen-chip test, then productization); the fourth-chip generalization test is next.

Research Log — Index Addendum (Entries 071-072)

Files: entry071_mc_dropout.py, entry071_deploy_train.py, entry072_score_all_pairs.py. Data: quantumbridge_data/entry071_deploy_params.json (deployable weights, dropout_rate=0.15), entry072_lookup_with_uncertainty.json (rescored demo lookup, all 3 chips with calibrated uncertainty).

ENTRY 073 — The True Zero-Shot Fourth-Chip Test — September 5, 2026 — Status: COMPLETE, HONEST MIXED RESULT

Osaka: A Chip the Model Has Never Touched

The Cleanest Cold-Transfer Test This Project Has Run

Why This Test Is Different From Every Prior One

Brisbane (Entry 061) was added specifically to test cross-chip generalization, but became a training chip almost immediately (Entry 065's combined model) -- so no result since has reflected a chip the DEPLOYED model has literally never seen. FakeOsaka (127 qubits, same heavy-hex class as the other three, 144 real calibrated edges) was set up and a 708-circuit eval-only bell-pair dataset was grown for it, but -- critically -- never folded into any training pipeline. The Entry 071 deployed model, trained on Kyiv+Sherbrooke+Brisbane, was scored cold against it. Osaka has no per-chip normalization statistics in the model at all; the average of the three training chips' stats was used as the best available substitute, since a real deployed tool facing an unfamiliar chip would need some answer here.

Results — Osaka, Zero Training Exposure

Metric	GNN (Entry 071)	v4.1 closed-form	GNN on trained chips (ref.)
MAE	5.82	5.57	1.38
R^2	0.623	0.686	0.964
floor-collapse MAE (n=49)	15.77	22.38	0.50-0.56
mean predicted std	3.27 pts	n/a	0.81 pts

Two honest findings, not one. First: on aggregate accuracy, the GNN does NOT clearly beat the hand-derived v4.1 closed-form formula on a genuinely novel chip -- they are roughly tied, with v4.1 very slightly ahead on both MAE and R^2. This is worth stating plainly rather than downplaying: the GNN's advantage over the closed-form model, demonstrated repeatedly on chips it has trained on, does not automatically transfer to a chip with zero exposure. Second: on floor-collapse specifically -- the failure mode v4.1 structurally cannot see at all, and the entire reason the GNN was built (Entry 036 onward) -- the GNN still meaningfully outperforms v4.1 even zero-shot (15.77 vs 22.38 MAE), suggesting the floor-collapse detection capability generalizes better than raw point-accuracy does.

The Uncertainty Mechanism Passes Its Hardest Test

Mean predicted uncertainty on Osaka (3.27 pts) is roughly 4x wider than on the model's home chips (0.81 pts) -- and the actual MAE degradation is also roughly 4x (5.82 vs 1.38). The MC-Dropout uncertainty band, calibrated in Entry 071 on held-out data from the training chips, correctly and proportionally widens when the model is genuinely out of its depth on a chip it has never seen at all. This is the most important validation this uncertainty mechanism has had: it is not just calibrated on interpolation within known chips, it also degrades gracefully and honestly under true extrapolation, which is exactly the property a deployed tool needs to avoid confidently wrong answers on unfamiliar hardware.

Entry 073 — Conclusion

The true zero-shot test gives a mixed, credible answer rather than a clean win: the GNN's raw accuracy advantage over the closed-form baseline does not fully survive a genuinely unseen chip, but its floor-collapse detection and its uncertainty calibration both do. For a project whose stated long-term goal is a deployable tool anyone can query on any chip, this is the single most useful result to have on record before productization begins: it tells a future user exactly what to expect (roughly v4.1-level point accuracy, better floor-collapse catching, and honestly wide error bars) on a chip the model was never trained on, rather than an optimistic claim that would not hold up.

Research Log — Index Addendum (Entry 073)

Files: setup_osaka.py, entry073_grow_osaka.py, entry073_zero_shot_test.py. Data: quantumbridge_data/entry073_osaka_bell_dataset.json (708 eval-only circuits, never trained on), entry073_osaka_graph_dataset.json, entry073_zero_shot_results.json.

ENTRY 074 Live Inference API — v1 Productization Begins — September 5, 2026 — Status: COMPLETE

From Static Demo to Real Backend

The First Concrete Step of the Roadmap's Third Phase

Motivation

Entries 060-073 established the science: a validated GNN with calibrated MC-Dropout uncertainty, tested three independent ways (leave-one-chip-out transfer, pooled multi-chip generalist, and Entry 073's true zero-shot fourth-chip test). But everything the public could actually touch -- the live demo -- ran on a static, precomputed 24,003-row lookup table, not a real service. Productizing the big "any circuit, any chip" vision outright would have outrun what's actually been validated. Instead, this entry builds a tightly-scoped v1: a real FastAPI backend that does live Qiskit routing, feature extraction, and GNN inference on demand, for exactly the three chips and one circuit type (bell pairs) the model was actually trained and validated on.

What Was Built

api/main.py exposes three endpoints: GET /health (scope and validation caveats, returned in every response, not buried in docs), GET /chips, and POST /predict, which takes a chip and two qubit indices and runs the full live pipeline -- SABRE-routed transpile, v4.1 closed-form prediction, graph feature extraction (entry044_build_graphs.graph_for_record), and a 20-sample MC-Dropout forward pass using the Entry 071 deployed weights -- computed fresh per request, not looked up. Requesting an unsupported chip returns an explicit refusal citing Entry 073's findings rather than silently extrapolating.

Validation Against the Offline Pipeline

Pair	v4.1 (offline)	v4.1 (live)	GNN (offline)	GNN (live)
kyiv 0-1	0.9766	0.9766 (exact)	0.9775	0.9775
kyiv 0-11	0.762	0.762 (exact)	0.7985	0.7981
sherbrooke 0-11 (fc)	0.715	0.715 (exact)	0.4974	0.4974 (exact)
brisbane 0-101	0.6854	0.6854 (exact)	0.758	0.7693

The v4.1 closed-form prediction matches the precomputed table exactly on every tested pair, confirming the live routing pipeline reproduces the offline one bit-for-bit. The GNN mean matches closely but not exactly -- the small differences (well under a percentage point on three of four pairs) are expected MC-Dropout sampling variance, not a bug, and the floor-collapse Sherbrooke pair matches exactly (0.4974, std=0 both times), which is itself informative since that's the one case where dropout noise doesn't move a saturated prediction. Latency was 18-110ms per request in local testing.

Entry 074 — Conclusion

This is a small, deliberately incomplete step -- no auth, no rate limiting, no hosting, no circuit types beyond bell pairs -- but it is the actual architectural change productization needed: a callable service instead of a static table, validated against the offline pipeline it replaces, with every one of its stated limitations disclosed in its own API responses rather than left implicit. Deployment (choosing and configuring a host) is the user's remaining step, documented in api/README.md.

Research Log — Index Addendum (Entry 074)

Files: api/main.py, api/requirements.txt, api/README.md.

ENTRY 075 Wiring the Demo to the Live API — September 5, 2026 — Status: COMPLETE

A Progressive Upgrade, Not a Hard Cutover

The Demo Works Identically Today and Upgrades Automatically Once Deployed

What Changed

quantumbridge_live_demo.html now tries a live request to the Entry 074 API first (4-second timeout via AbortController), and only falls back to the precomputed 24,003-row table if the API is unreachable or a base URL hasn't been configured. Every result is now labeled with an explicit badge -- green "live prediction" or grey "cached / precomputed" -- so a viewer always knows which path served their answer, rather than the two being silently indistinguishable. Because API_BASE_URL defaults to an empty string, the demo's behavior is completely unchanged until someone deploys the API and sets that one constant -- this is a progressive upgrade wired in ahead of time, not a dependency that had to ship together with a live deployment.

CORS

The API (Entry 074) added CORSMiddleware with a permissive allow_origins=["*"] policy, verified locally with a real preflight OPTIONS request carrying the GitHub Pages origin header -- confirmed the server responds with the correct access-control-allow-origin and access-control-allow-methods headers. This is deliberately permissive since /predict is a public, read-only, no-auth endpoint with no user data; it should be tightened before adding anything that isn't purely read-only.

What Ground Truth Display Still Relies On

Live requests don't run a fresh Aer simulation per pair (too slow for interactive use), so the "Simulated hardware result" field still comes from the local precomputed table when available, merged in alongside the live v4.1/GNN predictions -- a live prediction can still show real ground truth if that specific pair happened to be simulated during training or evaluation, it just won't trigger a brand-new simulation on demand.

Entry 075 — Conclusion

The frontend is ready for a live backend the moment one exists at a real URL -- deployment itself (choosing and configuring a host) remains the user's step, documented in api/README.md's new "Wiring up the live demo" section. Nothing about the current, already-deployed demo changes until that URL is set.

Research Log — Index Addendum (Entry 075)

Files: quantumbridge_live_demo.html (API_BASE_URL, fetchLive(), badge system), api/main.py (CORS middleware), api/README.md (wiring instructions).

ENTRIES 076-078 The Fourth Chip Graduates, the Fifth Chip Holds — September 16, 2026 — Status: COMPLETE

Folding Osaka Into Training, Setting Up Quebec as the New Holdout

A Combined 4-Chip Model, Tested Zero-Shot Against a Genuinely Unseen 5th Chip

Why

Entry 073 held Osaka out cleanly to run this project's first true zero-shot fourth-chip test. That job is done -- the result is on record. Continuing to waste a fully-simulated 708-circuit chip as a permanent holdout has diminishing research value once the question it was built to answer has been answered once. This entry graduates Osaka into training to build the strongest combined model to date, and sets up a fifth chip, Quebec (FakeQuebec, 127 qubits, 144 edges), as the new clean holdout -- preserving the project's ability to keep running this test honestly going forward, chip after chip.

Entry 076 — Quebec Holdout Dataset

Grew an eval-only bell-pair dataset for Quebec exactly as Entry 073 did for Osaka: same six hop-distance bins, same 4-seed x 4096-shot Aer/MPS simulation, same explicit exclusion from every training pipeline. Reached 710 circuits. This dataset is never folded into anything -- its entire value is staying untouched.

Entry 077 — Combined 4-Chip Model

Osaka's 708 training-eligible circuits (Entry 073's original growth) were folded into the graph dataset alongside Kyiv/Sherbrooke/Brisbane, taking the pool from 5,690 to 6,398 graphs. Retrained both the combined generalist (random 80/20 CV, same architecture and per-chip normalization as Entry 069) and the deployable MC-Dropout model (Entry 071's pattern, dropout active throughout, trained on all 6,398 graphs).

Chip	n (CV test)	MAE (pts)	R2	floor-collapse MAE	n_fc
Kyiv	421	1.08	0.981	2.04	23
Sherbrooke	404	1.29	0.961	1.43	43
Brisbane	306	1.14	0.966	0.29	7
Osaka	149	1.19	0.977	2.84	9
Overall	1280	1.17	0.972	--	--

Entry 077 combined-generalist held-out CV results (random 80/20 split, all 4 chips pooled). Compare to Entry 069's 3-chip generalist: overall MAE=1.07, Kyiv=0.77, Sherbrooke=1.45, Brisbane=0.99. Kyiv and Brisbane got slightly WORSE with a 4th, more diverse chip added to the pool; Sherbrooke improved. This is the expected nuanced pattern, not the uniform win across every chip that was hoped for going in -- pooling more diverse data does not universally help every individual chip's point accuracy.

Calibration check (separate held-out CV model, never the deployed one): MAE=1.36, R2=0.965, mean predicted std=0.89pts, corr(std,|error|)=0.414 -- positive and meaningful, slightly weaker than Entry 071's 0.553 but still confirms the uncertainty estimate is tracking real error, not noise.

Entry 078 — True Zero-Shot Test Against Quebec

The real payoff. The Entry 077 deployed model -- trained on Kyiv+Sherbrooke+Brisbane+Osaka, ZERO Quebec exposure -- was scored cold on all 710 Quebec circuits, using the average of the four training chips' normalization stats (Quebec has none, mirroring Entry 073's method exactly).

Model	MAE (pts)	R2	floor-collapse MAE	n_fc
GNN (Entry 077, 4-chip)	3.02	0.775	10.84	42
v4.1 closed-form	5.35	0.589	21.06	42

Zero-shot comparison on Quebec (n=710, mean predicted std=1.66pts).

This is a clear win on BOTH axes, not the tie Entry 073 found on Osaka. Entry 071's 3-chip model scored MAE=5.82 / R2=0.623 / floor-collapse MAE=15.77 zero-shot on Osaka, roughly matching v4.1's 5.57 / 0.686 /

22.38 on aggregate accuracy while still beating it on floor-collapse. The 4-chip Entry 077 model instead beats v4.1 outright on every number: 44% lower aggregate MAE, 32% higher R2, and less than half v4.1's floor-collapse error. Uncertainty widened from ~0.89pts (same-chip) to 1.66pts on the unseen chip -- proportionally less dramatic than Entry 073's ~4x widening on Osaka, consistent with the model's actual error also degrading less on this genuinely harder test.

Entries 076-078 — Conclusion

The user's hoped-for outcome -- one model that beats all four training chips at every single parameter -- did not hold: Kyiv and Brisbane's same-chip point accuracy regressed slightly with a 4th, more heterogeneous chip added to the pool. But the more important number moved the right way by a wide margin: true zero-shot generalization to a genuinely unseen 5th chip improved substantially over the 3-chip model's equivalent test, converting a rough tie against the closed-form baseline into a decisive win on both raw accuracy and the floor-collapse detection that has been this project's central use case since Entry 036. More diverse training data trades a little same-chip precision for meaningfully better out-of-distribution generalization -- the honest, unglamorous, and genuinely useful finding this round produced.

Research Log — Index Addendum (Entries 076-078)

Files: setup_quebec.py, entry076_grow_quebec.py (quebec_bell_dataset.json, 710 circuits, eval-only), entry077_build_graphs.py (entry077_graph_dataset.json, 6398 graphs), entry077_train_gnn.py, entry077_combined_train.py (entry077_combined_params.json), entry077_mc_dropout.py, entry077_deploy_train.py (entry077_deploy_params.json, the new deployed model), entry078_zero_shot_test.py (entry078_zero_shot_results.json).

ENTRY 079 Correcting the Kyiv/Brisbane "Regression" — September 16, 2026 — Status: COMPLETE

It Was a Split-Composition Artifact, Not a Real Capacity Effect

Entry 077's Reported Kyiv/Brisbane Dip Does Not Survive a Controlled Comparison

The Problem With Entry 077's Original Comparison

Entry 077 reported Kyiv MAE rising 0.77→1.08 and Brisbane 0.99→1.14 when Osaka joined training, read as the model trading a little same-chip precision for broader generalization. That comparison used two DIFFERENT random 80/20 KFold splits (seed=42 each time, but over datasets of different length — 5,690 graphs for Entry 069, 6,398 for Entry 077) — so the actual circuits landing in each "held-out test set" were not the same circuits. A difference in test-set composition alone can move per-chip MAE by this much; the comparison was confounded from the start.

Controlled Re-Test

Re-scored the Entry 077 combined model on the EXACT same 1,138 held-out circuits Entry 069 used (the first 5,690 rows of entry077_graph_dataset.json are in identical order to entry068_graph_dataset.json, confirmed by spot-check, so Entry 069's KFold test indices apply unchanged). This isolates the model-capacity question from split-composition noise.

Chip	Entry 069 (3-chip)	Entry 077 (4-chip), same test circuits	Delta
Kyiv	0.77	0.84	+0.07 (noise-level)
Sherbrooke	1.45	1.09	-0.36 (better)
Brisbane	0.99	0.80	-0.19 (better)
Overall	1.07	0.91	-0.16 (better)

MAE in points, on the identical held-out circuit set both times. Floor-collapse MAE on this same set: Kyiv=0.75, Sherbrooke=0.54, Brisbane=0.17 — all healthy, no regression there either.

Entry 079 — Conclusion

There is no real capacity-dilution effect here. When compared fairly — same circuits, same evaluation — the 4-chip model matches or beats the 3-chip model on every one of the original three chips, not just on generalization to the unseen fifth. Entry 077's "nuanced, not-uniform" framing was itself an artifact of an apples-to-oranges comparison. No architecture or weighting fix is warranted. The corrected, honest verdict: adding Osaka to training improved the model across the board — same-chip accuracy AND zero-shot generalization to a genuinely new chip — with no accuracy trade-off found on a controlled re-test.

Research Log — Index Addendum (Entry 079)

No new files — this entry is a controlled re-analysis of entry077_combined_params.json against Entry 069's original KFold test_idx, run directly against the existing dataset files.

ENTRIES 084-088 Does the Generalization Trend Hold at 5 Chips? — September 16-17, 2026 — Status: COMPLETE

A Third Data Point, and a Broken Fake-Backend Snapshot Along the Way

Quebec Graduates to Training, Cusco Becomes the New Holdout

Plan

Entry 077/078 found the jump from 3 to 4 training chips turned a rough tie against v4.1 (Osaka) into a clear win (Quebec). One data point is not a trend. This round repeats the cycle once more -- fold Quebec into training, stand up a 6th chip as the new holdout, retrain, and see whether the zero-shot advantage keeps growing or was a one-off.

Entry 084 — A Broken Fake-Backend Snapshot (Kyoto, Abandoned)

FakeKyoto was the first candidate for the new holdout chip. Its topology and coherence exported cleanly (127 qubits, 144 edges), but every one of its 144 two-qubit gate-error calibration values was exactly 1.0 (100% error) -- compare Kyiv/Sherbrooke/Brisbane/Osaka/Quebec, which all have realistic 0.003-0.02 typical ranges. Every single Kyoto bell-pair circuit floor-collapsed, even trivial adjacent-qubit pairs (hop=1 success rate ~47%, essentially random). v4.1's own MAE on this data was 31.5 points with R2=-805 -- not a model failure, a broken calibration snapshot in this Qiskit version. Abandoned before any training time was spent on it; FakeCusco (also 127 qubits/144 edges, realistic error range 0.005-1.0 with a healthy mean of 0.16) replaced it.

Entry 085 — Fold Quebec Into Training (5 Chips)

Quebec's 710 circuits (Entry 076) joined Kyiv/Sherbrooke/Brisbane/Osaka, taking the training pool to 7,108 graphs. Retrained the combined generalist and the deployable MC-Dropout model on all five. Held-out CV: overall MAE=1.18, R2=0.960. Per chip: Kyiv=0.86, Sherbrooke=1.19, Brisbane=1.09, Osaka=1.28, Quebec=2.14 (Quebec's own floor-collapse MAE=8.45, n_fc=14, visibly the hardest chip on its first exposure to training -- consistent with Entry 079's finding that a newly folded chip needs to be judged on the SAME held-out circuits across model versions before drawing conclusions, not taken as a standalone number).

Entry 086/087/088 — Cusco as the New Zero-Shot Holdout

Cusco's eval-only growth stalled well short of the usual ~700-circuit target -- throughput dropped from the typical 60-90 circuits/round to under 10/round with no single bin starving (hop-distance spread stayed reasonable throughout). Rather than burn further rounds chasing a stalled generator, stopped at 228 circuits: still large enough for a meaningful zero-shot read, smaller than Osaka's 708 or Quebec's 710. This is disclosed, not hidden.

Model	MAE (pts)	R2	floor-collapse MAE	n_fc
GNN (Entry 085, 5-chip)	4.01	0.784	3.79	63
v4.1 closed-form	9.32	0.364	17.50	63

Zero-shot comparison on Cusco (n=228, mean predicted std=1.17pts). Cusco's mean two-qubit gate error (16.3%) is noticeably higher than Quebec's (7.4%) or Osaka's (5.9%) -- it is simply a noisier chip, which is why both models' raw MAE are higher here than on Quebec. The relative comparison is the meaningful one.

The Trend Across Three Zero-Shot Tests

Training chips	Holdout	GNN MAE	v4.1 MAE	GNN/v4.1 ratio
3 (Kyiv/Sherbrooke/Brisbane)	Osaka (Entry 073)	5.82	5.57	1.04x (roughly tied)
4 (+ Osaka)	Quebec (Entry 078)	3.02	5.35	0.56x (44% better)
5 (+ Quebec)	Cusco (Entry 088)	4.01	9.32	0.43x (57% better)

MAE in points. The ratio column is the fair comparison across chips of different intrinsic difficulty: it answers "how much better is the GNN than the plain physics formula on a chip it has never seen", independent of how hard that particular chip happens to be.

Entries 084-088 — Conclusion

The trend holds, and holds cleanly: going from 3 to 4 to 5 training chips has monotonically widened the GNN's advantage over the closed-form baseline on chips it has never trained on -- tied, then 44% better, then 57% better. This is now three independent zero-shot tests pointing the same direction, not a single lucky result. The honest caveat: Cusco's holdout sample (228) is smaller than the other two due to a generation throughput stall, so its exact numbers carry more sampling noise than Osaka's or Quebec's -- but the direction of the result (another clear win, wider than Quebec's) is consistent with the trend rather than an outlier reversing it.

Research Log — Index Addendum (Entries 084-088)

Files: setup_kyoto.py (abandoned, broken calibration), setup_cusco.py, entry087_grow_cusco.py (entry087_cusco_bell_dataset.json, 228 circuits, eval-only), entry085_build_graphs.py (entry085_graph_dataset.json, 7108 graphs), entry085_train_gnn.py, entry085_combined_train.py (entry085_combined_params.json), entry085_mc_dropout.py, entry085_deploy_train.py (entry085_deploy_params.json, the new deployed model), entry088_zero_shot_test.py (entry088_zero_shot_results.json).

ENTRIES 089-091 — First Step Past "One Gate That Matters" — September 22, 2026 — Status: FIRST PASS COMPLETE

Parallel Multi-Bell-Pair Circuits: the First Template With More Than One Thing That Matters

A Deliberate First Step Toward Arbitrary-Circuit Support, Not a Leap to It

Why This Template

Every template so far (bell/GHZ/star/chain) has exactly one entangling event that determines success, which is what made cheap ground truth and simple feature engineering possible. Arbitrary circuits break both assumptions at once. Parallel multi-bell-pair circuits -- N independent bell pairs on disjoint qubits, run simultaneously -- are the smallest step past "one thing matters" while keeping ground truth cheap (still just correlation-checking, now joint across N pairs) and success unambiguous (all N pairs correlated).

Entry 089 — Sanity Check: Is There Even a Signal?

Before investing in feature-schema work, checked whether parallel pairs behave differently from independent single pairs at all. Compared real joint success (Aer ground truth) against the product of each pair's own isolated v4.1 prediction -- what you'd expect under zero crosstalk. Across 20 test circuits (N=2 or 3 pairs, Kyiv): mean gap 8.2 points, max 15.6, min 2.5, in BOTH directions (joint success sometimes higher, sometimes lower than independence predicts). Real, substantial, non-trivial signal -- confirmed the template is worth building for.

A throughput problem surfaced during growth: simulation cost does not stay flat with hop distance the way single-pair circuits do -- a single hop~20 multi-pair circuit took 36 seconds vs under a second at hop~1, since N simultaneous entangled clusters push MPS simulation cost up sharply. Capped hop distance at 8 (vs 26 for single-pair templates) to keep growth throughput practical; grew 310 circuits (N=2/3, Kyiv only) for this first pass.

Entry 090 — Feature Schema Extension

Extended the graph builder with two new dimensions no prior template needed: a 7th node feature (how many DIFFERENT pairs' SWAP routing ever touched this physical qubit -- a direct proxy for the shared-resource channel crosstalk travels through), and a 5th edge feature (which of the N pairs this gate belongs to). The existing is_final logic (real entangling gate vs SWAP) already generalized correctly to multiple non-swap gates without modification. Confirmed on the built dataset: 190 of 4,975 total nodes were touched by more than one pair's routing -- the exact physical-sharing signal this schema exists to represent.

Entry 091 — First Model, First Result

Model	MAE (pts)	R2
GNN (Entry 091, first pass)	6.21	0.752
product-of-marginals (no-crosstalk baseline)	11.56	0.188

Single 80/20 split, n=310 (Kyiv only, N=2/3 pairs, hop≤8). 46.3% lower MAE than the naive independence assumption.

Entries 089-091 — Conclusion

This is a first-pass proof of concept, playing the same role Entry 037 played for single bell-pairs: small dataset, one chip, not yet production quality -- but it answers the question this round of work existed to answer. Parallel multi-bell circuits carry real, learnable structure that a naive independence assumption misses by double digits, and the feature-schema extension needed to capture it (shared-qubit tracking, per-pair gate tagging) works as designed. The next step, when resumed, is the obvious one: grow the dataset properly, extend across chips, and run this template through the same fold-in/holdout/zero-shot discipline used for chip generalization -- but the open question of whether this direction was worth pursuing at all is now answered.

Research Log — Index Addendum (Entries 089-091)

Files: entry089_grow_multibell.py (entry089_multibell_sanity.json, 310 circuits), entry090_build_multibell_graphs.py (entry090_multibell_graph_dataset.json), entry091_train_multibell.py (entry091_multibell_params.json, entry091_multibell_results.json).

ENTRIES 092-093 Beyond a Single Number — September 25, 2026 — Status: COMPLETE

Failure-Mode Breakdown, Weak-Link Identification, Chip Recommender

Three New Prediction Features, Built on Existing Validated Physics — No Retraining Needed

Why This Was Cheap

exact_dwell_cost (Entries 024-032) already computes success as a product of independent survival terms -- final-gate error, decoherence accumulated during SWAP dwell time, single-qubit gate cost, readout error -- it just never returned them separately, only the combined number. entry092_decomposed_prediction.py duplicates the SAME validated logic (same bug fixes, same closed-form derivations) but tracks each channel independently instead of folding them into one accumulator. Checked against the original exact_dwell_cost on 8 test pairs across two chips: every decomposed total matched the original to floating-point precision. This is exposition of physics already in the codebase, not a new model with new failure modes of its own to validate.

Entry 092a — Failure-Mode Breakdown

Given a circuit, reports not just "73% success" but which channel is costing the most: gate_error_survival, decoherence_survival, sq_gate_survival, readout_survival, ranked by loss. Example (Kyiv, qubits 77-82): total 60.4%, dominant failure mode decoherence (31.1 points of loss) vs gate error (0.6 points) -- on a long route, decoherence during the SWAP chain overwhelmingly outweighs the entangling gate's own error, exactly the kind of actionable distinction a single aggregate number can't offer.

Entry 092b — Weak-Link Identification

The per-SWAP-leg dwell log the original formula already builds (physical qubits, duration, raw gate error, dephasing) directly identifies the single worst leg by combined gate-error + dephasing score. For the same 77-82 example: weak link is edge (80,79), consistent with the decoherence-dominant verdict above. No new ground truth needed -- this was always computable from data the formula already touches, just never surfaced.

Entry 093 — Best Chip/Route Recommender

Turns the model from "grade this circuit" into "tell me where to run it." Built entirely on the existing 5-chip precomputed lookup (entry081) -- a ranking/filtering layer, not a new model. recommend(min_success, max_hop, max_uncertainty, chips, top_k) returns ranked candidates; the top result gets a live failure-mode breakdown attached automatically. Verified against three realistic queries: best 5 pairs with no filters (returned a mix of Kyiv and zero-shot Quebec candidates near 98%), best trained-chip pair under hop≤5 with explanation, and a zero-shot-only query ("chip I haven't tried yet") -- all returned sane, ranked results.

Shipped

Both new capabilities are live in the API, not just standalone scripts: POST /predict/breakdown and GET /recommend, tested end-to-end against a running server (both returned correct 200 responses with the expected structure). README updated with usage examples.

Entries 092-093 — Conclusion

All three requested prediction features (failure-mode breakdown, weak-link ID, chip recommender) are built, validated, and wired into the live API in one pass -- made possible because the physics-informed v4.1 formula already carried this information internally from Entries 024-032, it just needed to be exposed rather than re-derived. No new training data or model retraining was required for any of the three. Next up per the agreed plan: multi-bell-pair template scaling (Entries 089-091's track).

Research Log — Index Addendum (Entries 092-093)

Files: entry092_decomposed_prediction.py, entry093_recommend.py, api/main.py (/predict/breakdown, /recommend), api/README.md.

ENTRY 094 Multi-Bell Model Scales to 3 Chips — September 25, 2026 — Status: COMPLETE

From 310 Kyiv-Only Circuits to a 650-Circuit, 3-Chip Combined Model

Per-Chip Normalization Fix (Entry 048) Applied to the Multi-Bell Template

Dataset Growth

Sherbrooke and Brisbane multi-bell-pair circuits were grown alongside Entry 089's original 310 Kyiv-only records, using the same RNG-seeding and signature-dedup fixes: Sherbrooke to 220, Brisbane to 120 (both below the initial 300-500 target -- growth throughput softened on both chips the same way it did for Cusco in Entry 087, and the same honest-disclosure call applies: these are usable sample sizes, not the originally hoped-for ones). Combined dataset: 650 multi-bell circuits across 3 chips.

Fold-In Result (Standard 5-Fold CV, All 3 Chips Seen in Training)

Chip	n (test fold)	GNN MAE	Product-of-marginals MAE
Kyiv	65	3.60 pts	7.96 pts
Sherbrooke	46	6.49 pts	13.51 pts
Brisbane	19	5.19 pts	9.86 pts
Overall	130	4.86 pts	10.20 pts

Overall: 52.4% lower MAE than the naive-independence baseline, up from Entry 091's Kyiv-only 46.3% -- the crosstalk signal is holding, and if anything strengthening, once the model sees chip-to-chip variation in how much the parallel pairs interact. No chip's fold-in error is wildly out of line with the others, unlike the false Kyiv/Brisbane alarm the single-pair pipeline hit in Entry 077 (retracted in Entry 079).

Entry 094 — Conclusion

The multi-bell template generalizes past a single chip: per-chip normalization, carried over unchanged from the single-pair pipeline's Entry 048 fix, was sufficient to combine 3 chips into one model without a chip-scale artifact dominating the loss. Fold-in performance improved (52.4% vs 46.3%) rather than degrading with the added chip diversity.

ENTRY 095 Multi-Bell Cross-Chip Zero-Shot — September 25, 2026 — Status: COMPLETE (mixed result, disclosed honestly)

Leave-One-Chip-Out Zero-Shot: 2 of 3 Folds Generalize, 1 Does Not

A Real Negative Result — Reported As-Is, Not Smoothed Over

Method

Mirrors the single-pair pipeline's zero-shot discipline (Entries 073/078/088): train on N-1 chips, predict cold on the held-out Nth, never touched during training. With only 3 chips in the multi-bell dataset so far, all three leave-one-chip-out folds were run rather than reserving a single permanent holdout chip -- with a dataset this small, permanently cutting a third of it for one dedicated holdout isn't worth the rigor it would buy. The held-out chip's features are normalized using the AVERAGE of the two training chips' per-chip stats (the same fallback used for Quebec in Entry 078, since the held-out chip's own stats are unavailable by construction).

Results

Holdout chip	Trained on	n (test)	GNN MAE	Baseline MAE	GNN vs baseline
Kyiv	Sherbrooke, Brisbane	310	7.15 pts	10.23 pts	30.1% better
Brisbane	Kyiv, Sherbrooke	120	8.29 pts	10.83 pts	23.5% better
Sherbrooke	Kyiv, Brisbane	220	20.72 pts	12.67 pts	63.6% WORSE

Sherbrooke as the held-out chip is a clear regression: R2 = -0.515 (worse than predicting the mean), versus the naive product-of-marginals baseline's R2 = 0.248 on the same test set. Kyiv and Brisbane holdouts both show a real, meaningful zero-shot advantage over the baseline, consistent in direction (though not yet magnitude) with the single-pair template's zero-shot trend.

Why Sherbrooke Likely Fails Here

The leading candidate explanation is data scale, not a Sherbrooke-specific data problem: each zero-shot fold here trains on only 2 chips totalling 430-530 circuits -- an order of magnitude less than the single-pair pipeline had (6,398-7,108 circuits) by the time its own zero-shot trend became reliable across 3-5 chips (Entries 073-088). The single-pair pipeline's early zero-shot attempts, at comparably thin per-chip data, were also inconsistent before the dataset scaled up -- this isn't being treated as evidence the multi-bell template itself is unlearnable cross-chip, but as evidence the current 650-circuit, 3-chip dataset is below the scale where zero-shot reliably works, the same conclusion the single-pair track reached at a similar early stage. No fix is being force-applied here (e.g. discarding Sherbrooke, cherry-picking a flattering holdout) -- the negative result stands as logged.

Entry 095 — Conclusion

Cross-chip zero-shot generalization for the multi-bell template is NOT yet reliable: 2 of 3 leave-one-chip-out folds show a real advantage over the naive baseline, but the third (Sherbrooke) shows a clear regression. This is reported honestly as a mixed, inconclusive result rather than rounded up to a clean win -- the most likely cause is the dataset still being roughly 10x smaller per chip than what made the single-pair pipeline's zero-shot trend trustworthy. The fold-in result (Entry 094, all chips seen) remains solid; it is specifically the COLD zero-shot case that needs more data before it can be trusted. Next step: continue growing Sherbrooke and Brisbane toward the original 300-500/chip target, then re-run this exact leave-one-chip-out test to see whether the Sherbrooke fold improves with more training data, before drawing any firmer conclusion about the template's cross-chip generalization.

Research Log — Index Addendum (Entries 094-095)

Files: entry094_train_multibell.py, entry095_zero_shot_multibell.py, quantumbridge_data/entry094_multibell_{params,results}.json, quantumbridge_data/entry095_zero_shot_multibell_results.json.

ENTRY 096 Multi-Seed Variance Check — Sherbrooke's Zero-Shot Gap Is Real — October 1, 2026 — Status: COMPLETE

More Data Didn't Fix It — And Multi-Seed Testing Shows Why That Matters

Dataset Grown 650 → 985 Circuits; Zero-Shot Re-Tested; Then Checked for Seed Noise

Step 1 — Grow the Dataset

Following the plan from Entry 095, Sherbrooke and Brisbane were grown further: Sherbrooke 220 -> 380, Brisbane 120 -> 295 (Kyiv held at 310, already near its original target). Combined multi-bell dataset: 985 circuits, up from 650. Graphs rebuilt (entry090) and the fold-in model retrained (entry094): 53.9% lower MAE than the naive-independence baseline, consistent with the 650-circuit result (52.4%) -- fold-in performance held steady at the larger scale.

Step 2 — Re-run the Zero-Shot Test (Single Seed)

Re-running Entry 095's leave-one-chip-out test at seed=0 gave a mixed and initially confusing picture: Kyiv holdout improved slightly (30.1% -> 34.5% better than baseline), Sherbrooke improved substantially (-63.6% -> -15.4%, still worse but far less bad), but Brisbane holdout FLIPPED from +23.5% (650 circuits) to -10.1% (985 circuits) -- the opposite of the hoped-for trend. This result alone would have been reported as "partially fixed, with a new regression on Brisbane," which turned out to be the wrong read.

Step 3 — Multi-Seed Check: Is This Real or Just Noise?

Before writing up the Brisbane flip as a finding, each of the 3 holdout folds was re-run at seeds 1 and 2 (3 seeds total per chip, same train/test split, only the model's random initialization and minibatch order changed) to see whether these numbers are stable.

Holdout chip	seed=0	seed=1	seed=2	mean	range
Kyiv	+34.5%	+45.2%	+41.6%	+40.4%	[+34.5%, +45.2%]
Brisbane	-10.1%	+33.9%	+30.2%	+18.0%	[-10.1%, +33.9%]
Sherbrooke	-15.4%	-118.5%	-70.0%	-68.0%	[-118.5%, -15.4%]

(GNN MAE vs. product-of-marginals baseline MAE, percent better; negative = GNN worse than the naive baseline.) The Brisbane "regression" seen at seed=0 was a single noisy run: 2 of its 3 seeds are strongly positive (+33.9%, +30.2%), and its mean across seeds (+18.0%) is solidly positive. Kyiv is stable and strongly positive at every seed tested. Sherbrooke is the opposite case: NEGATIVE at all 3 seeds, including two runs far worse than seed=0 alone suggested (-118.5%, -70.0%). This rules out training-seed noise as the explanation for Sherbrooke -- it's a stable, reproducible generalization failure, not a fluke.

Why Sherbrooke Specifically? A Partial Explanation

Checking each chip's raw per-node calibration means (T1, T2, readout error, T1/T2 ratio) against the average of the OTHER two chips (the fallback normalization zero-shot uses, since the held-out chip's own stats are unavailable by construction) shows Sherbrooke has the largest mismatch on 3 of 4 features: T2 (22.0% off the kyiv+brisbane average), readout error (43.5% off), and T1/T2 ratio (42.4% off) -- notably higher than Kyiv's largest mismatches (11.6-20.6%). Brisbane's single largest mismatch (T1/T2 ratio, 41.7%) is comparable in size to Sherbrooke's worst case, which is why normalization mismatch is only a PARTIAL explanation, not a complete one -- it doesn't cleanly predict that Brisbane would still generalize well while Sherbrooke wouldn't. The full mechanism remains an open question, logged honestly as such rather than overclaiming a tidy explanation that the data don't fully support.

Entry 096 — Conclusion

Growing the dataset from 650 to 985 circuits did NOT close Sherbrooke's zero-shot gap -- multi-seed testing confirms this is a real, reproducible chip-specific generalization failure (mean -68.0% worse than the naive baseline across 3 seeds), not a data-scale or single-seed artifact. Kyiv and Brisbane, by contrast, both generalize well and reliably once seed noise is accounted for (means +40.4% and +18.0%). Practical takeaway for any live-facing use of this template: Sherbrooke should NOT be treated as a reliable zero-shot target for the multi-bell model given current evidence, and this should be disclosed rather than smoothed over if the template is ever wired into a public demo or API, mirroring how Quebec and Cusco's zero-shot status is disclosed for the single-pair pipeline. This is logged as an open research question, not a solved problem -- a plausible next step, if pursued, would be a larger multi-seed ensemble (average predictions across several seeds) to see whether it meaningfully narrows Sherbrooke's gap, but that has NOT been tried yet and is not assumed to work.

Research Log — Index Addendum (Entry 096)

Files: entry096_seed_variance_multibell.py, quantumbridge_data/entry096_seed_variance_results.json. Builds on entry089_grow_multibell.py (regrowth), entry090_build_multibell_graphs.py (rebuild), entry094_train_multibell.py (retrain), entry095_zero_shot_multibell.py (zero-shot harness, reused for all seed runs here).

ENTRY 097 API/Demo Model Version Lag, Fixed — October 2, 2026 — Status: COMPLETE

Quebec Stopped Being a Zero-Shot Holdout in Entry 085 — The Live API Didn't Know

A Honesty/Consistency Bug, Not a Model Bug

The Problem

Entries 108-112 built and validated the 5-chip entry085 model (kyiv/sherbrooke/brisbane/osaka/quebec, trained together) and found it beat the plain physics formula on Cusco -- the project's new zero-shot holdout -- by a growing margin as chip count increased. But Entry 112's demo/API wiring work ran before Entry 085 existed in its final form, and api/main.py's MODEL_PATH was never bumped: it stayed on entry077 (the 4-chip model) indefinitely. Worse, /predict kept refusing Quebec with a message calling it 'this project's current zero-shot holdout chip' -- true in Entries 076-078, false since Entry 085 folded Quebec into training. This was flagged as a known, unaddressed inconsistency at the end of the Entries 094-095 work and left for this session.

The Fix

Component	Before	After
api/main.py MODEL_PATH	entry077 (4-chip)	entry085 (5-chip, trains on Quebec)
Quebec via /predict	Refused (stale zero-shot message)	Live prediction, same as any trained chip
Cusco via /predict	N/A (not referenced)	Refused -- new zero-shot holdout (Entries 087-088)
/recommend lookup table	entry081 (4-chip + Quebec zero-shot fallback)	entry097_lookup_5chip.json (all 5 scored normally)
entry093 TRAINED_CHIPS	4 chips, Quebec flagged is_zero_shot	5 chips, no chip flagged zero-shot
Live demo LUT	Old 4-chip + Quebec-fallback scores	Rescored with entry085, all 5 chips consistent
Live demo Quebec badge	Permanent 'zero-shot / never trained'	Removed -- Quebec treated like other trained chips

The lookup table powering /recommend and the live demo needed a full rescore, not just a pointer change: Entry 081's table was built with the 4-chip model, and Quebec's rows in it came from Entry 083's zero-shot fallback method (averaged normalization stats from the 4 training chips, since Quebec had no real stats of its own at the time). entry097_rescore_5chip.py reuses Entry 081's exact scoring logic against entry085 instead, scoring Quebec the same way as every other chip now that it has genuine chip-specific normalization stats from being trained on directly. All 40,005 pairs (5 chips x 8,001 pairs) rescored in one pass.

Verification

Ran the API locally end-to-end: /health reports 5 supported chips and the entry085 model name; /predict on Quebec now returns a live prediction (verified qubits 0-10: v4.1=0.8224, GNN=0.8407); /predict on Cusco correctly refuses with the new zero-shot message; /predict on Kyiv still works as a sanity check; /recommend returns ranked candidates with is_zero_shot=false across all 5 chips; /predict/breakdown works on Quebec. The live demo HTML's embedded script was re-validated with node --check after the edits (caught and fixed one unescaped-apostrophe typo introduced during the text swap) and its LUT was spot-checked to confirm Quebec's new scores loaded correctly.

Entry 097 — Conclusion

The API, /recommend lookup table, and live demo now all agree with each other and with the project's actual current model: 5 trained chips (including Quebec), Cusco as the one true zero-shot holdout. This closes a real inconsistency that had been sitting unaddressed since Entry 112 -- not a model-quality issue, but exactly the kind of documentation/deployment drift that undermines the project's honesty-first approach to presenting results if left

unfixed.

Research Log — Index Addendum (Entry 097)

Files: entry097_rescore_5chip.py, quantumbridge_data/entry097_lookup_5chip.json, api/main.py (MODEL_PATH, chip lists, refusal messages), api/README.md, entry093_recommend.py (LOOKUP_PATH, TRAINED_CHIPS), quantumbridge_live_demo.html (LUT data, Quebec UI logic).

ENTRY 098 Dataset Grown to 1,590 — Sherbrooke Gap Confirmed Again — October 3, 2026 — Status: COMPLETE

985 -> 1,590 Circuits (+61%): Kyiv/Brisbane Improve, Sherbrooke Doesn't Move

A Second, Larger-Scale Confirmation of Entry 096's Finding

Growth

Continued growing all three multi-bell chips from Entry 096's 985-circuit dataset: Kyiv 310->555, Sherbrooke 380->525, Brisbane 295->510. Combined dataset: 1,590 circuits, a 61% increase, and more than double the original 650-circuit first pass. Graphs rebuilt, fold-in model retrained: 60.1% lower MAE than the naive-independence baseline, up from 53.9% at the 985-circuit scale and 52.4% at 650 -- fold-in quality keeps improving smoothly with scale, as expected.

Zero-Shot Re-Test at the New Scale

Holdout chip	985 circuits	1,590 circuits	Direction
Kyiv (mean/seed0)	+40.4% (3-seed mean)	+43.2% (seed 0)	Improved, stayed strongly positive
Brisbane (mean/seed0)	+18.0% (3-seed mean)	+32.3% (seed 0)	Improved, stayed positive
Sherbrooke (3-seed mean)	-68.0% (range -118.5% to -15.4%)	-61.0% (range -123.7% to -15.1%)	No meaningful change

Sherbrooke's 3 seeds at the new scale: -44.1%, -123.7%, -15.1% -- all still negative, same wide spread, same rough mean as before (-61.0% vs -68.0%). A 61% increase in training data moved Sherbrooke's zero-shot result by less than 10 percentage points on the mean, well within the seed-to-seed noise band already established in Entry 096. This is the clearest evidence yet that Sherbrooke's zero-shot failure is NOT primarily a data-scarcity problem -- if it were, this scale increase should have shown visible improvement the way Kyiv and Brisbane's results did.

Entry 098 — Conclusion

Growing the dataset further did what it was supposed to do for Kyiv and Brisbane (both generalization results improved) but did not meaningfully change Sherbrooke's outcome, reinforcing Entry 096's conclusion with a second, independent, larger-scale data point rather than just repeating the same test. This makes a pure "needs more data" explanation for Sherbrooke's gap less likely, and points back toward Entry 096's partial structural explanation (Sherbrooke's calibration profile -- T2, readout error, T1/T2 ratio -- differing more from the other two chips' average than Kyiv's or Brisbane's does) as the more probable direction, though that explanation remains partial and unconfirmed, not a closed case. Given the user's call to prioritize this research track while real-hardware access is pending, growing the dataset further is not expected to resolve this on its own -- if this gets revisited, a more targeted investigation (e.g. deliberately probing what about Sherbrooke's calibration the model struggles to extrapolate to) would likely be more productive than further brute-force data growth.

Research Log — Index Addendum (Entry 098)

Files: quantumbridge_data/entry089_multibell_sanity.json (1,590 circuits), quantumbridge_data/entry090_multibell_graph_dataset.json (rebuilt), quantumbridge_data/entry094_multibell_{params,results}.json (retrained), quantumbridge_data/entry098_seed_variance_1590_results.json.

ENTRIES 099-101 Sherbrooke Explained, Fixed, and Confirmed on 5 Chips — October 10, 2026 — Status: COMPLETE

The Sherbrooke Gap Was a Normalization Artifact, Not a Data or Chip Problem

Diagnosis, Own-Stats Normalization, Ensembling, and Extension to Osaka/Quebec

Entry 099 — Diagnosis

Per-chip anatomy first: the product-of-marginals baseline underestimates real joint success by +5.1 pts on Kyiv and +7.6 on Brisbane but overestimates by 1.8 on Sherbrooke, whose baseline is also the weakest correlate of truth (r=0.51). That alone does not explain the failure: a constant bias correction does not rescue Sherbrooke (oracle own-bias MAE 13.0 vs baseline 12.6), and the GNN's -44% was far worse than any simple correction. The actual culprit was the held-out chip's feature normalization. Entries 078/095 normalize an unseen chip with the AVERAGE of the training chips' statistics; Sherbrooke's calibration (T2, readout error, T1/T2 ratio) sits 22-44% away from the kyiv+brisbane average, so the model saw systematically shifted inputs. Normalizing the held-out chip with its OWN feature statistics (no labels used) removes the shift.

Entry 100 — Own-Stats Normalization and Ensembling (3 chips, 10 seeds each)

Holdout	Avg-stats: mean (pos seeds)	Own-stats: mean (pos seeds)	Own-stats 10-seed ensemble
Kyiv	+41.1% (10/10)	+45.6% (10/10)	+58.4%
Sherbrooke	-34.5% (3/10)	+18.6% (10/10)	+23.9%
Brisbane	+33.1% (9/10)	+17.3% (10/10)	+24.9%

Own-stats flips Sherbrooke from mostly negative to positive on every seed, improves Kyiv, and makes Brisbane worse on the mean but far more stable (min +10.5% vs -10.6%). Averaging predictions across seeds helps Kyiv (+13 pts) and Brisbane (+8) but is inconsistent for Sherbrooke at 3-5 seeds (it only pays off at 10), so ensembling is a modest, not decisive, lever. A mechanical finding along the way: the training loop was not JIT-compiled; adding jax.jit cut a fold from ~150 s to ~5 s, which is what made 10-seed statistics practical (numerics differ slightly from the earlier non-JIT runs).

Entry 101 — Extension to Osaka and Quebec (5 chips, 2,080 circuits)

Holdout (trained on other 4)	Own-stats mean over 10 seeds	Positive seeds
Kyiv	+53.0%	10/10
Quebec	+36.2%	10/10
Sherbrooke	+28.0%	10/10
Osaka	+20.5%	10/10
Brisbane	+18.6%	10/10

With 4 training chips every holdout beats the naive-independence baseline on all 10 seeds, and Sherbrooke improves further (+18.6% with 2 training chips to +28.0% with 4) -- it is not an outlier chip, and the trend of generalization improving with training-chip count seen in the single-pair pipeline reappears here. Osaka (265 circuits) and Quebec (225) are smaller than the first three chips (510-555), disclosed as a caveat.

Entries 099-101 — Conclusion and Caveat

Entries 096/098 concluded the Sherbrooke failure was a stable chip-specific limitation and that more data would not help; that data-scale conclusion stands, but the implied "chip is intrinsically harder" reading was wrong -- the cause was the averaged-stats fallback. Important caveat on deployment: own-stats normalization computes statistics from the new chip's own circuit features (no ground-truth labels), so a new chip needs a modest unlabeled sample of circuits transpiled on it to compute them; that is cheap compared with labeled data but is not the same as truly cold, zero-information inference as in the Entry 078 averaged-stats setting. All numbers are fake-backend + Aer, as always.

Research Log — Index Addendum (Entries 099-101)

Files: entry099_zero_shot_ownstats.py, entry100_ensemble_zero_shot.py, entry100_run.py, entry100_ensemble_eval.py, entry101_zero_shot_5chip.py, entry101_run.py; data: entry100_10seed_results.json, entry100_ensemble_results.json, entry101_5chip_10seed_results.json; entry089/090 extended to Osaka and Quebec (2,080 circuits).

ENTRIES 102-104 Mixed Bell/GHZ3 Template Pilot — November 10, 2026 — Status: PILOT (small sample)

A Second Circuit Template: Parallel Components of Mixed Type

221 Circuits, 3 Chips; Pilot Result, Not a Conclusion

Entry 102-103 — Template and Features

Each circuit runs 2-3 independent components on disjoint qubits simultaneously; a component is a bell pair (H, CX) or a 3-qubit GHZ (H, CX a-b, CX a-c). Success = every component fully correlated in the same shot (real Aer MPS simulation, 8,192 shots; hop cap 4 per entangling leg for simulation cost). Baseline = product of independent per-component v4.1 predictions (bell and GHZ both use existing exact_dwell_cost paths). The graph builder reuses the multi-bell 7-node/5-edge schema, with component index in place of pair index, so multi-bell-trained models apply directly. Data: Kyiv 74, Sherbrooke 75, Brisbane 72 = 221 circuits (far fewer than the multi-bell sets; throughput collapsed in the sandbox and was not pushed further). The baseline has MAE 10.3 pts and correlates 0.76 with ground truth: a real, learnable gap exists.

Entry 104 — Results (own-stats normalization, GNN vs independent-components baseline)

Test	Result	Notes
A. Fold-in (mixed data, all 3 chips)	+19.8% mean (5 seeds: +12.5 to +29.1)	MAE 8.2 vs baseline 10.2
B. Leave-one-chip-out, Kyiv held out	+5.5% mean (4/5 seeds positive)	trained on 2 chips (~150 circuits)
B. Sherbrooke held out	-7.0% mean (1/3 positive)	3 seeds only
B. Brisbane held out	-4.3% mean (1/3 positive)	3 seeds only
C. Template transfer: bell-only model -> all mixed circuits	+16.0% mean (3/3 positive: +18.9, +20.2, +8.7)	no GHZ component ever seen in training

Cross-chip generalization trained on the mixed pilot data alone is weak/inconclusive, which is expected with ~150 training circuits (the multi-bell track needed >1,000 before holdouts were reliable). The notable result is C: a model trained only on bell-pair multi-bell data, applied to mixed circuits containing GHZ components it never saw, still beats the independence baseline by 16% on every seed -- nearly the fold-in level of a model trained on mixed data.

Entries 102-104 — Conclusion and Caveats

Early evidence that the multi-bell model's learned crosstalk/joint-effect correction carries over to a new component type. Caveats: (1) transfer test chips are the same 3 chips seen in training (template is unseen, chips are not); (2) the model receives the independent-product baseline as an input feature, so part of the transfer may be baseline recalibration rather than new physics, because v4.1 already handles GHZ; (3) 3-5 seeds and 221 circuits only; (4) all simulator-based. Treat as a pilot motivating a larger mixed dataset, not as a validated capability; it is not exposed in the API.

Research Log — Index Addendum (Entries 102-104)

Files: entry102_grow_mixed.py, entry102_seq.py, entry103_build_mixed_graphs.py, entry104_mixed_eval.py; data: entry102_mixed.json, entry103_mixed_graph_dataset.json, entry104_*_results.json.*